

Lecture 4

UML, SDS Features, STS Features, Lab 2

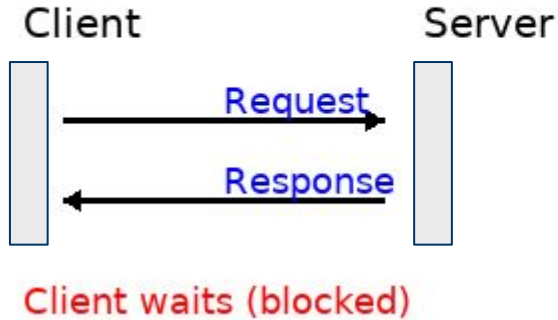
Suthep “Jogie” Madarasmi, Ph.D.

Weeks Taught by Aj. Jogie

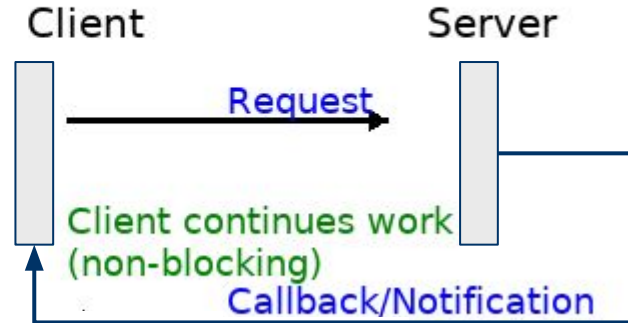
Date	Topics	Activities
W3. 18 Aug	Specification, Architecture, Tech Stack, Data Modeling, and Lightweight UML	Lab 2-1. Ticket Creation: engineering contract, Spec DD, Test DD, DB increment & seed, REST API contract
W4. 25 Aug	Test Design, TDD, Verification, Automated Testing, and Definition of Done	Lab 2-2. Create Ticket / My Tickets / Ticket Detail screens, attachments, Zen Green theme UI, full test suite (unit/API/UI/E2E)
W10. 13 Oct	Midterm Review and Individual Practical Integration. (Holiday)	Lab: Individual Practical Challenge & Demo (midterm integration); begin Team Sprint 1: working vertical slice
14–28 Oct	Midterm Exam (covers Weeks 1–7)	
W11. 3 Nov	Cloud Deployment and Delivery to Users	Lab: Cloud Deployment Lab. Deploy the team application to the cloud (GCP) and deliver to users; Team Sprint 2
W13. 17 Nov	Metrics, Reviews, Retrospectives, Ethics, Responsible AI Use, and AI Usage Report	Lab: Metrics, code review & retrospective; Responsible-AI reflection and AI Usage Report; final project hardening
1–11 Dec	Final Exam (covers Weeks 8–14)	After Finals: Project Demo.

Synchronous vs. Asynchronous APIs

Synchronous API



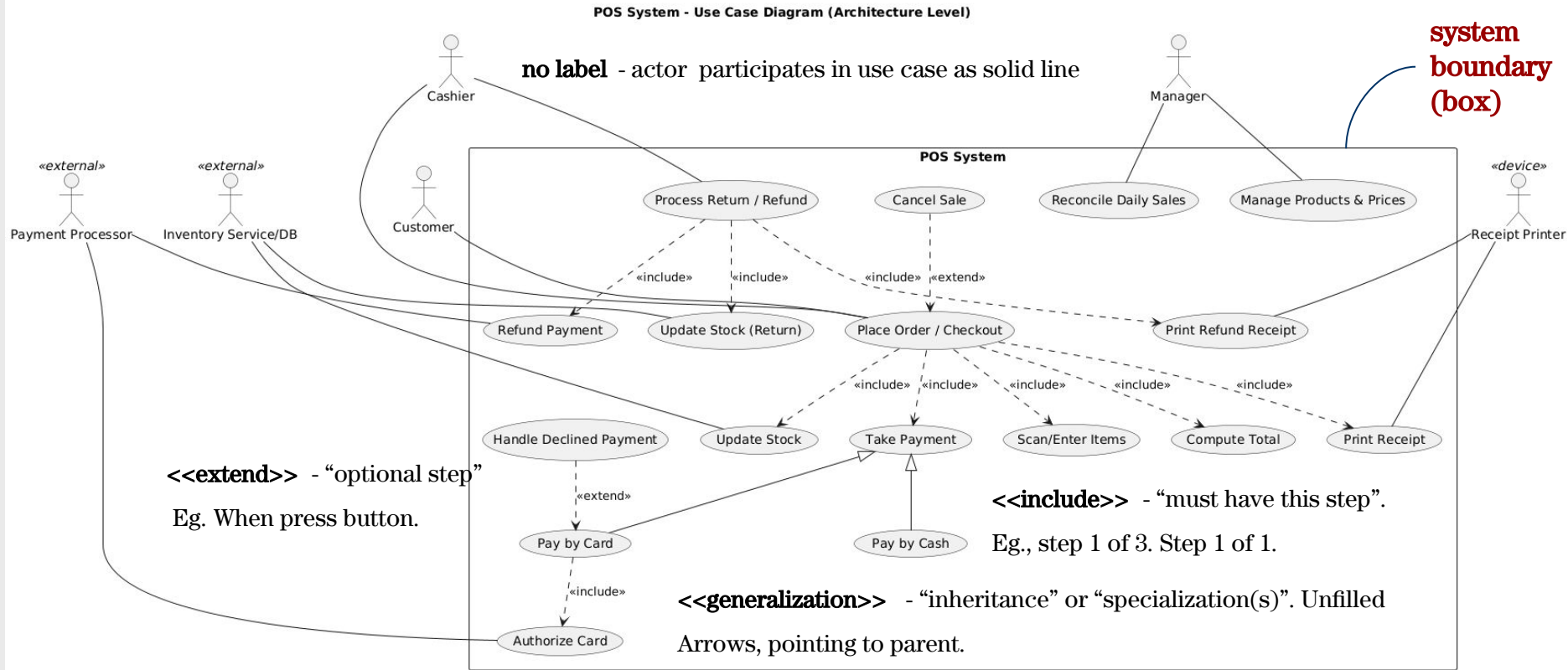
Asynchronous API



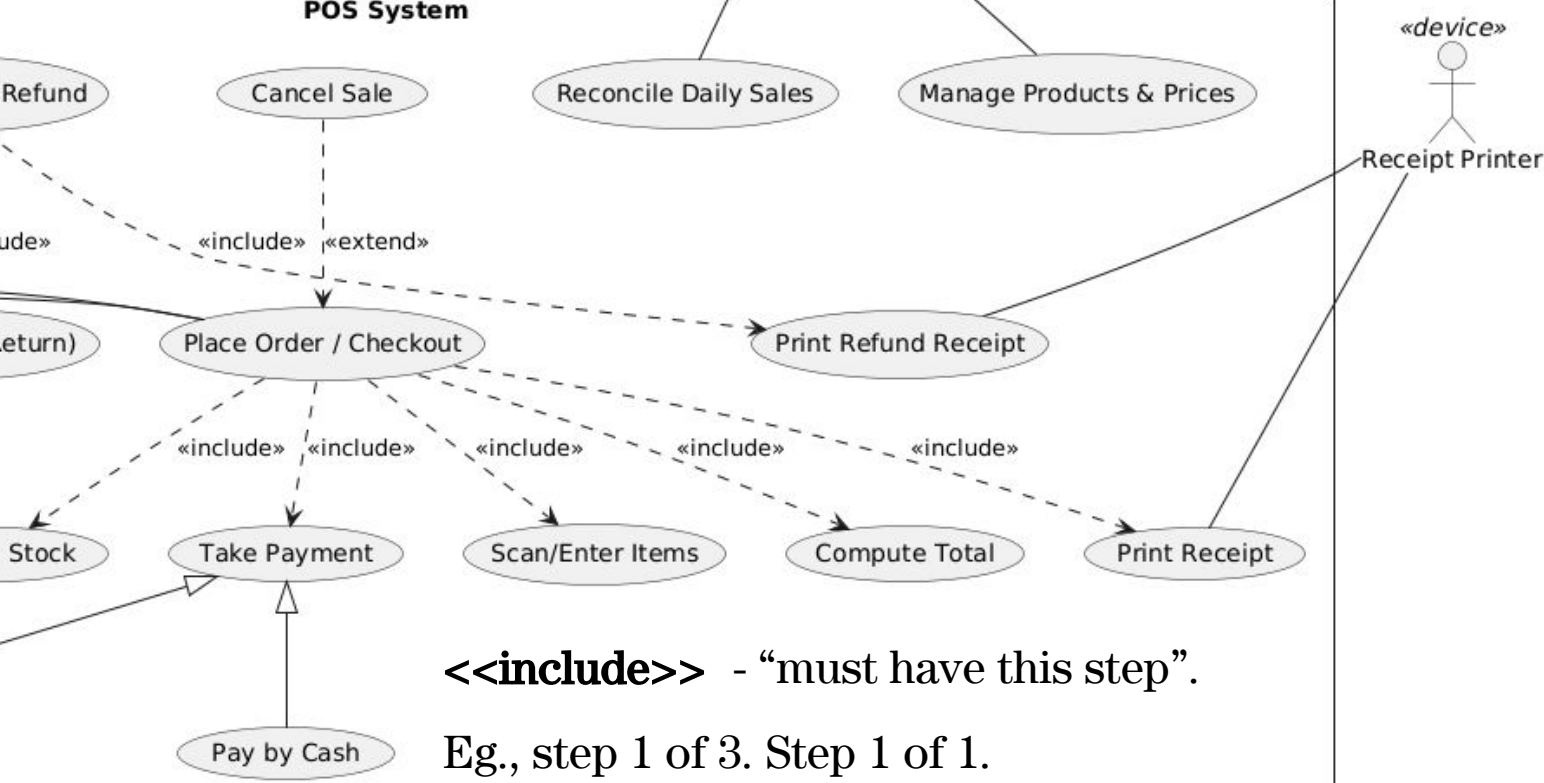
UML Diagrams

For Requirements and Design Specs

POS System Use Case Diagram



Sample Exam Problem: Draw a UML Use Case diagram at the architectural level. For the main workflows “Checkout” and “Refund”, go 1-2 levels deeper to cover the entire workflow.

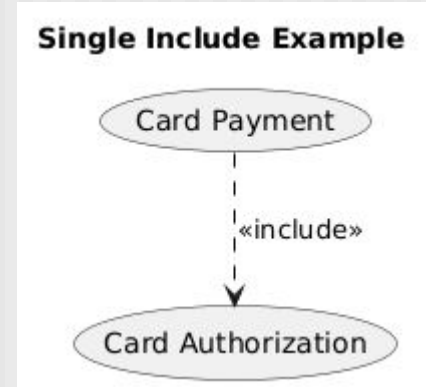
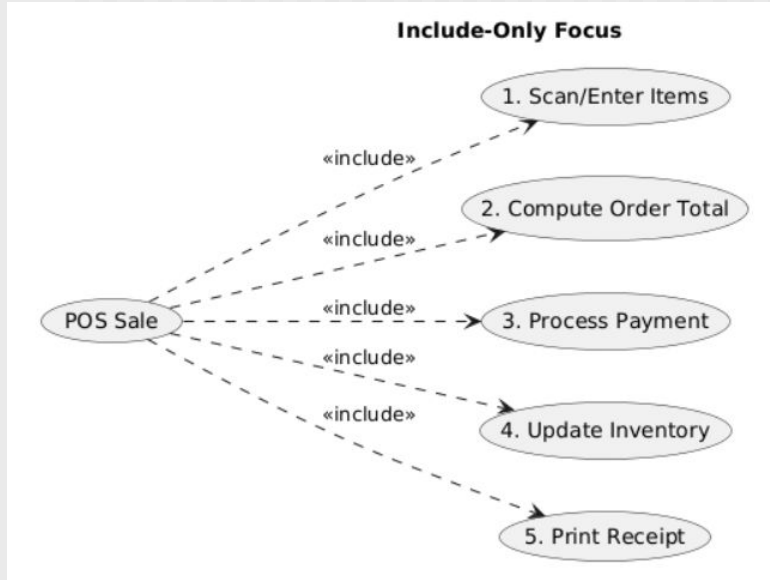


<<include>> - “must have this step”.

Eg., step 1 of 3. Step 1 of 1.

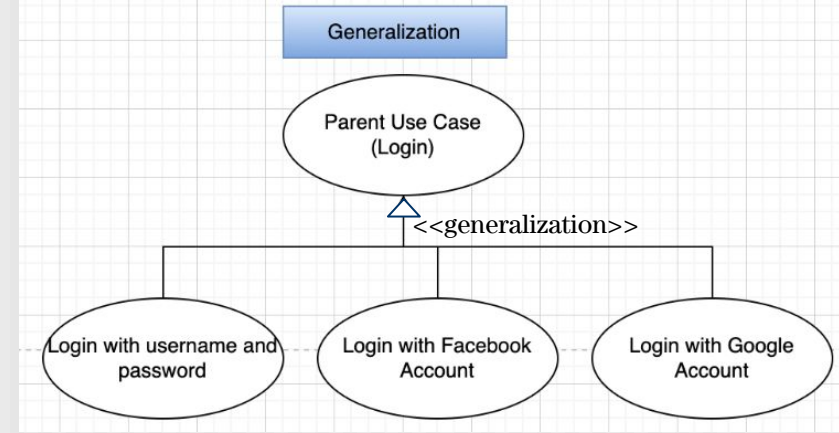
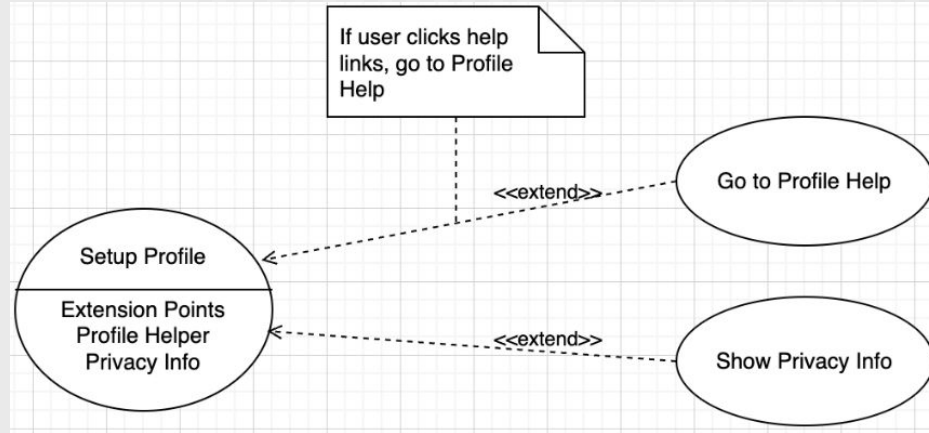
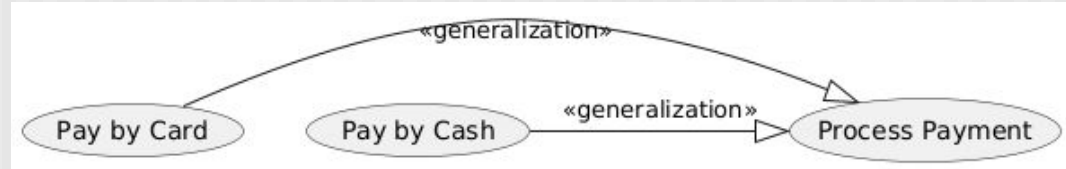
<<generalization>> - “inheritance” or “specialization(s)”. Unfilled triangle pointing to Parent.

Link labels in Use Case Diagrams



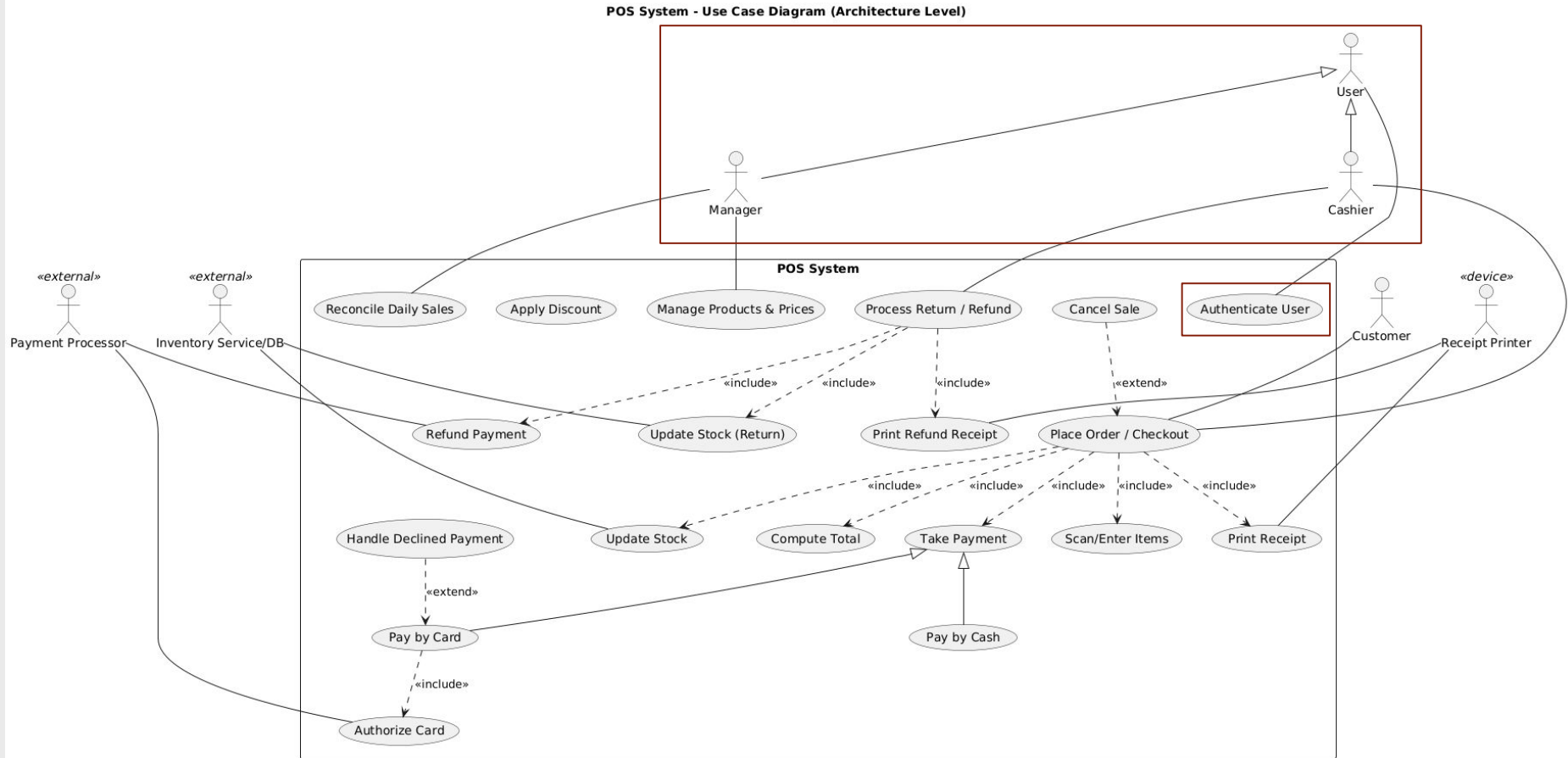
1. **no label** - actor participates in use case as solid line, no arrows.
2. **<<include>>** - Subprocesses. “must have this step n as a part of m steps”. May be 1 of 1.
3. **<<extend>>** - Optional. “does this if some condition met eg., press button”
 - “option n of m”. Arrow points from extended use case to the main use case.
4. **<<generalization>>** - “inheritance” or “specialization(s)”. Users: Manager, Cashier.

Link labels in Use Case Diagrams



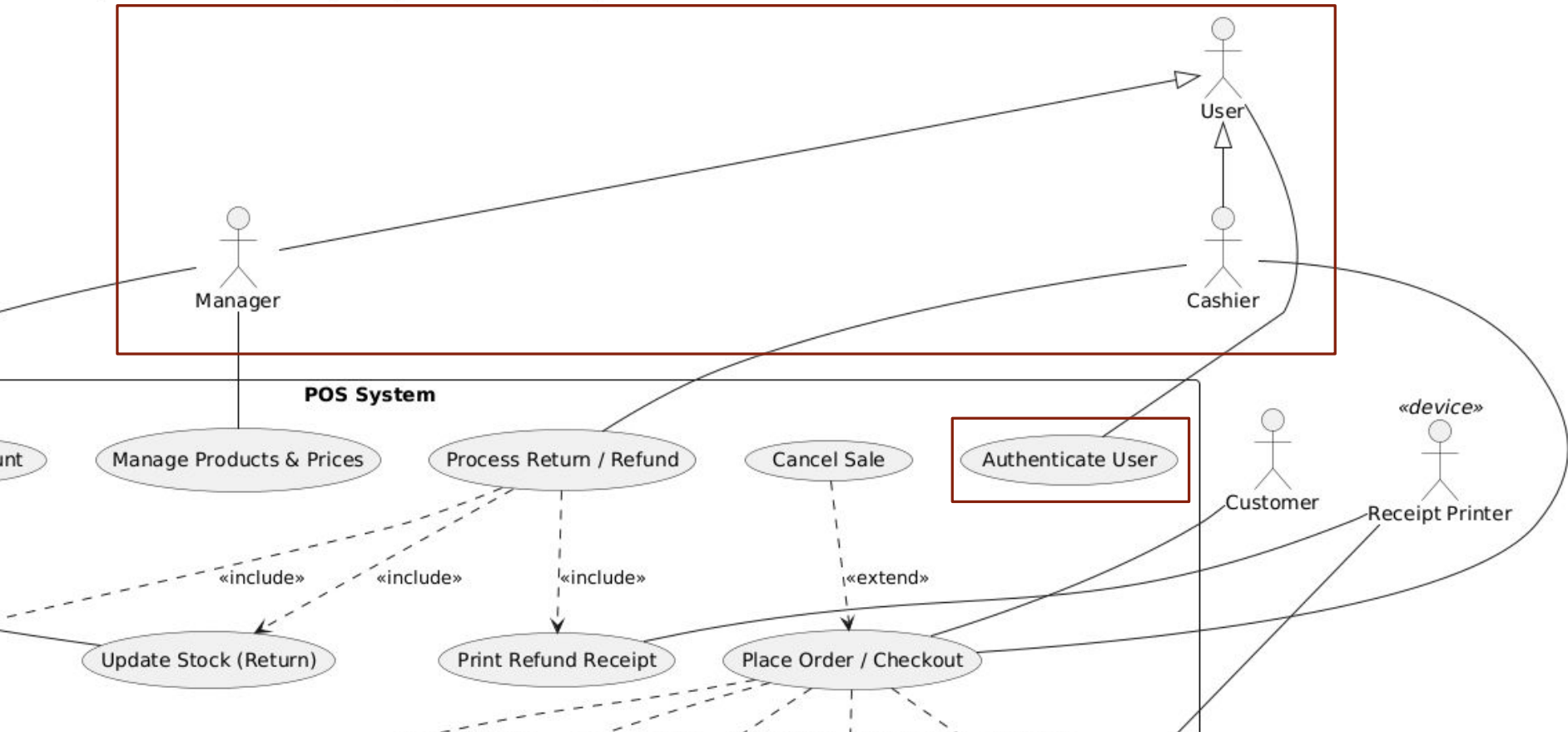
1. **no label** - actor participates in use case as solid line, no arrows.
2. **<<include>>** - “must have this step n as a part of m steps”. May be 1 of 1.
3. **<<extend>>** - Optional. “does this if some condition met eg., press button”
- “option n of m”. Arrow points from extended use case to the main use case.
4. **<<generalization>>** - “inheritance” or “specialization(s)”. Users: Manager, Cashier.

Showing Manager/Cashier as Users. Users Must Authenticate.



Showing Manager/Cashier as Users. Users Must Authenticate.

POS System - Use Case Diagram (Architecture Level)



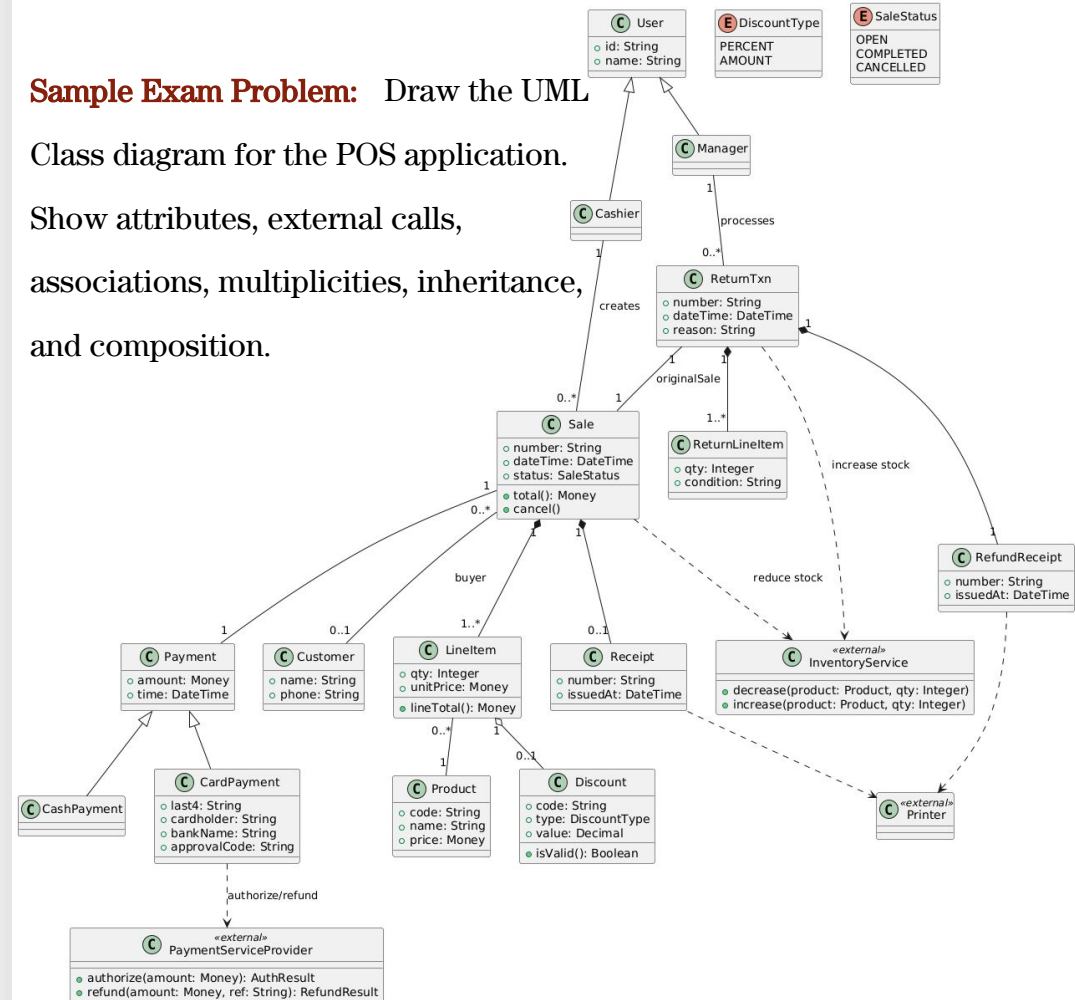
POS Class Diagram

- Attributes** = class fields + their types.
- Associations + multiplicities** = relations with 1..*, 0..1, etc. Use dashed arrows for external links.
- Composition** = strong whole-to-part links. “owns-a, born/die together”.
 - Sale ◆-- LineItem. Sale ◆-- Receipt
 - Return ◆-- ReturnLineItem, Return ◆-- RefundReceipt
- Aggregation** = white filled diamond ◇-. Weak whole-to-part links. No need to use it. **Aggregation.**
- Important inheritance Generalization** arrows (unfilled head) point to the **parent**.
 - Cashier, Manager → User.
 - CashPayment, CardPayment → Payment.

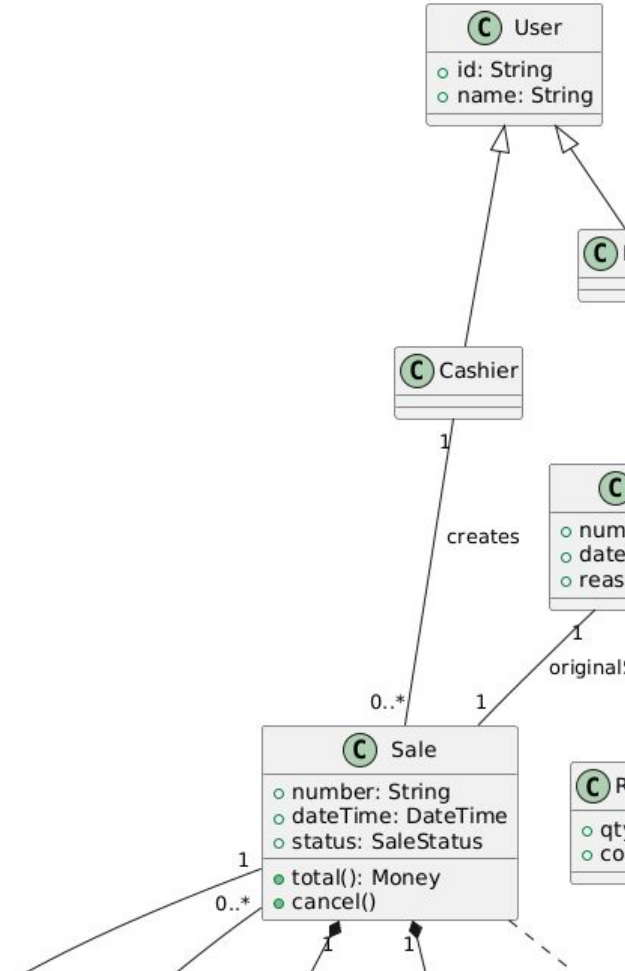
Sample Exam Problem: Draw the UML

Class diagram for the POS application.

Show attributes, external calls, associations, multiplicities, inheritance, and composition.



1. **Attributes** = class fields + their types.
2. **Associations + multiplicities** = relations with 1..*, 0..1, etc. Use dashed arrows for external links.
3. **Composition** = strong whole-to-part links. “owns-a, born/die together”.
 - a. Sale ◆-- LineItem. Sale ◆-- Receipt
 - b. Return ◆-- ReturnLineItem, Return ◆-- RefundReceipt
4. **Aggregation** = white filled diamond ◇-- Weak whole-to-part links. No need to use it. ~~Aggregation.~~
5. **Important inheritance Generalization** arrows (unfilled head) point to the **parent**.
 - 1) Cashier, Manager User.
 - 2) CashPayment, CardPayment Payment.



Workflow vs. Scenario using Activity vs. Sequence Diagrams

Workflow (Activity Diagram) = the **whole business process** with branches.

Scenario (Sequence Diagram) = **one concrete path** through that workflow. Example:

UI → Controller → Payment Service → Bank API

1. **Basic Flow / Main Flow** → the standard, expected sequence of steps when everything goes right.
2. **Alternative Flow** → valid variations via another “if/else” path (e.g., choosing Bank Transfer instead of Credit Card).
3. **Exception Flow** → error or failure flows (e.g., payment declined, invalid password).

POS **Order** Workflow. Workflow vs. Scenarios.

Activity Diagram: Workflow (with branches for options)

1. **Start** → **Scan/Add items** → Show cart total
2. **Apply discounts?** → [Yes] validate & apply → [No] continue
3. **Choose payment method** → Cash | Card | E-wallet
4. **Handle payment**
 - Cash → accept cash → give change
 - Card/E-wallet → call payment service → auth result
5. **Payment success?**
 - [Yes]: print/issue receipt → update inventory → End
 - [No]: **Offer change to cash** → if cancel, void order → End

Sample Scenario Paths in the POS **Order** Workflow.

1. Success Scenario: Pay by Cash (Happy Path)

- **Sequence (participants):** Customer → Cashier/POS UI → POS App → Inventory → Receipt Printer
- **Messages:** addItem* → computeTotal → recordPayment(cash, amount) → updateStock → printReceipt

2. Success Scenario: Pay by Card with Online Auth (Alt path)

- **Participants:** Cashier/POS UI → POS App → Payment Service → Inventory → Receipt Printer
- **Messages:** authorize(amount, cardToken) → authOK → capture → updateStock → printReceipt

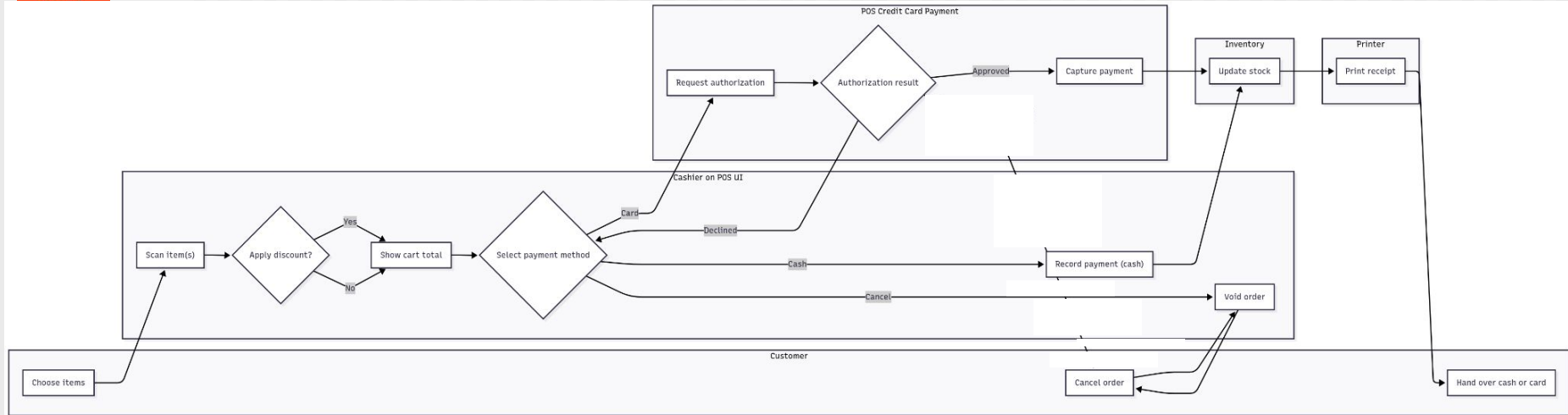
3. Card Declined Scenario and Switch to Cash (Alt path)

- **Messages:** authorize → authFailed → alternatePayment? yes by cash → recordPayment(cash) → updateStock → printReceipt

4. Out-of-Stock Detected at Checkout and Item Changed (Alt path)

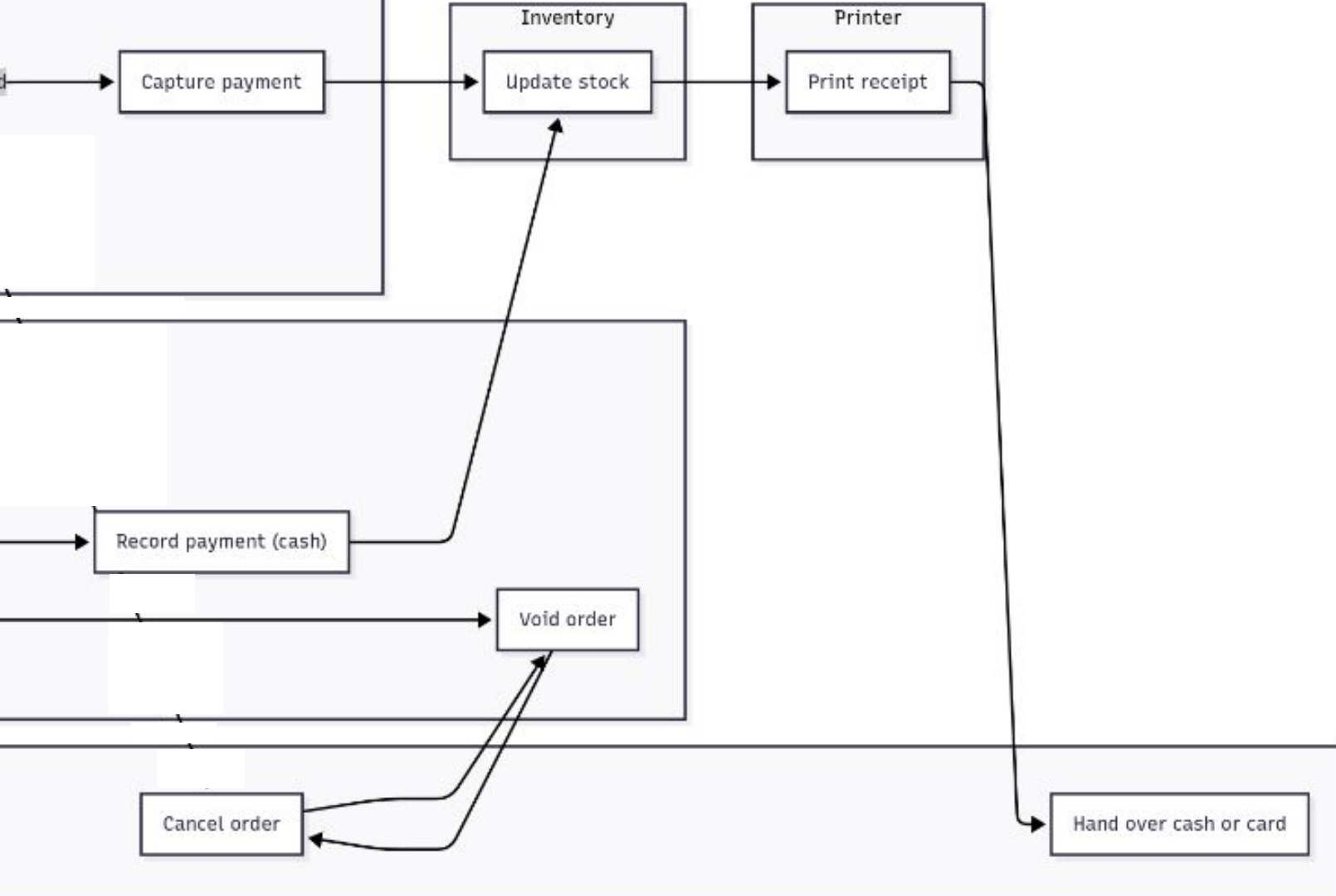
- **Messages:** addItem → checkAvailability → outOfStock → Remove → recomputeTotal → go to payment

Activity Diagram for POS Order Workflow

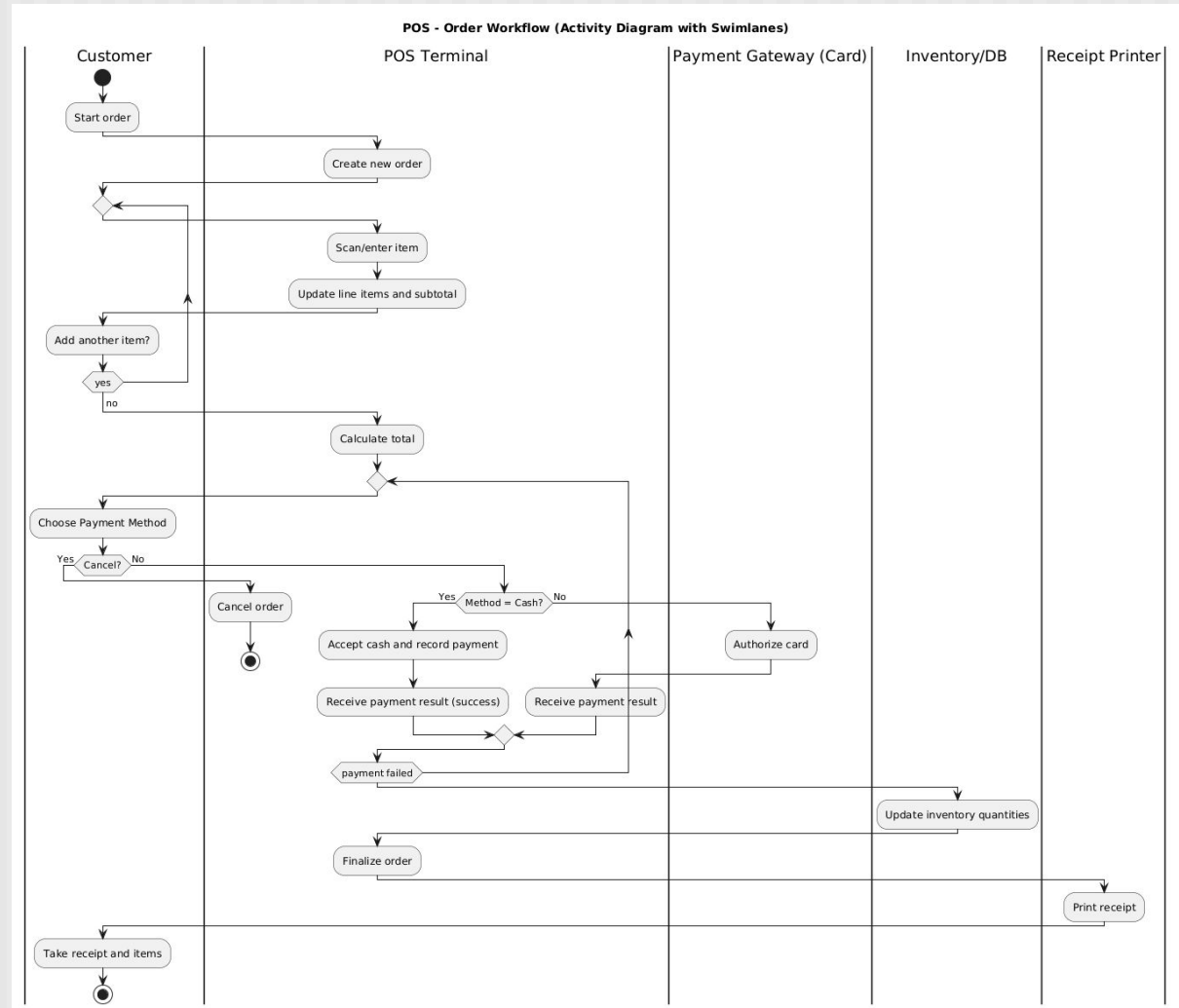


“Each swimlane is a **partition** representing a responsible Actor (user, role, external component). The actions inside the lane are owned by that partition.”

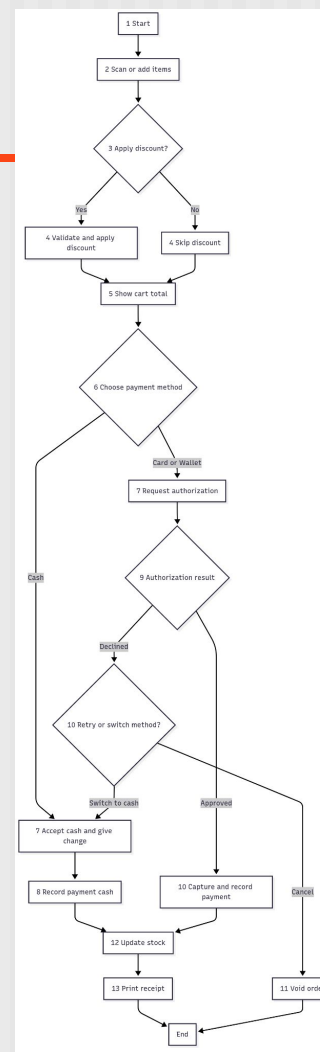
Sample Exam Problem: Draw the UML Activity diagram with partitions (swimlanes) for the POS Order workflow from select items to payment. Note that 1) customers' card payment may be declined 2) customer may decide to cancel before payment.

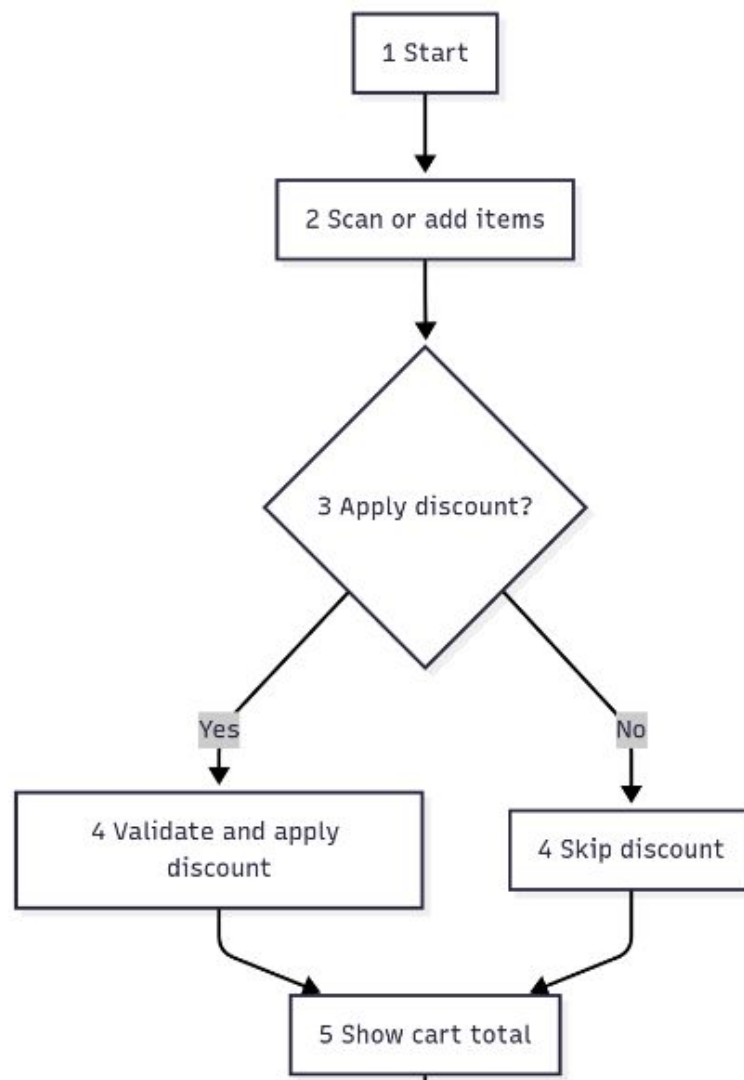


Activity Diagram for POS Order Workflow: Vertical Swimlanes.

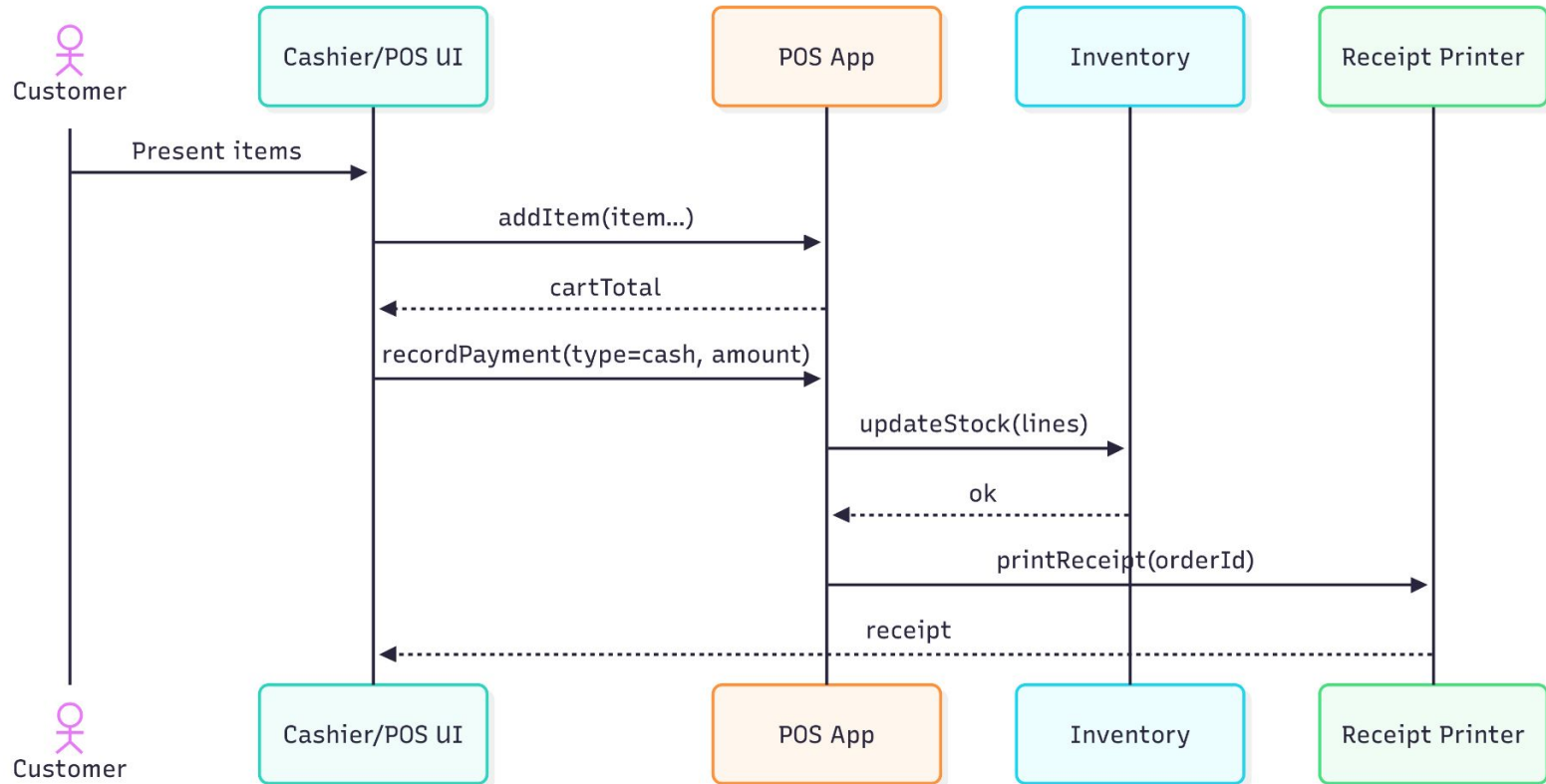


Activity Diagram (No Swimlane)





Sequence Diagram: Pay by Cash Scenario

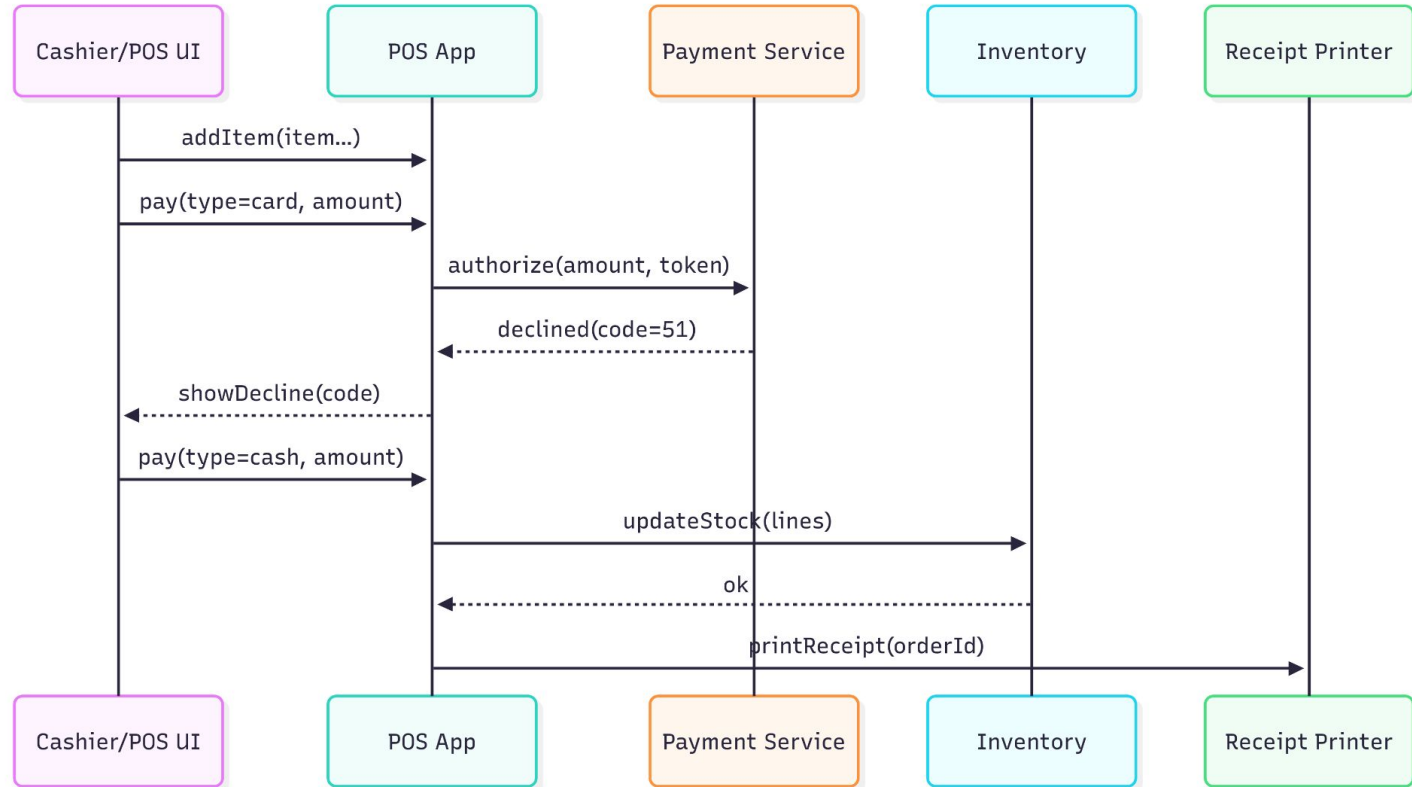


Sample Exam Problem: Draw the UML Sequence Diagram for scenario: Customer successfully buys items, paying by cash.

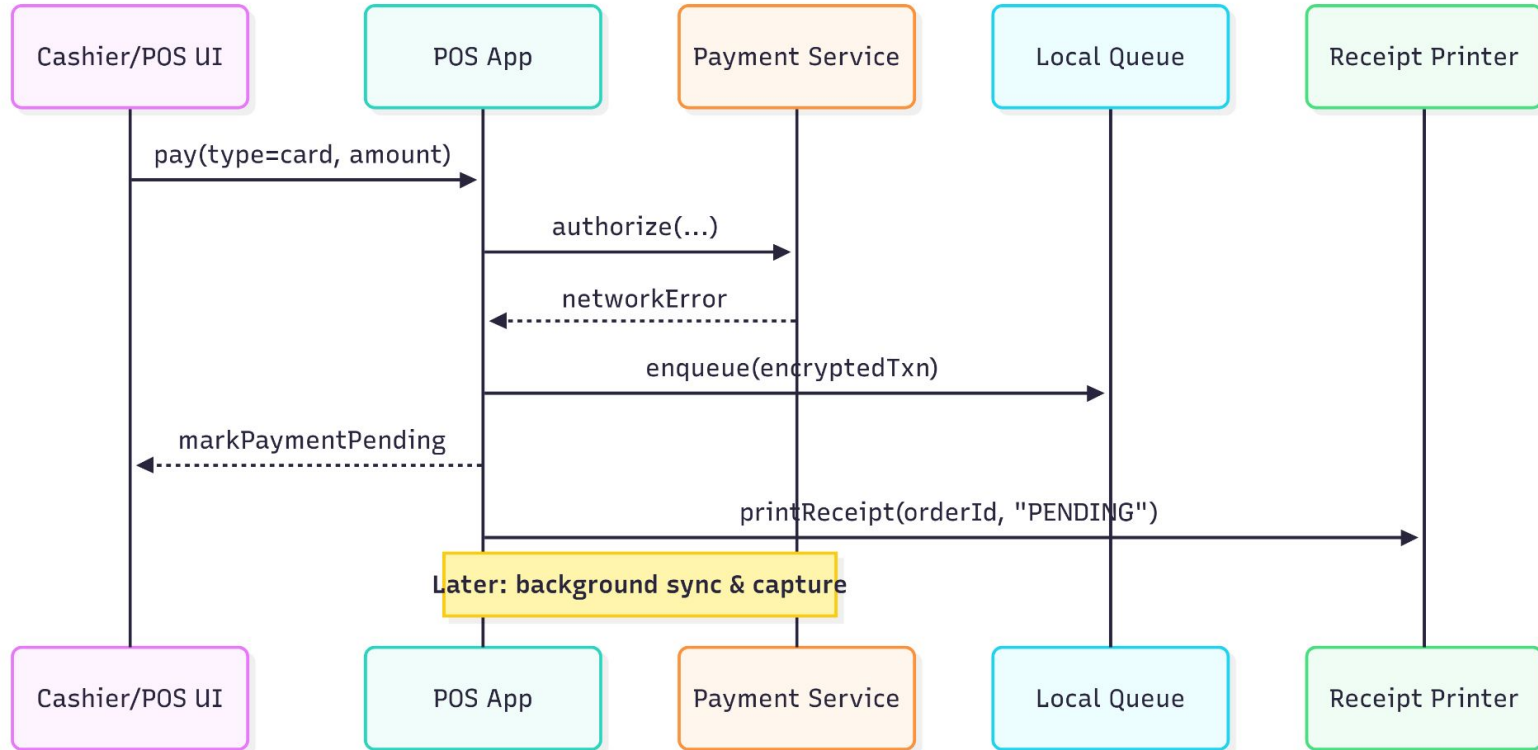
Sequence Diagram: Credit Card Declined, Switch to Cash Scenario

Sample Exam

Problem: Draw the UML Sequence Diagram for scenario: Customer buys items, paying by credit card, card declined, switches to pay by cash.



Sequence Diagram: **Payment System is Down** scenario



Note: This diagram is a partial scenario focusing on the payment part of the scenario. A scenario is usually end-to-end.

POS Returns and Refunds Workflow . Workflow vs. Scenarios.

Activity outline for Workflow.

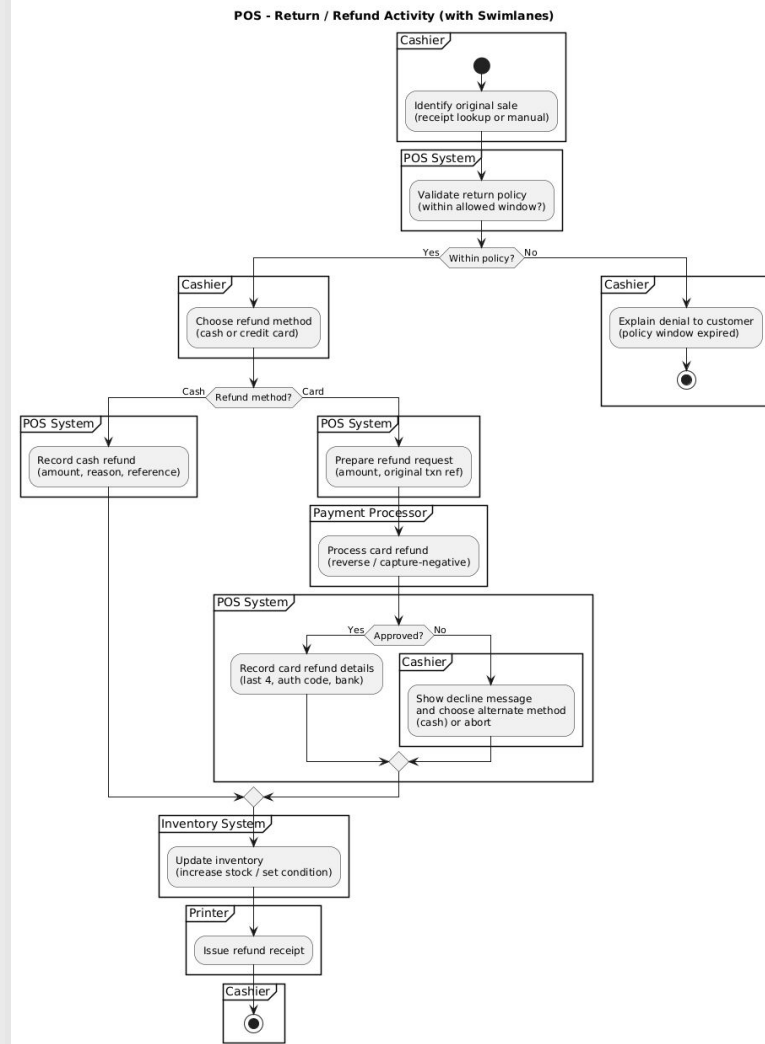
1. **Start** → **Identify original sale** (receipt lookup/manual)
2. **Validate return policy** (window, condition)
3. **Choose refund method** (original tender, store credit)
4. **Execute refund** (reverse/capture-negative)
5. **Update inventory** (return-to-stock or damaged) → Issue refund receipt → **End**

Scenarios

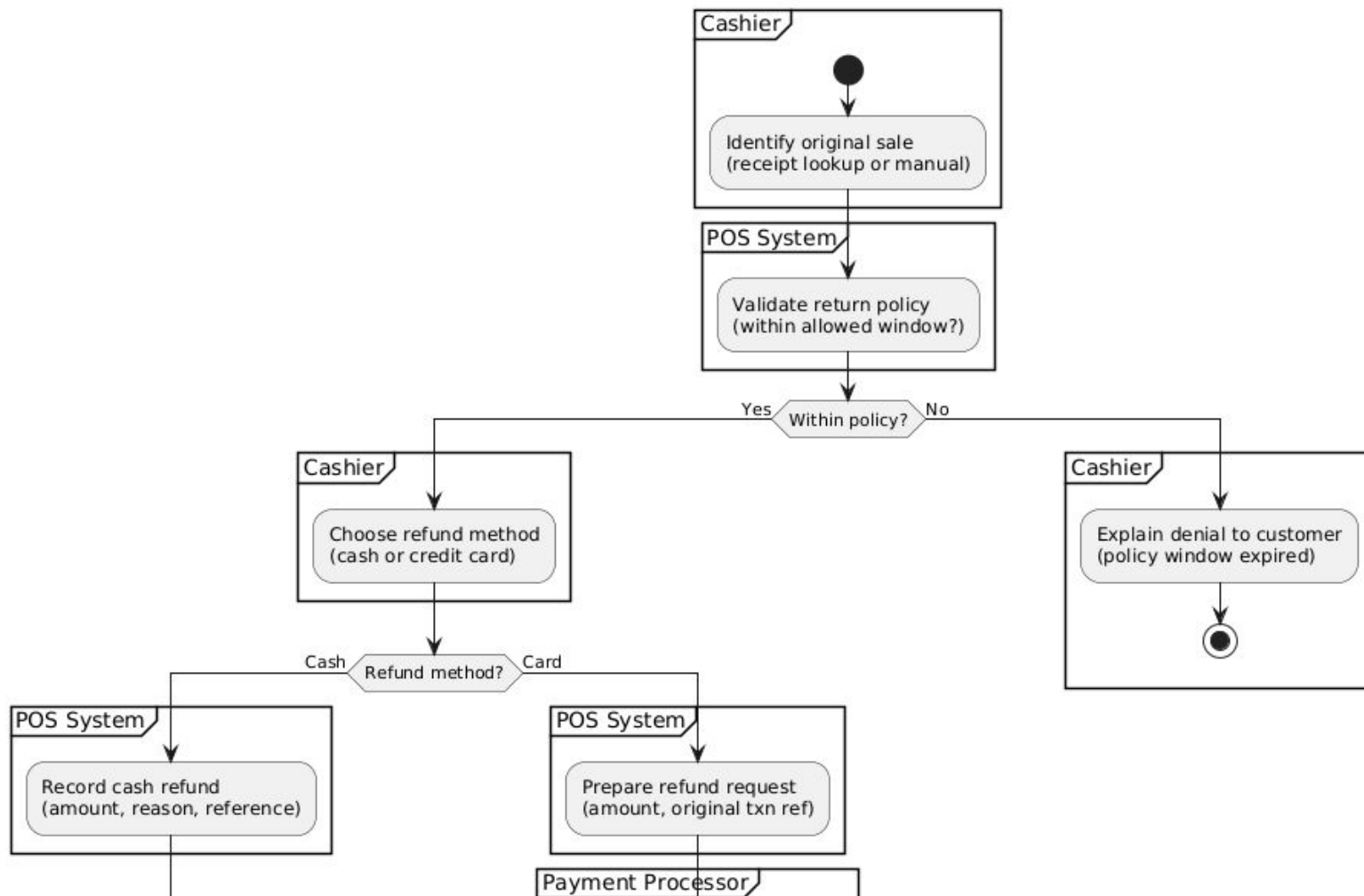
1. **Refund to Original Card (Happy Path). Sequence:** refundRequest → validatePolicy → refund(cardTxnId, amount) → refundOK → restock → printRefundReceipt
2. **Refund Denied: Too many days past. Sequence:** validatePolicy → denyReturn(reason) → offerStoreCredit? → End
3. **Partial Refund On Only One Line Item. Sequence:** selectLine → compute Pro-rated Tax → refund Partial → restockPartial

POS Return/Refund Activity Diagram with Partitions (swimlanes)

Sample Exam Problem: Draw the UML Activity Diagram for workflow: Customer returns items. Note return may be denied if against store return policy. Also, card payment may have problems.



POS - Return / Refund Activity (with Swimlanes)



Hotel Reservation Example

Functional Requirements (FR): All-in Hotel Reservation System

All-Inn is a simple online booking system where guests can reserve rooms at many member hotels through <http://www.all-inn.com>. All-Inn takes a 10% commission for each booking from the member hotel. The system supports three Room Types for every member hotel: Regular Room, Superior Room, Suite Room. The system uses Microsoft Outlook/SMTP as the external Email Service to send emails to users.

1. All Users (Guest, All-in Admin, Hotel Admin)

- Log in.
- Change own password.
- Request reset for forgotten password. Automated email link will be sent to user.

2. Guest User (Guests are registered All-Inn members)

- Search for rooms by entering check-in date, check-out date, city, and country.
- View a list of hotels with room types and total price (sum of nightly rates). Options: View Detail or Reserve.
- View hotel details with pictures, room type details with pictures, nightly prices, total price, and hotel location shown via integration with Google Maps.
- Make a reservation: see summary (hotel, room type, nights, prices), then edit name, email, and contact info.
- Pay by credit card via K-Bank Payment Gateway.
- View on-screen booking confirmation with confirmation number with booking info.
- System reduces available rooms and sends confirmation email to the hotel and to the guest.
- Other functions: Cancel My Reservation, View My Reservations, Change My Profile.

Condensed FR: All-in Hotel Reservation System

3. All-Inn Admin User

1. Create new user accounts: Guest, Hotel Admin, or All-Inn Admin.
2. Reset any user's password. Email link will be sent to user.
3. Add new hotels: hotel code, name, location, city, country, pictures, description, email.
4. Add room types for hotels: hotel, room type, description, pictures.
5. View **Room Summary Report** for a selected hotel and date range (hotel, date, room type, rooms supplied, rooms reserved, rooms available, unit price, total reserved value, total commission value).

4. Hotel Admin User

1. One admin user can manage only one hotel.
2. Set up daily room supply and price (hotel, date, room type, rooms supplied, unit price).
3. View reservations by All-in starting on a given check-in date (reservation details, guest info, total price).
4. View **Room Summary Report** for their own hotel over a selected date range.

Hotel UI.

Show a Hotel List View after selecting Date Range and City

Hotels List View		Guest: Sing Songkram	
[Close]			
Country:	_Thailand_ [LoV]		
Province:	_Chonburi_ [LoV]	City:	_Pattaya_ [LoV]
Check-In Date:	_07/06/2025_ [Date]	Check-Out Date:	_10/06/2025_ [Date]
Number Nights:	3	Room Type:	☒ Regular ☒ Special ☐ Suite
[FILTER][CLEAR]			
List of Hotels with Availability for Above Selection: (select row to view hotel/room information)			
Hotel Name	Address	Lowest Room	Priced / Night
Holiday Villa	121, Pattaya North, Thailand		2,000.00
Cheapo Hotel	221, Pattaya South, Thailand		700.00
Quality Beds	71, Pattaya South, Thailand		1,500.00
Haunted Inn	21/49, Pattaya, Thailand		350.00

Sample Exam

Problem: Show a simple Hotels List View UI. Guest selected country, province, city plus the check-in, check-out dates. System then lists the qualified hotels with starting prices for the nights selected. Guest can select hotel to view room price details.

Hotel UI. Room booking view after Dates / Hotel is selected

Reservation Form View Generic Design

Guest: Sing Songkram

[\[Back to Select Hotel/Dates\]](#) [\[Make Payment\]](#)

Booking No.:

Booking Date:

Check-in Date:

Check-out Date:

Number Nights:

Guest User:

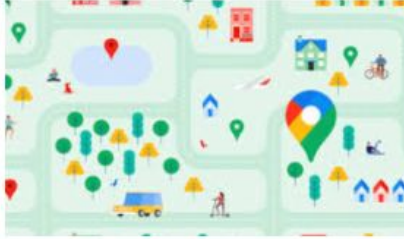
Hotel:

City:

Hotel Image:



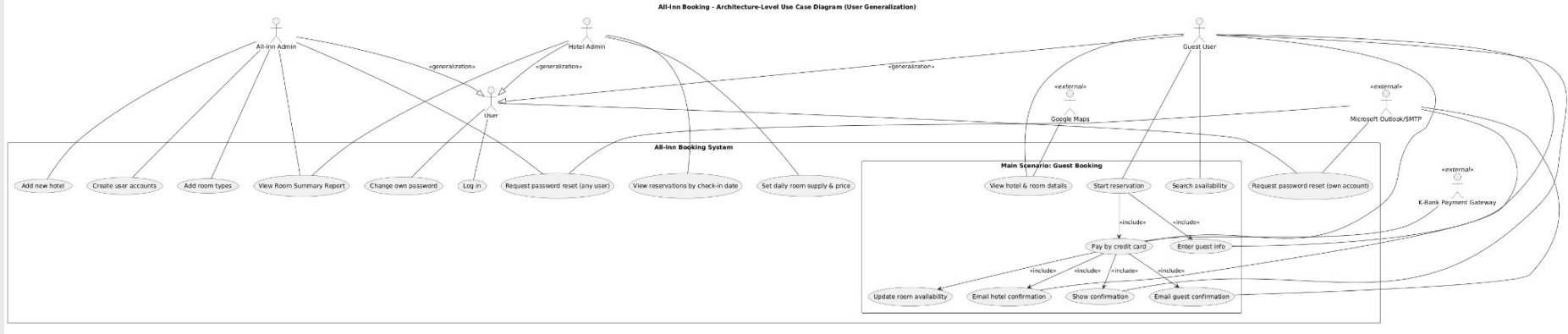
Hotel Map View:



Room Type	Total Price	Average Price / Night	Number Rooms Available	*Number Rooms To Book	Your Price	
Regular	6,000.00	2,000.00	3	2	12,000.00	View Image
Superior	9,000.00	3,000.00	8	1	9,000.00	View Image
Suite	12,000.00	4,000.00	1			View Image
Your Total Price:					21,000.00	

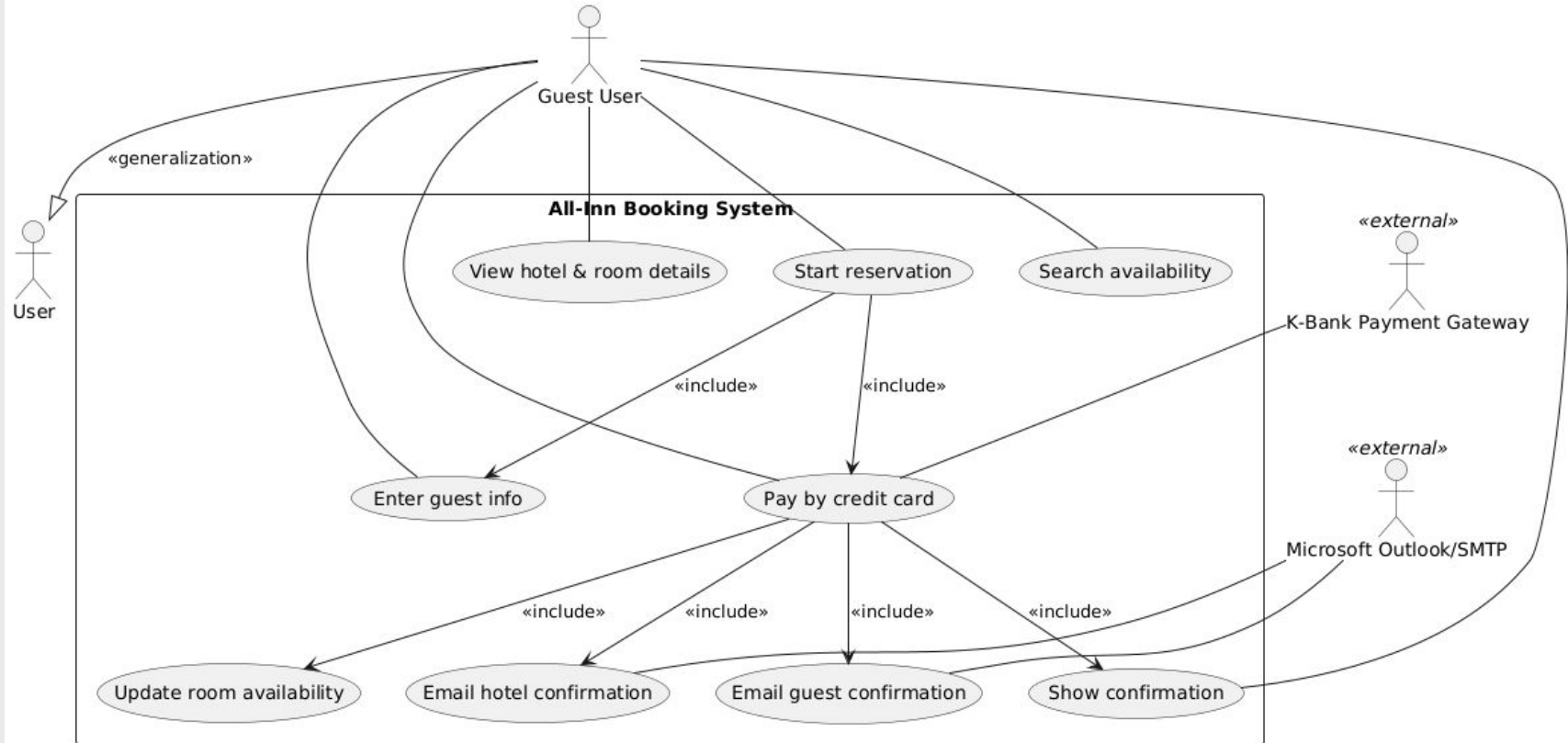
Sample Exam Problem: Show the Booking Form UI where hotel information including image and Google Map panel is presented along with available rooms. Assume the hotel and date ranges have already been selected from a previous list view. Each available room type and price for the nights of stay (e.g., 2 nights) will appear. In one booking, the guest can choose many rooms for each room type.

Hotel Reservation Use Case Diagram

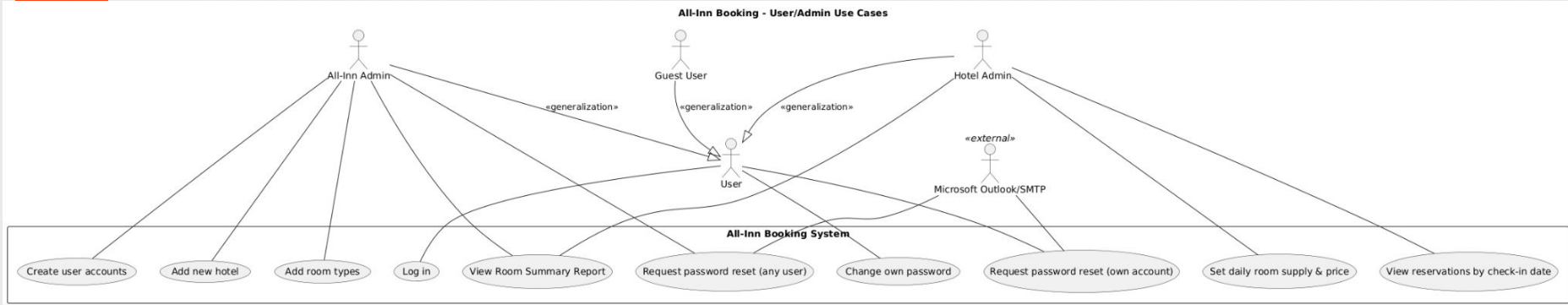


Hotel Reservation Use Case: Zoom-In Right

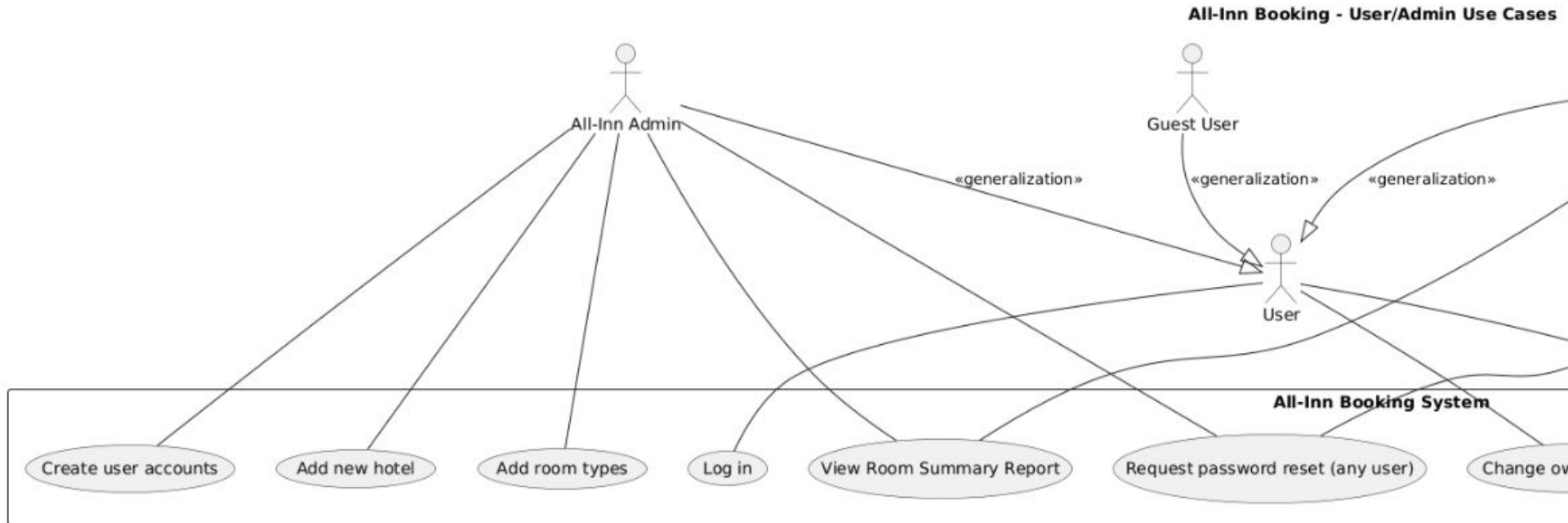
All-Inn Booking - Guest User Use Cases



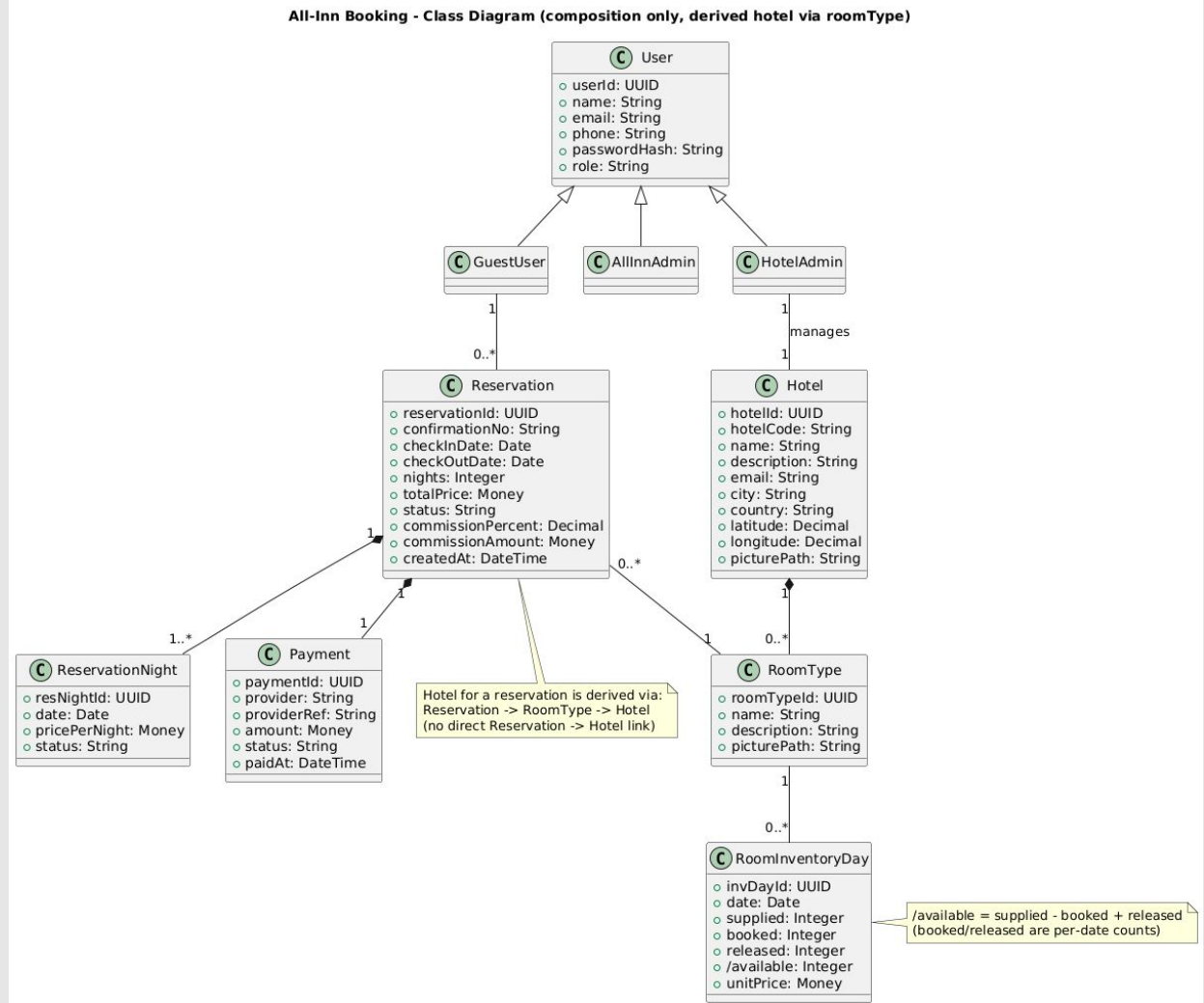
Hotel Reservation Use Case: Zoom-In Left



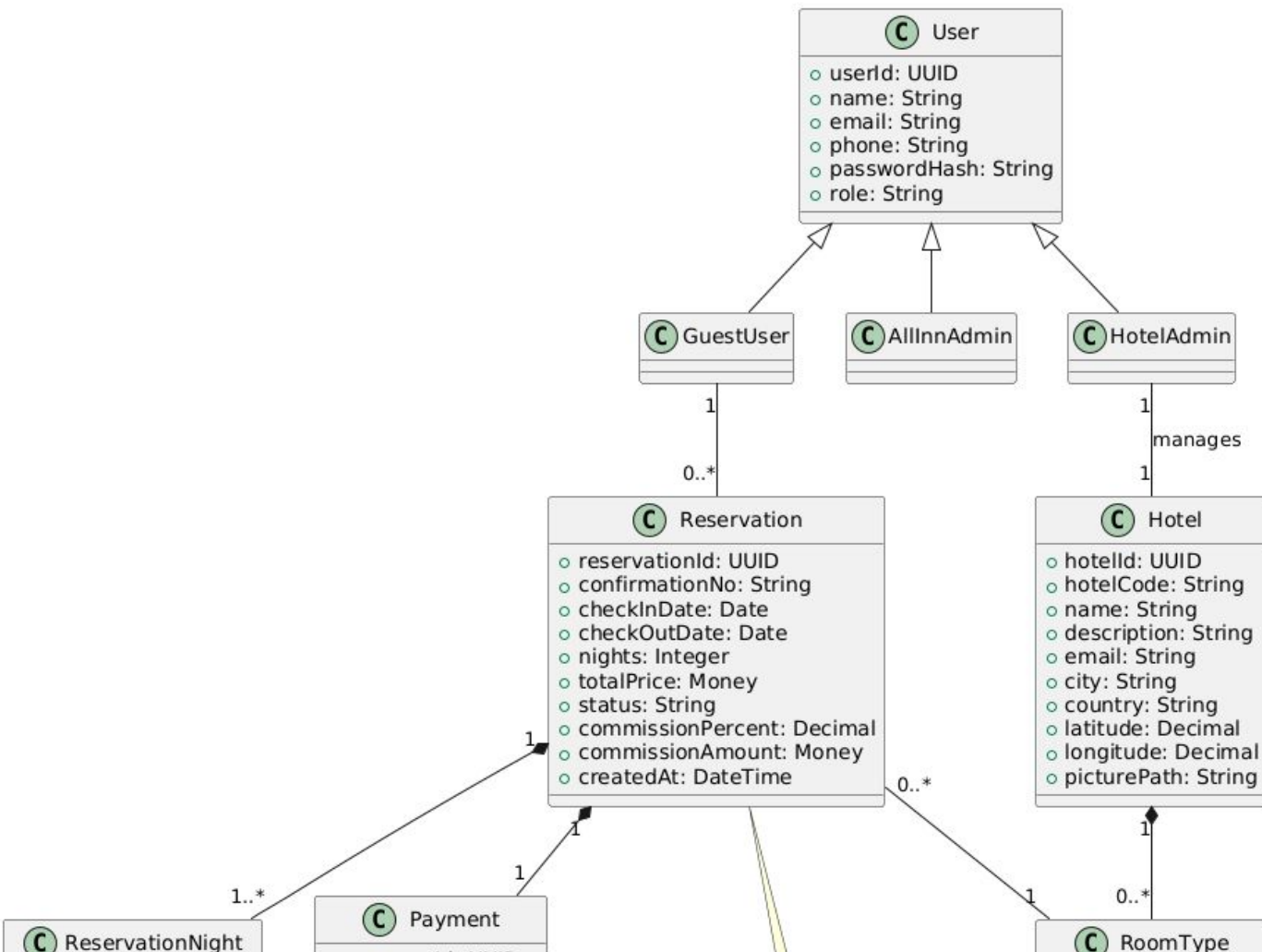
Zoom-in Left



Hotel Class Diagram



All-Inn Booking - Class Diagram (composition only, derived hotel via roomType)



Activity Diagram

Sample Exam Problem: In a main workflow: the Guest searches hotels given a city with date range, views room prices in that hotel, makes a booking of 1 or more rooms, pays via K-Bank, system confirms a booking. Payment may be denied, where the guest can try again until she succeeds or decides not to book. Draw an **activity diagram** .

Activity Diagram

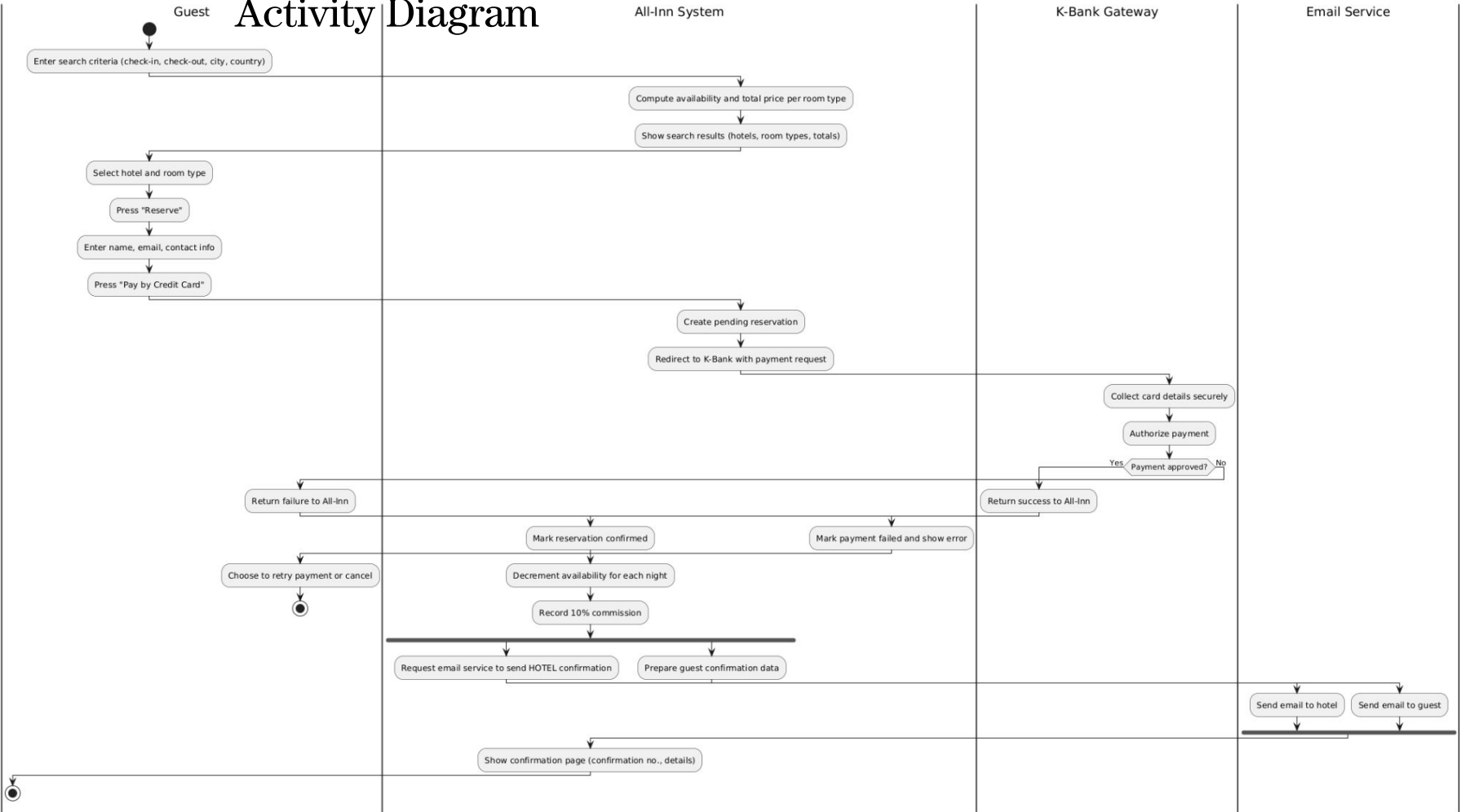
All-Inn Booking - Main Purchase Workflow (Activity Diagram)

Guest

All-Inn System

K-Bank Gateway

Email Service



All-Inn Booking - Main Purchase Workflow (Activity Diagram)

Guest

All-Inn System



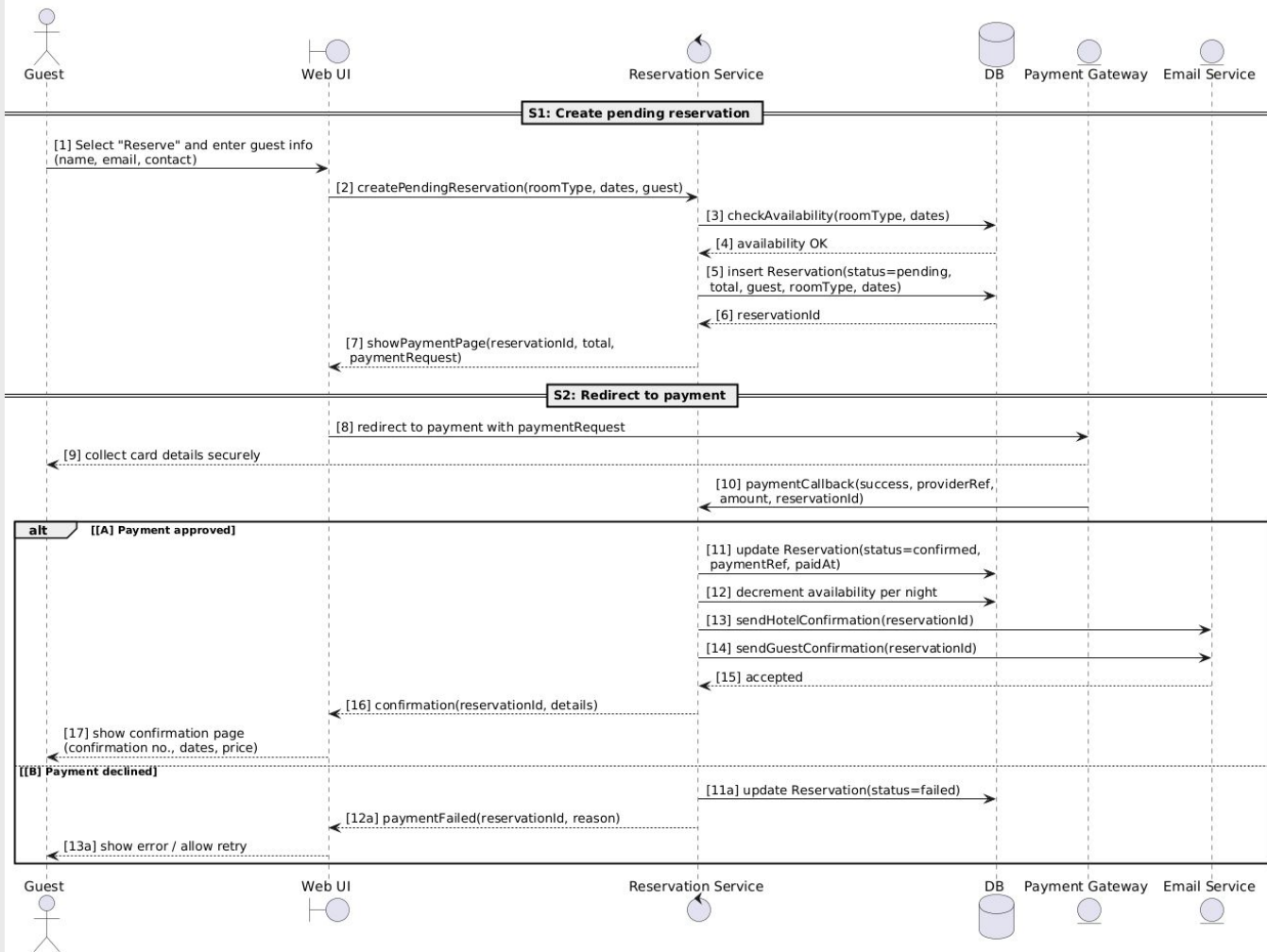
Sequence Diagram

Example Exam Problems:

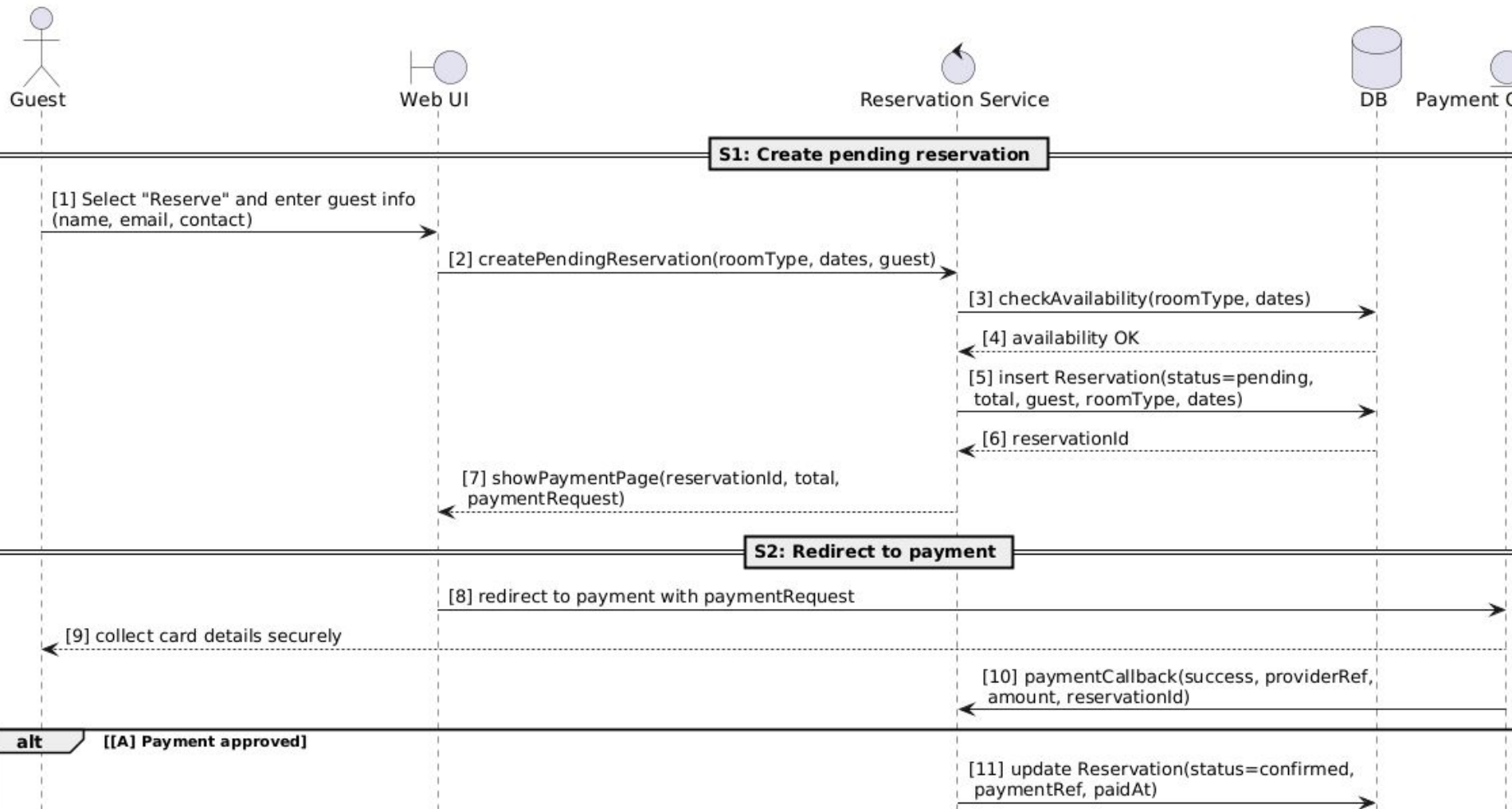
- A. Show the sequence for scenario when Guest reserves a room and successfully pays with a credit card.
- B. Show the sequence for scenario when Guest reserves a room and has a declined payment, but allowed to try again and then succeeds with another card.

Who are the Actors? Answer: Guest, Web UI, Reservation Service, Payment Gateway, DB, Email Service.

All-Inn - Sequence: Guest reserves and pays (labeled)



All-Inn - Sequence: Guest reserves and pays (labeled)



Software Service Ticket Example

Functional Requirements List

ABC Co. has hundreds of customers using its various software applications. ABC wants all customers to report change requests (includes bugs) by visiting the website http://www.ABC.co.th/service_ticket. Here are the requirements:

- Four types of users can Login: Customer - submits ticket, Change Control Engineer (CCE) - approves it for work by programmer, Programmer - works on the ticket, IT Admin - only for creating users etc. (not involved in ticket processing)
- After login the top of the page shows the person's name, user type (Customer / Change Control Engineer (CCE) / Programmer / IT Admin), and user's company.
- The Ticket users (Customer, CCE, and Programmer) see a list of all service tickets grouped by Software Name + Company sorted by due date. The Customer sees only their company's tickets. A user can pick a listed ticket to view the ticket and its actions list.
- There is a panel for the Ticket Users to create a new ticket or edit an existing ticket. Ticket fields: Ticket ID (auto number), Ticket Date, Program Name (list), the Form/Report/Component Name to change, Type of Problem (bug fix, missing feature, added work, other), Problem Description, Priority (high, regular, low), Requested by User Name, User Company Code, Request Date, Initial Due Date, Last Due Date, Last Status. Attachments: allow one picture (e.g., screen capture) and one file (doc/zip/etc.). These are kept in the network file system.
- Each ticket shows an Action Records History list with: Action Date, Done by User Name, user's role (customer / change control engineer / programmer), Action Taken Notes, New Due Date, New Service Status.

Functional Requirements List

- When a New Ticket is saved, one Action Record is automatically generated with the New Due Date set from Ticket's Initial Due Date, and New Service Status set to "Requested".
- After any action is saved, the Ticket's Last Status and Last Due Date attributes are updated using that latest action.
- New Actions Records can be added to the Ticket only by CCE and Programmers. Action records cannot be edited/deleted once saved.
- Change Control Engineers (CCE) review and approve requests can add actions to tickets that are "Requested". They can add an action for a ticket to set "Approved" (and optionally a New Due Date and Notes) or "Closed by CCE." They cannot modify the ticket.
- Programmers can add actions to tickets that are "Approved" or "In Progress".
 - They pick a ticket to work on, add an action record and keep Status as "In Progress," but add Notes and a New Due Date.
 - They may add more action records along the way (e.g., update due date or add Notes).
 - When finished, they add an action record with Status: "Closed by Programmer" and New Due Date: Completion date.
- A closed ticket (either "Closed by CCE" or "Closed by Programmer") cannot receive new actions.
- Other functions are done by IT Admin Users: Manage Company Info, Manage User Info (includes user type and company), Manage User Services, and Manage Software Info.

Service Ticket UI.

Ticket Form View Generic Design (Minimalist for Exam)

Service Ticket Form View Generic Design

User: Ying Kesuka Role: Customer

[Tickets](#)
[Users](#)
[Software](#)
[Company](#)
[Logout](#)
[Reports](#)
[Authorization \(App Controls\)](#)

[\[Close\]](#)
[\[New\]](#)
[\[Save\]](#)
[\[Clear\]](#)
[\[Print\]](#)
[\(Form Controls\)](#)

Ticket No.:

*Ticket Date: [Date]

Issued by User:

*User Company: [LoV]

*For Software: [LoV]

*Software Component:

*Problem Type: ▼

*Priority: ▼ {Low, Medium, High}

*Problem Description:

*Initial Due Date: [Date]

Attach Image: [Browse][Open]

Attach File: [Browse][Open]

Last Due Date:

Last Status:

List of Action Records:

[New Action]

#	Action ID	Action Date	Action Done By	Role of User	Work Done Notes	New Due Date	New Status
1	AC25-01	15/09/25	Somchai	Customer	Started	20/09/25	Requested
2	AC25-01	16/09/25	Bam	CCE	Approved	20/09/25	Approved
3	AC25-01	18/09/25	Kate	Programmer	Started working on bug fix. Must move due date.	22/09/25	In-Progress

Sample Exam Problem: Show a simple UI for a Service Ticket Form View. It should have ticket information on the header and a list of action records for that ticket. Also show menu items for the core use cases in this application. Also show the Tickets List View UI for Programmer Role.



Ticket List View For Programmer to Work On Ticket

User: Ake Numchoke **Role:** Programmer

[Tickets](#) [Users](#) [Software](#) [Company](#) [Logout](#) [Reports](#) [Authorization \(App Controls\)](#)

[Close][New]

Filter Status: ☐Requested ☒Approved ☒In-Progress ☐Closed by CCE ☐Closed by Programmer

- CCE will by default only see status: Requested. Programmers by default will see status: Approved and In-Progress. Customer sees all (limited to customer's company).

Search / Group / Filter: [OK]

Next Action: ☒ Open Form (Primary Key) ☐ Select Record (Foreign Key)

Click Row to select a record: (Will open form or select the record, depending on “Next Action”)

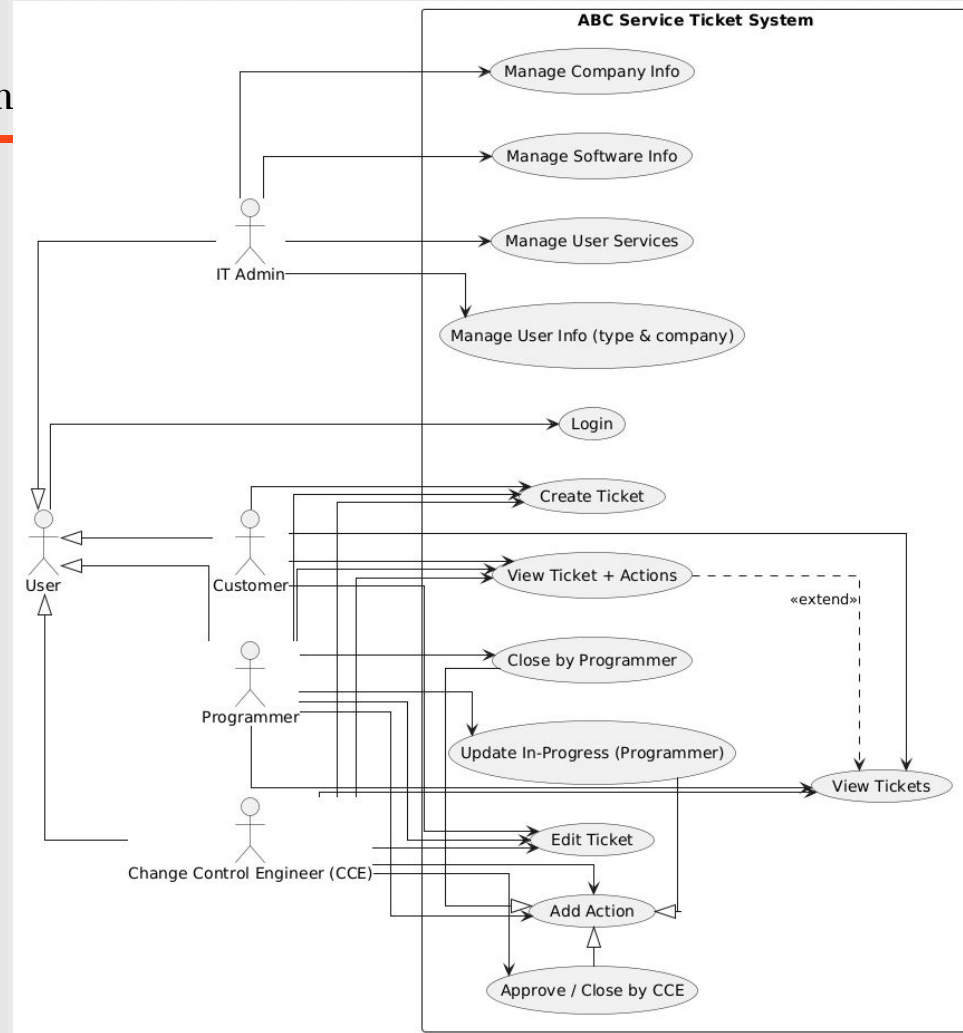
[illegible]

3

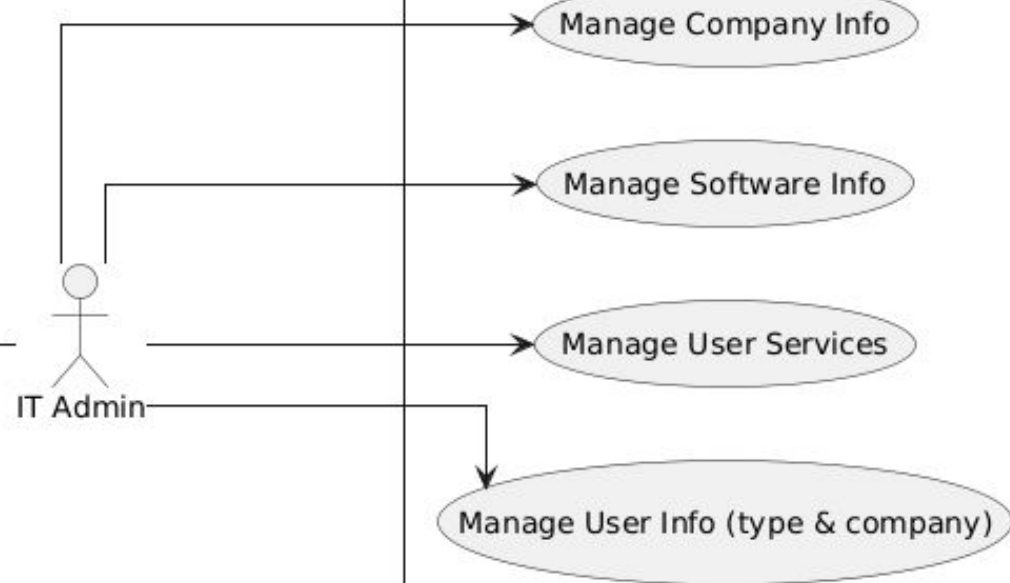
Ticket Use Case:

Sample Problem: Correct this diagram

1. All links from actor to use case should have no arrows.
2. Remove all links from actors to "View Ticket + Actions" as this is only accessible via "View Tickets" which it already extends.
3. Remove all actor links to "Create Ticket". It should extend "View Tickets".
4. Remove all actor links to "Edit Ticket". It should extend "View Tickets".
5. Remove all links to "Approve / Close by CCE". Add an <<extend>> link from it to "View Ticket + Actions".
6. Remove all links to "Update In-Progress (Programmer)". Rename it to "Update In-Progress / Close by Programmer". Add an <<extend>> link from it to "View Ticket + Actions".
7. Remove "Close by Programmer".



ABC Service Ticket System

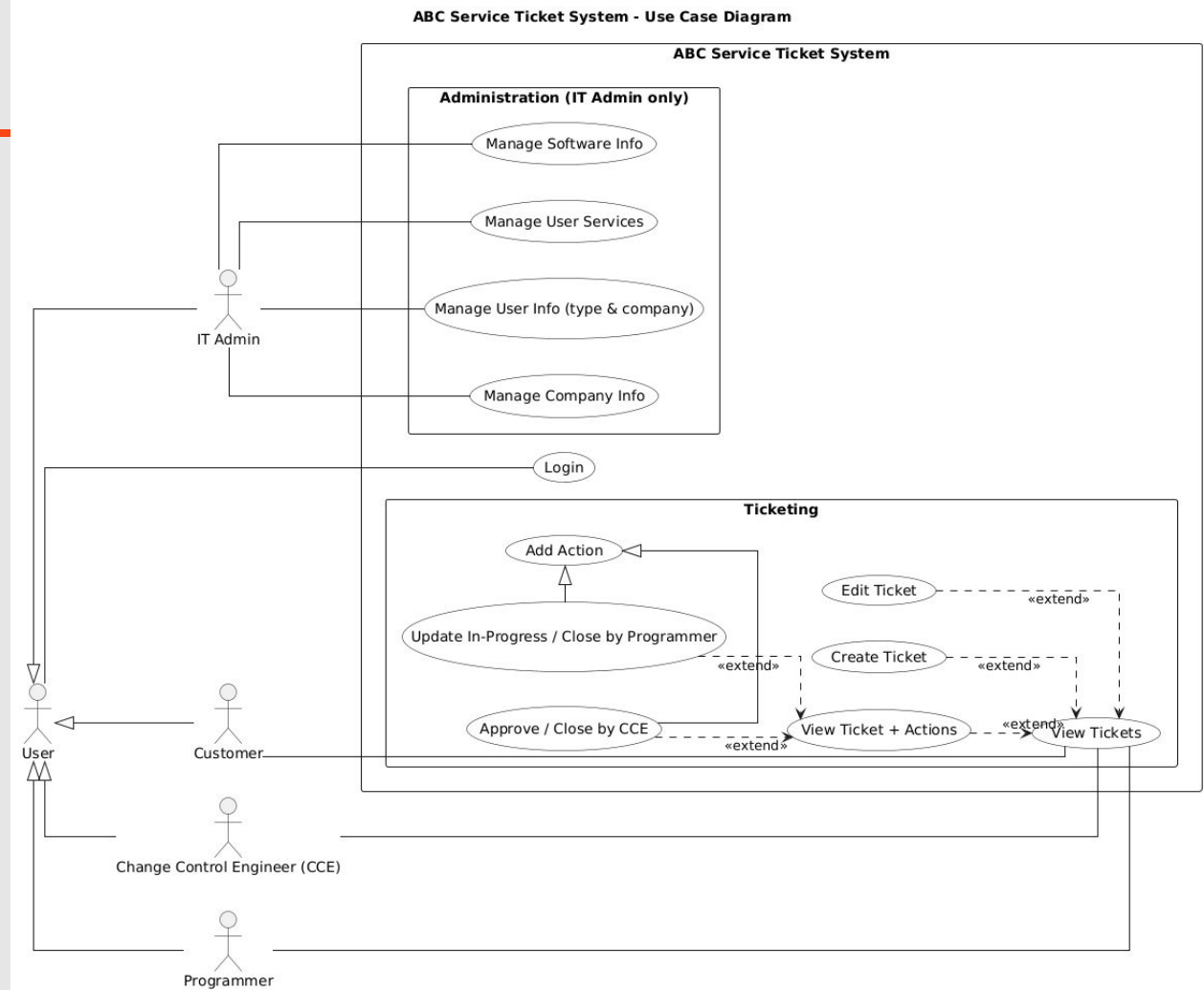


Login

Create Ticket

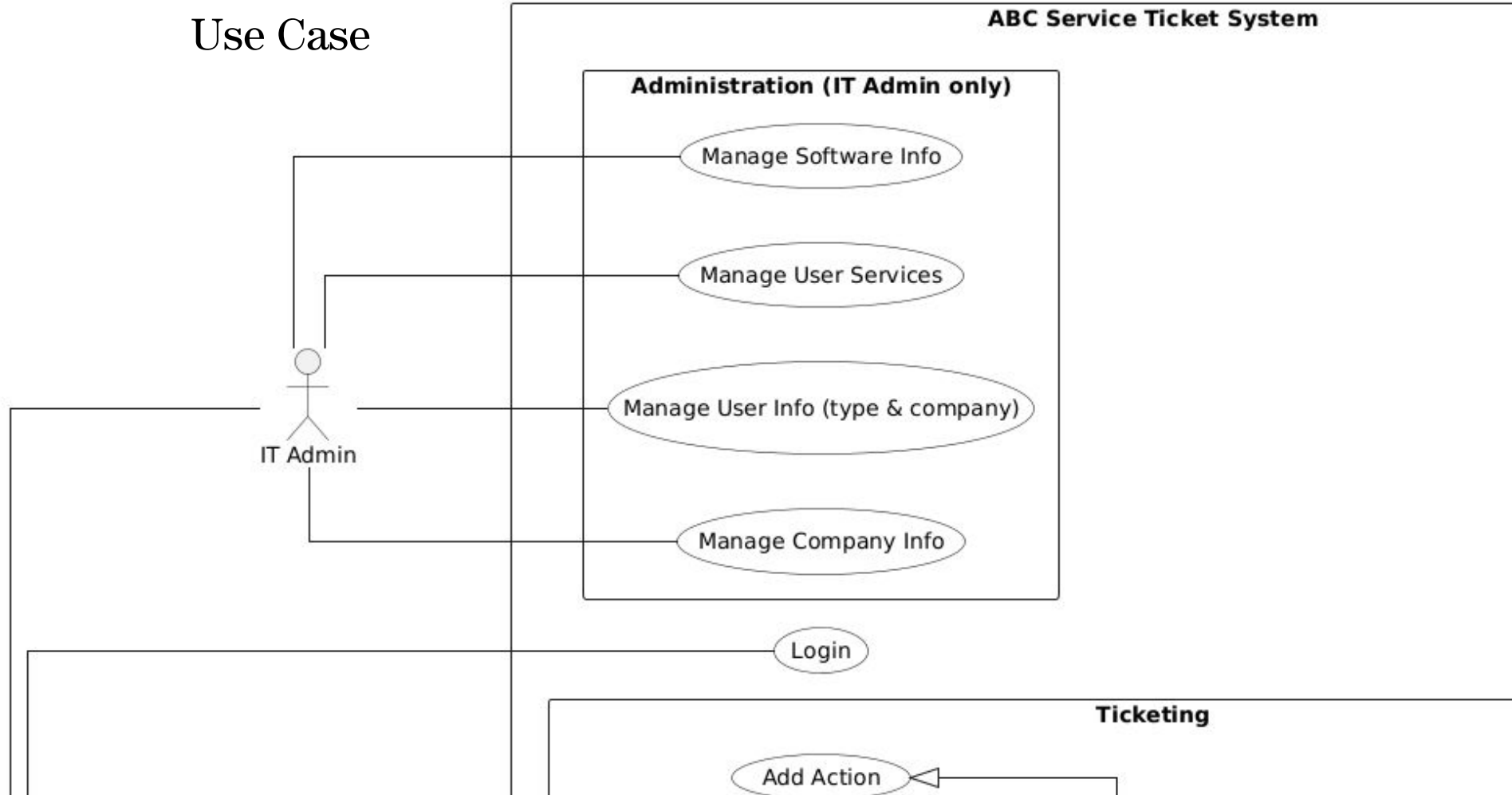
View Ticket + Actions

Corrected Use Case

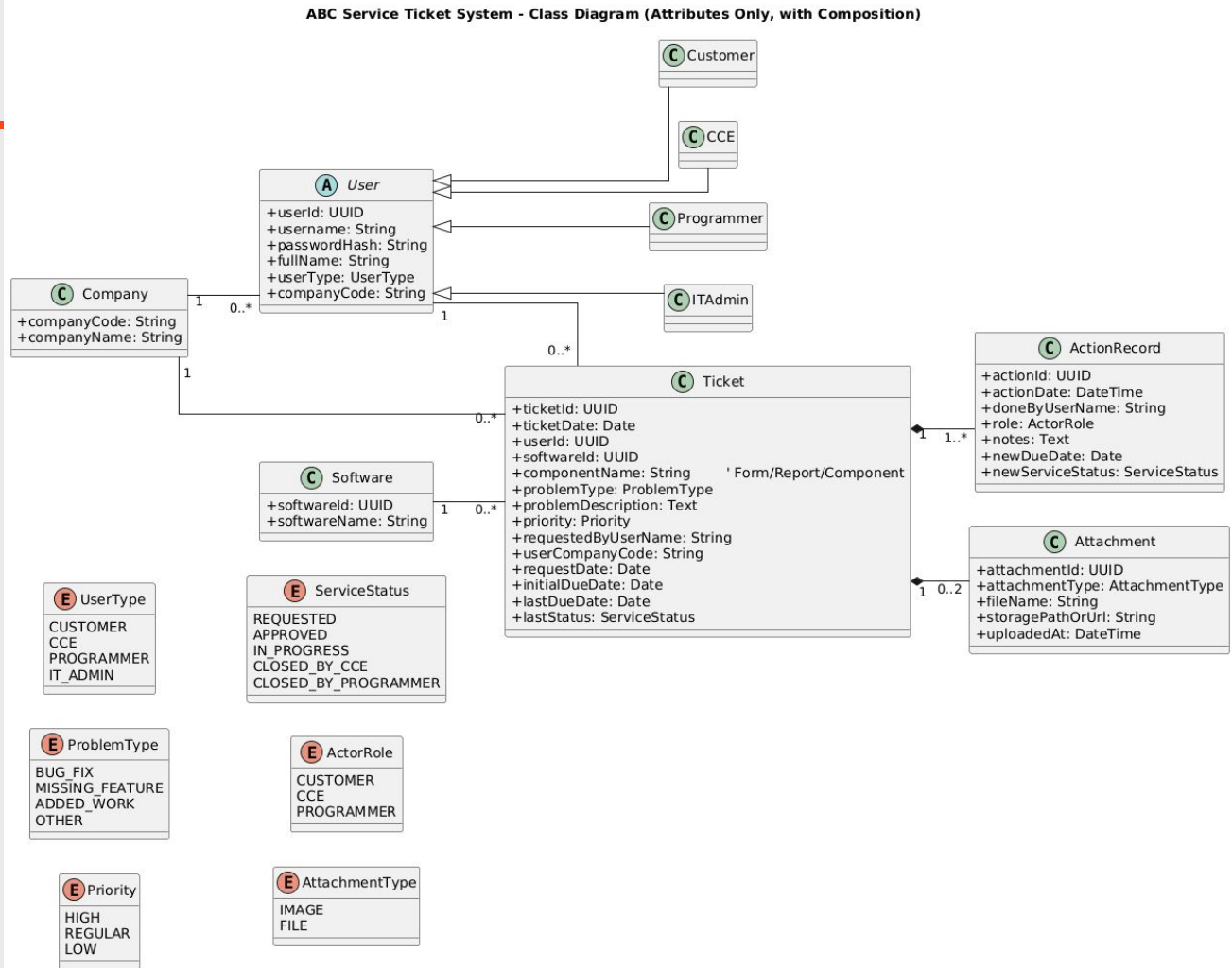


Corrected Use Case

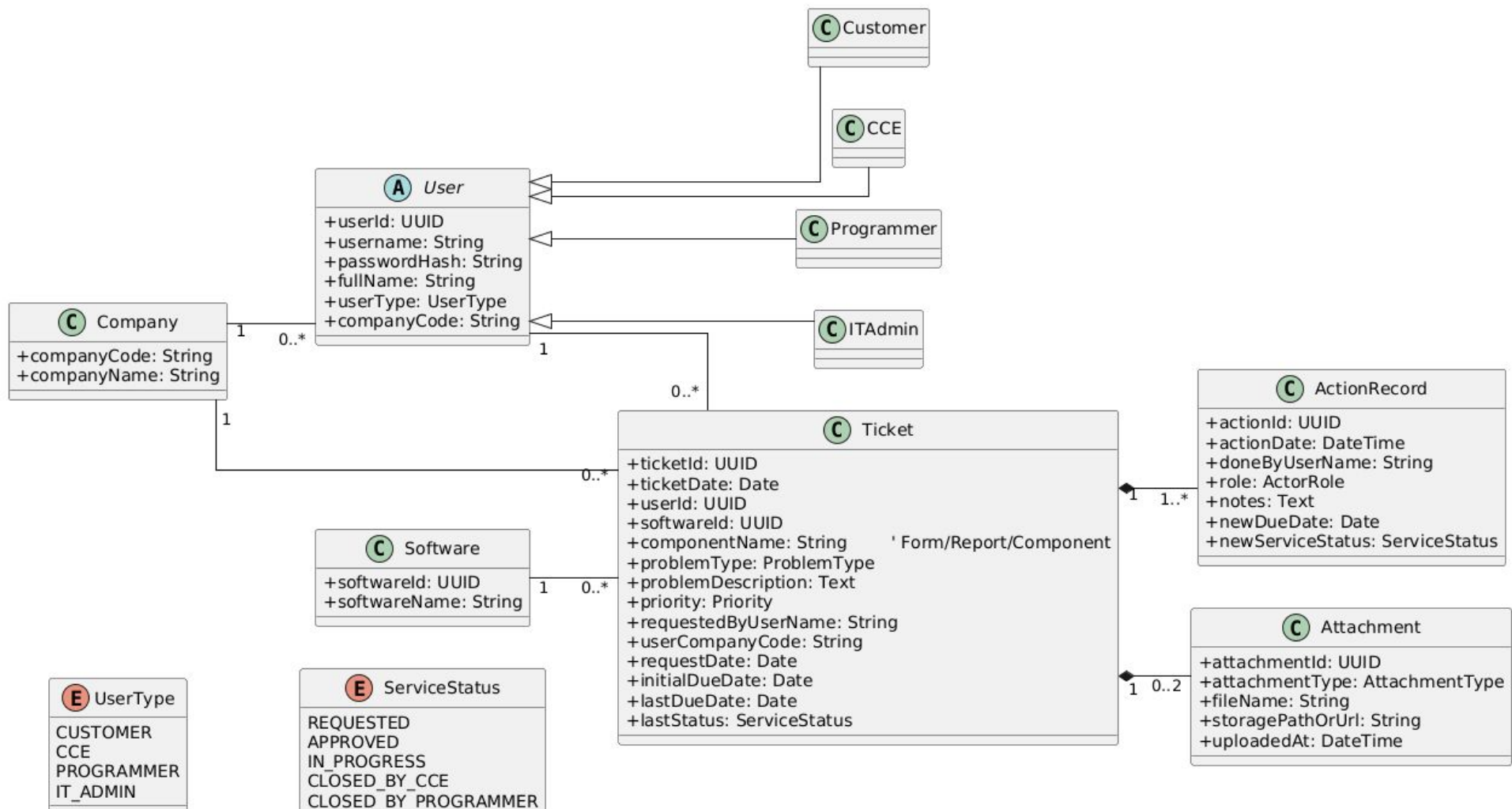
ABC Service Ticket System - Use Case Diagram



Ticket Class Diagram



ABC Service Ticket System - Class Diagram (Attributes Only, with Composition)



Activity Diagram for Ticket Lifecycle Workflow

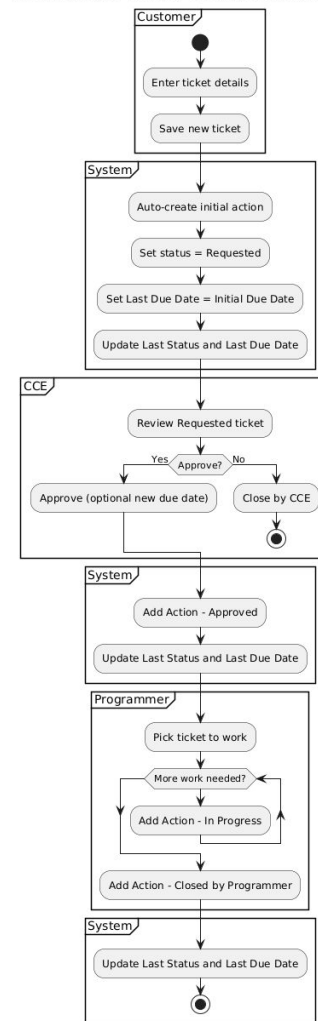
Sample Problem

Draw an **Activity Diagram** with swimlanes (Customer, CCE, Programmer, System) for the **ticket lifecycle** .

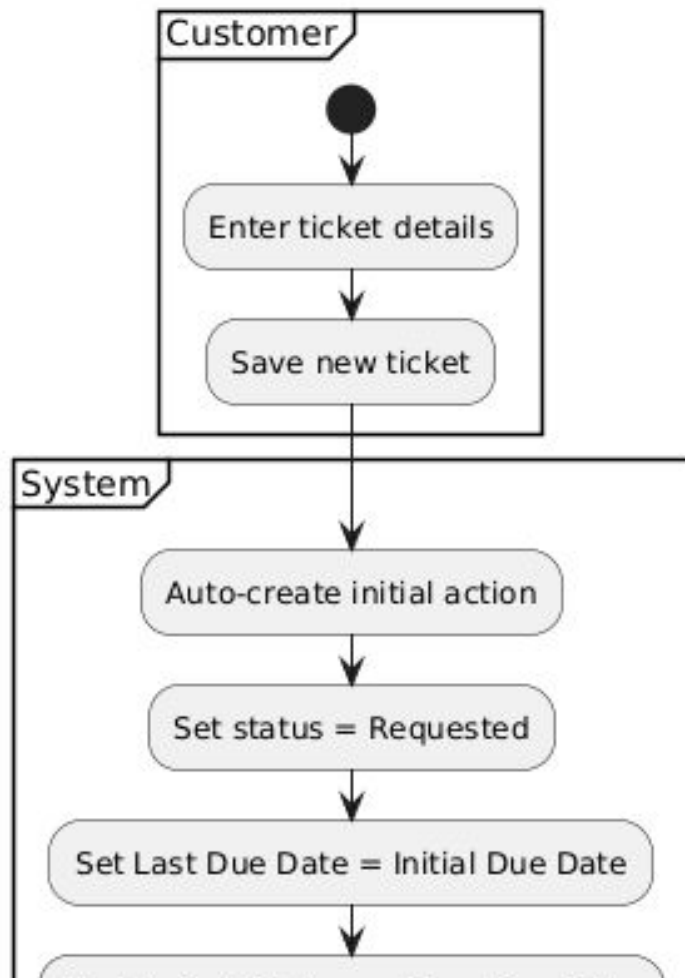
- Start with the **Customer** creating and saving a new ticket.
- The **System** auto-creates the initial action and sets the ticket status to *Requested*.
- The **CCE** reviews Requested tickets and either **approves** them (status → *Approved*) or **closes by CCE** (status → *Closed by CCE*).
- The **Programmer** works only on *Approved* tickets, adding *In Progress* actions until finishing with *Closed by Programmer*.
- After every action, the **System** updates the ticket's Last Status and Last Due Date.
- End when the ticket is *Closed*.

Activity for Ticket Lifecycle

Ticket Lifecycle - Activity Diagram (Swimlanes)



Ticket Lifecycle - Activity Diagram (Swimlanes)

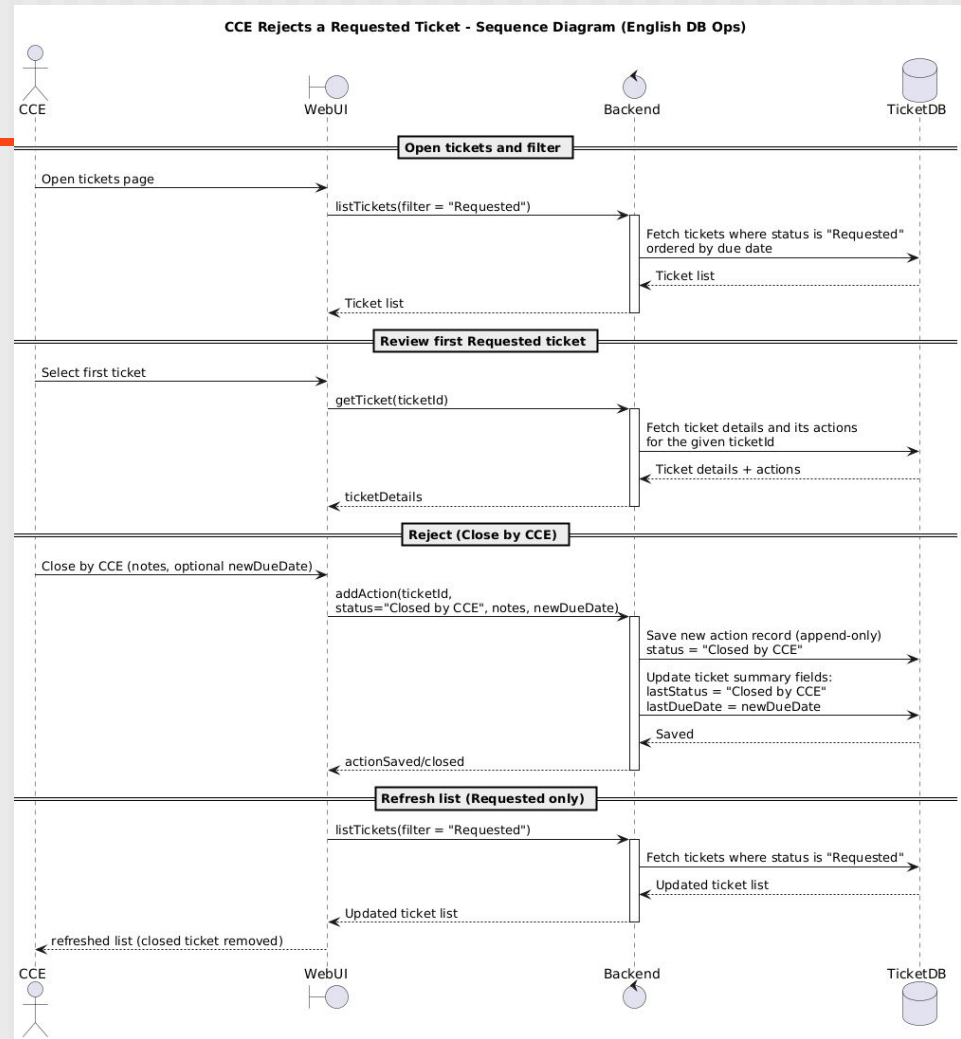


Sequence Diagram

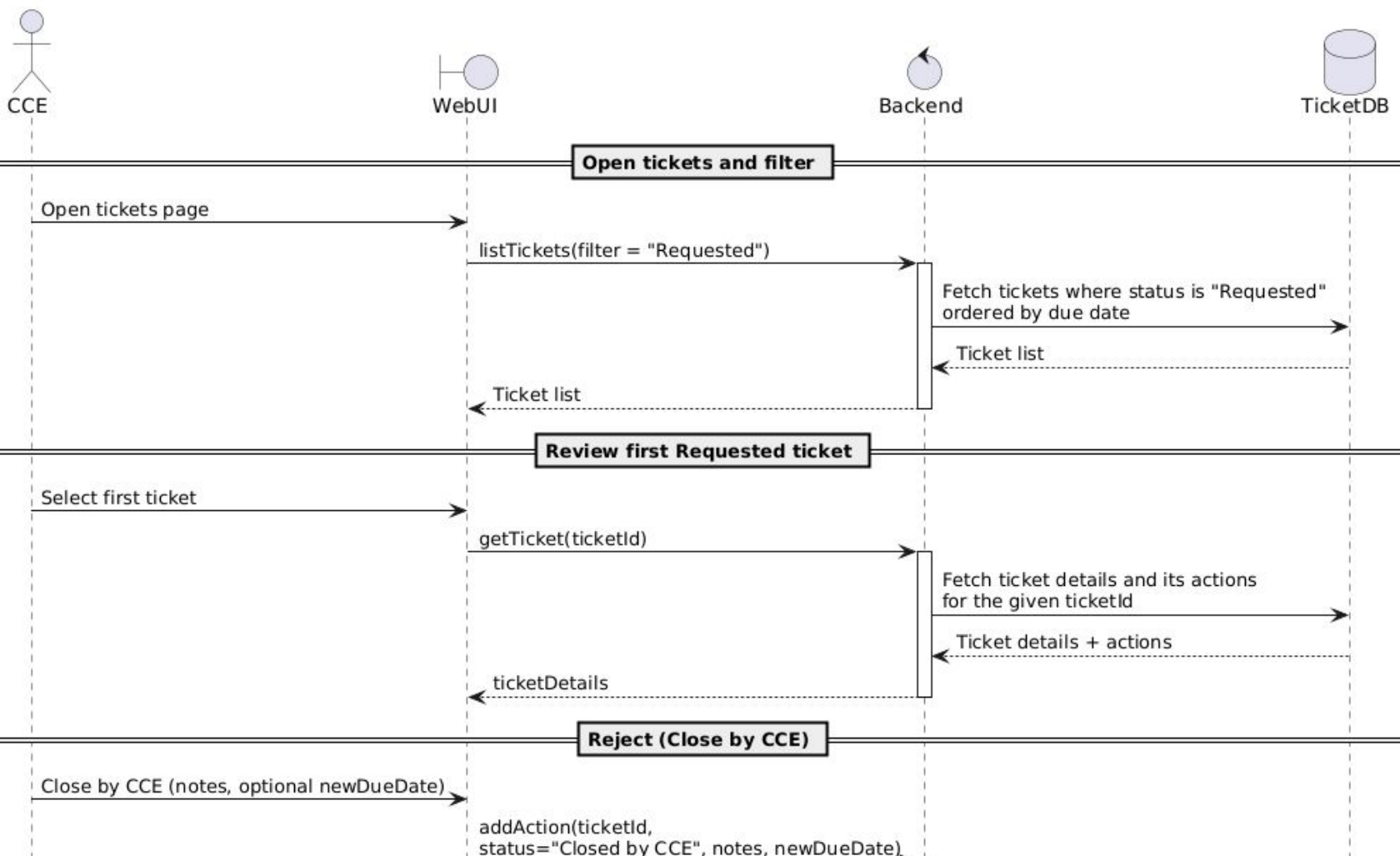
Sample Problem: Draw a Sequence Diagram for this scenario:

Assume the CCE is already authenticated. The CCE opens the tickets list view, filters tickets with status Requested, reviews the first such ticket, and rejects it (Close by CCE). The system makes the necessary changes as per requirements.

Sequence Diagram



CCE Rejects a Requested Ticket - Sequence Diagram (English DB Ops)



State Diagram

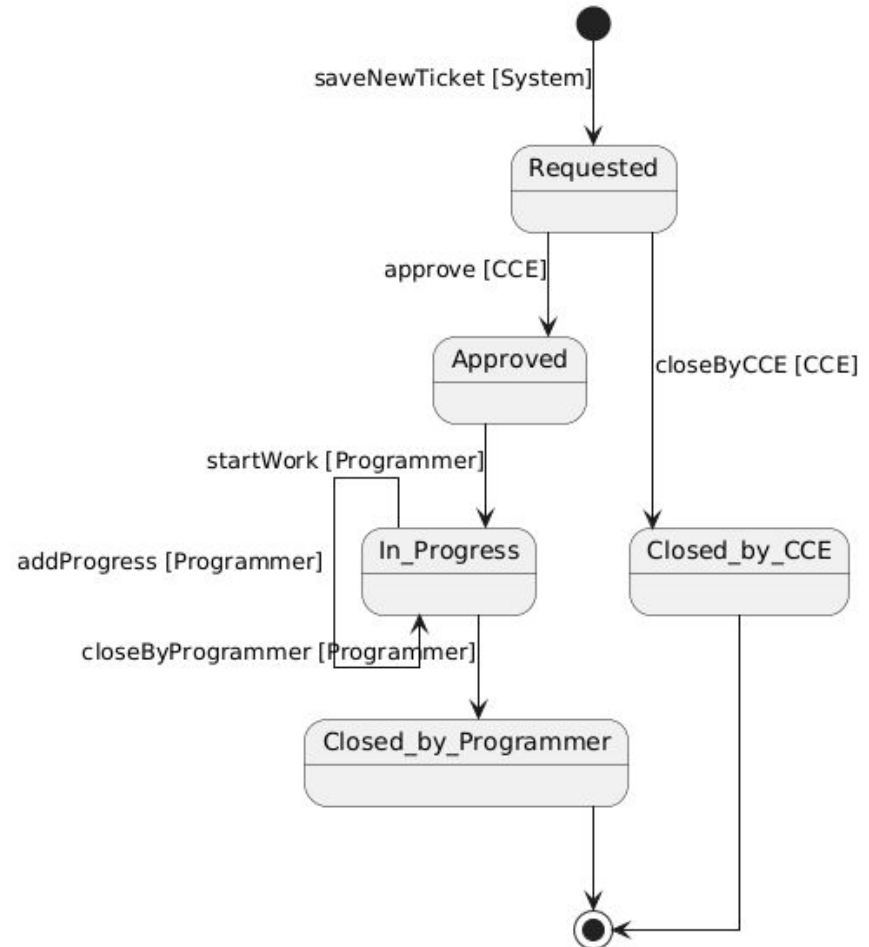
Sample Problem. Draw a **UML State Diagram** for a **Service Ticket** that models only its **status** change based on the requirements provided. Clearly show start and end states. Label each transition with the triggering event and (in brackets) the actor, e.g., **approve [CCE]**.

Solution:

- Initial state: when a ticket is saved, status becomes Requested (system sets this automatically).
- CCE actions (on Requested state only)
- Programmer actions on Approved and In Progress States
- Once closed (by CCE or Programmer), no further actions are allowed.

UML State Diagram

Ticket Status - UML State Diagram



University Grades Submission Example

For Krungthep University (KU)

Online Grades Submission Functional Requirements

Objective: Enable instructors at Krungthep University (KU) to submit student grades online instead of using paper forms.

Key Functions for Instructors

1. **Secure Login**
 - Instructors log in with their KU account.
 - Only valid accounts can access the system.
2. **View Courses Offered**
 - Instructors can see the list of courses and sections offered in their department for a chosen semester.
3. **View Registered Students**
 - For a selected course and section(s), instructors can see which students are registered.
 - This list can be brought directly into the online grade sheet.
4. **Maintain Course Grade Sheet**
 - Each grade sheet has a header (Course, Semester) and a list of registered students.
 - Instructors enter or update the grade for each student. Initially it is in draft mode.
 - Instructors may also print a copy of the grade sheet for their own records.

Online Grades Submission Functional Requirements

5. Submit Grades

- Once all grades are entered, the instructor changes from the sheet from **Draft** mode to **Ready** mode.
- Instructor can then submit the grade sheet (can only submit if in **Ready** mode).
- On submission:
 - Student transcripts are updated automatically.
 - Each student is notified of their grade by email.
- The system confirms that submission was successful.
- The grade sheet status is now in **Submitted** mode.

External Systems:

- **KU Single Sign-On:** Used to verify instructor login and manage passwords.
- **KU Registration System:** Supplies lists of offered courses and registered students.
- **KU Transcript System:** Records the official grades for each student.
- **KU Student Information System:** Provides student contact details (e.g., email).
- **KU's Email Service via Google Workspace:** Sends grade notifications to students.

Grade Submission UI.

Course Grade Sheet List View Generic Design (Minimalist for Exam)

Course List View Generic Design

User: Ying Kesuka Role: Instructor

Select Course Reports Logout

[Close]

For Semester: [LoV] - Default will be current semester

Owner Department: [LoV]

Search:

[FILTER][CLEAR]

Select a Row to View/Enter/Edit Grades for that Course-Section:

Course ID	Course Code	Course Name	Section	Semester	Grades Status
1933	CPE100	Intro Programming	1	2025-1	Draft
1935	CPE100	Intro Programming	2	2025-1	-
2115	CPE211	Hands on Computing	1	2025-1	Draft
2116	CPE211	Hands on Computing	2	2025-1	Submitted
2117	CPE211	Hands on Computing	3	2025-1	Ready
2118	CPE211	Hands on Computing	4	2025-1	-

Sample Exam Problem: Show Courses List View UI. It allows users to select the semester and department. The list of courses then appears for user to view/enter/edit grades for that course. Each Course is: Course Code + Section + Semester.

Grade Submission UI.

Course Grade Sheet Form View Generic Design (Minimalist for Exam)

Course Grade Submit Sheet Form View Generic Design

User: Ying Kesuka Role: Instructor

[Submit Grades](#) [View Courses](#) [Logout](#) [Reports](#)

[Close][Save][Clear] (Form Controls)

Grade Sheet No.: Entry Date: [Date]
Submitted by User: *User Role:
For Semester: *For Course-Section:
Number Students: Current State: [Change to Ready]
Grades Submitted Date: (autoset after Submit: Draft to Ready to Submit)
Instructor Remarks:

Enter Score and Grades for Students in this Course/Section:

#	Student ID	Student Name	Degree Program	Registration Type	Total Score	Grade
1	680112	Ake Amhit	BE. Computer Engineering	Regular (A to F)	97.00	_A_
2	680191	Bruno Baabin	BE. Computer Engineering	S/U	36.00	_U_
3	681012	Cindi Saetan	BS. Computer Science	Regular (A to F)	82.00	_B_
4	680133	Direk Dumrong	BE. Computer Engineering	Regular (A to F)	76.00	_C_
5	680201	Ellie Esterfan	BE. Computer Engineering	Regular (A to F)	51.00	_F_

Sample Exam Problem:

Show Grades Submit Form View for the instructors to enter the total score and grade for each student for a course (code + section + semester). Assume the course is already selected from the Course List View before coming to this form view.

Exam Problems from 2025 Midterm

UML Diagram Sample Problems

Example Exam Problem from Last Year

UML Example Exam Problem

1. Use the following requirements to complete various problems in this section

Requirements: KU Course Registration System

- | | |
|--|---|
| 1. All users must log in with their KU account through KU Single Sign-On. User roles such as student, university admin, and instructor are provided by this system. | 1. user ทุกคนต้อง log in ด้วย KU account ผ่าน KU Single Sign-On โดยที่ User roles เช่น student, university admin และ instructor จะมาจากระบบนี้ |
| 2. Department Admins can create new course information: course code, course name, description, credits, and fee. | 2. Department Admins สามารถสร้างข้อมูล course ใหม่ได้: course code, course name, description, credits และ fee |
| 3. Department Admins can set course offerings by first selecting a semester (e.g., 2025-1) and course code (e.g., CPE334). They then enter a sections list with details: section number, room, class time (e.g., “M W F 10:30–12:00”), display notes (e.g., ผู้สอน, หมายเหตุ), and maximum students allowed. For example, if CPE334 is offered in semester 2025-1 with sections 1, 2, 3, and 4, the system will create four course offerings for CPE334. | 3. Department Admins สามารถกำหนด course offerings ได้โดยเลือก semester (เช่น 2025-1) และ course code (เช่น CPE334) ก่อน จากนั้นกรอก sections list พร้อมรายละเอียด: section number, room, class time (เช่น “M W F 10:30–12:00”), display notes (เช่น ผู้สอน, หมายเหตุ) และจำนวน maximum students allowed ตัวอย่างเช่น ถ้า CPE334 เปิดใน semester 2025-1 พร้อม sections 1, 2, 3 และ 4 ระบบจะสร้าง course offerings สำหรับ CPE334 ทั้งหมด 4 รายการ |

 agents_md.zip Lecture 4 UML Example Exam Problem

UML Sequence Diagram with Swimlanes

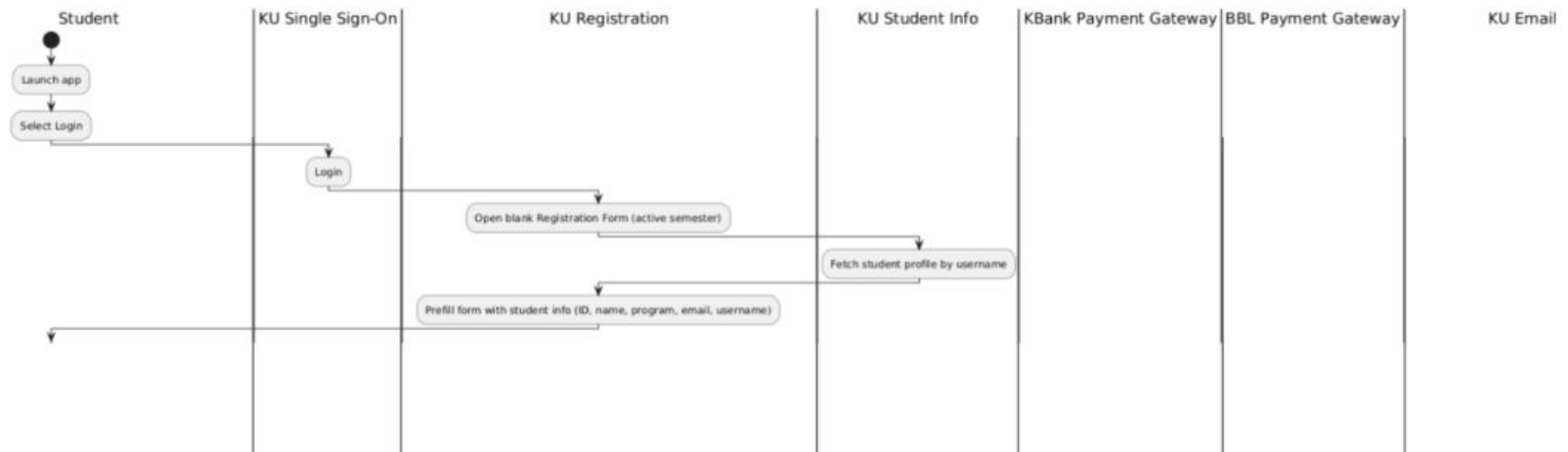
1.1 10 points. Complete this sequence diagram with swimlanes for the following end-to-end scenario:

1. A student logs in.
2. The system detects that the student already has a Draft Registration Form for the current semester containing two course offerings, so the student is presented with that form to continue.
3. The student adds one more course offering.
4. The student submits the form, but one of the three course offerings is full so submission fails.
5. The student is informed.
6. The student changes that course offering to a different section and submits again.
7. This time the submission is successful.
8. The student pays by bank transfer.
9. The registration is completed successfully.



UML Activity Diagram with Partitions

1.2 10 points. Complete the UML Activity Diagram with partitions (swimlanes) shown below. It models the main registration workflow in which a student logs in for the first time this semester, receives a blank registration form containing their information, selects courses, pays successfully, and receives confirmation. Be sure to cover all decision branches (e.g., payment method) and loops (e.g., resubmit until confirmation is received) using simple flowchart notation for decision nodes. The workflow represents a single session and ends with the student successfully registered.



UI Prototype

1.3 10 points. Design a generic UI for the Student Registration Form View. Assume the semester is fixed to the current semester and student information is loaded from the logged-in user. Show required UI fields using leading “*”, editable fields as textboxes, non-editable fields as striped textboxes as shown in the example below.

[Save] [Clear] [Close]

Invoice No. *Invoice Date: [Date]

*Customer Code: [LoV] Customer Name:

Notes:

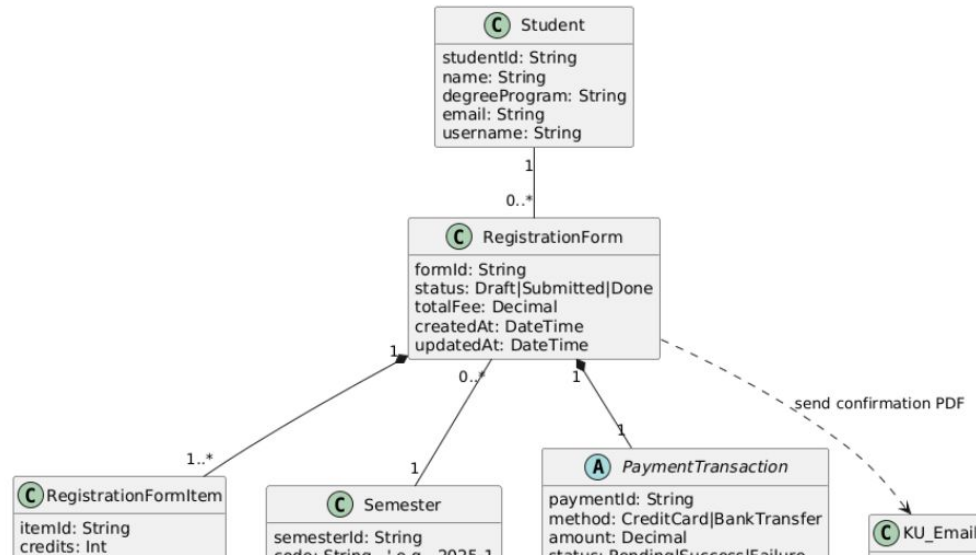
	*Product Code	Name	*Quantity	Unit Price	Price	
1	[LoV]					x ▲
2						x □
3						x ▼

Total Due:

UML Class Diagram

1.4 10 points. Draw the Class diagram for the data model used to store the Registration Form and its immediate (first-level) links to other classes. Do not include second-level links (i.e., anything more than one hop away from Registration Form's Header and Line Item classes). Show attributes, external calls, associations, multiplicities, inheritance, and composition. Do not show the class methods.

Registration Form Data Model — 1st-Level Links Only



Exam Problems from My Lectures

Given a Minimalist Functional Requirements List:

1. Prototype UI Design of List and Form Views for a Given Component
2. Draw a UML Use Case Diagram
- ~~3. Draw a UML Component Diagram~~
- ~~4. Draw a UML Package Diagram~~
5. Draw a UML Class Diagram
6. Draw UML Activity Diagram for a Given Workflow
7. Draw a UML Sequence Diagram for a Given Scenario in a Workflow
8. Draw a UML State diagram for a Given Component

Please familiarize yourself with my example Requirements

1. POS
2. All-in Hotel Reservation
3. Software Service Ticket
4. University Grades Submission

They may be used in exam problems for you to add some features or draw certain UML diagrams. You will save time by not having to read the same requirements during in the exam.

The above 4 Requirements List will be in English only. If I use any other Requirements List, it will be provided in both Thai and English.

Beyond Vibe Coding to Agentic Engineering

Transforming Intent into Professional Software

The Paradigm Shift for Generative AI Coding

From Chatting to Engineering

Generative AI has lowered the entry barrier, but enterprise software requires **rigor** over "vibes."

- ▶ Move from conversational fixes to **Spec-Driven Development**
- ▶ Automate the verification of architectural rules
- ▶ Human role: **Systems Director & Auditor**



Vibe Coding vs. Agentic Engineering

SDD: spec-driven development

TDD: test-driven development

Vibe Coding	Agentic Engineering (SDD/TDD)
Philosophy: "Just keep prompting until it works."	Philosophy: "Define the contract, then build to it."
Artifacts: Ephemeral chat history.	Artifacts: Living Specs (Markdown) and Unit Tests.
Risk: Hallucinations, breaking old features (regression).	Safety: Automated verification loops catch errors early.
Best for: Prototypes, UI demos, scripts.	Best for: Enterprise ERPs, APIs, production apps.

What is an Engineering Contract?

A precise, testable promise about how software behaves.

Core meaning

Given this input and system state, the software must produce this result, persist these changes, and report these errors.

A complete engineering contract specifies:

Inputs

Validations

Behavior

Persistence

Side effects

Errors

Outputs

Test cases

The contract is verified through:

ERP example: Create Invoice

Input customer_code, invoice_date, line_items[]

Validation customer exists; qty > 0; price >= 0

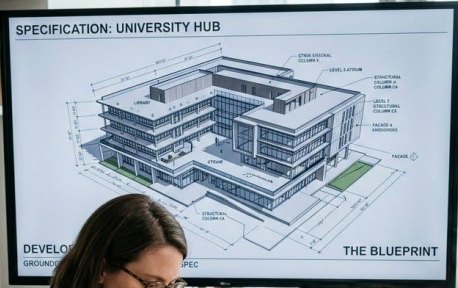
Behavior calculate subtotal, tax, and total

Persistence save Invoice and Invoice Line rows in one transaction

Error reject invalid lines; save nothing

Turns directly into tests:

creates valid invoice | rejects missing customer |
rolls back invalid line | totals are correct



Instead of direct prompting, we anchor development in a persistent **Specification (PRD)**.

- ▶ Agents consult the **Spec.md** for every decision
- ▶ Decouples high-level **Planning** from execution
- ▶ Maintains "Single Source of Truth" across files

The Right Approach



Wrong

Bad approach:

Ask AI to build an app.



Correct

Better approach:

Ask agents to satisfy a verifiable contract.

Engineering approach



Spec



Task
Contract



Code



Deterministic
Tests



Agent
Audits



Human
Review



Commit

Our Agentic Software Engineering Team

Architect

Translates Thai visuals and .md specs into an English **Technical Manifest**.

Coder

Generates raw *C#* and CSS based on the Architect's specific task list.

Logic Tester

Verifies id PKs, rowversion, and snake_case properties.

UI Auditor

Checks visual fidelity: Zen Green theme, English-only labels, and numeric right-alignment.

The Human Supervisor

HITL
Human-In-The-Loop

You are the Orchestrator

You don't write code; you **Review and Approve Manifests**. The agents propose the "How," but you own the blueprint.

This is the evolution of the software engineer: from **manual coder** to **systems director**.

Lab 2. TokTickIT

Sprint 2

Lab 1. Full Stack Hello World Starter.



CPE 334 Introduction to Software Engineering in the Age of AI Agents

Sections 1, 2, HS, 31 and 32. Semester: 1/2026.

Lab 1. TokTickIT (ตอกตึกกิต) Full-Stack Hello World Starter.

Score: ____ / 40

3. Lab 1 Request from Stakeholder

Here the stakeholder refers to the product owner, customer, or end user requesting this application.

“Before developing features for this ticketing app, the IT department wants evidence that the proposed technology stack works as one integrated system. Create a basic TokTickIT application that displays the service status and the supported request categories stored in the database.”

TokTickIT IT Service Desk

[Check System]

System Status: Online

Supported Request Categories:

- Account and Access
- Hardware
- Software
- Network

Lab 2. Requester Ticketing MVP with UI Foundations

CPE 334 Introduction to Software Engineering in the Age of AI
Sections 1, 2, HS, 31 and 32. Semester: 1/2026.

Lab 2. TokTickIT Requester Ticketing MVP with UI Foundations

Figure 1. Illustrative TokTickIT Ticket Detail screen using the Zen Green template design language. Lab 2 implements the Requester-facing Create Ticket, My Tickets, Ticket Detail, and Attachment functions. Later labs will add role-specific IT Staff controls, communication, workflow, and Actions Taken.

TokTickIT My Tickets Create Ticket Profile

My Tickets > Ticket Details Back to My Tickets

Ticket No. TKT-2025-001234	Ticket Date May 12, 2025 09:14 AM	Category Hardware	Related System Corporate Laptop
Requester Jennifer Anderson	Requested Priority Medium	IT Priority Medium	Current Status In Progress
Ticket Owner Michael Brown (IT Support)	Summary Laptop battery drains quickly		

Description
My laptop battery is draining much faster than usual even when the system is idle. This started happening after last week's Windows update.

Resolution Summary
No resolution summary available yet.

Public Comments 3 **Attachments** 2 **Service Actions** 1 **Event Log** 6

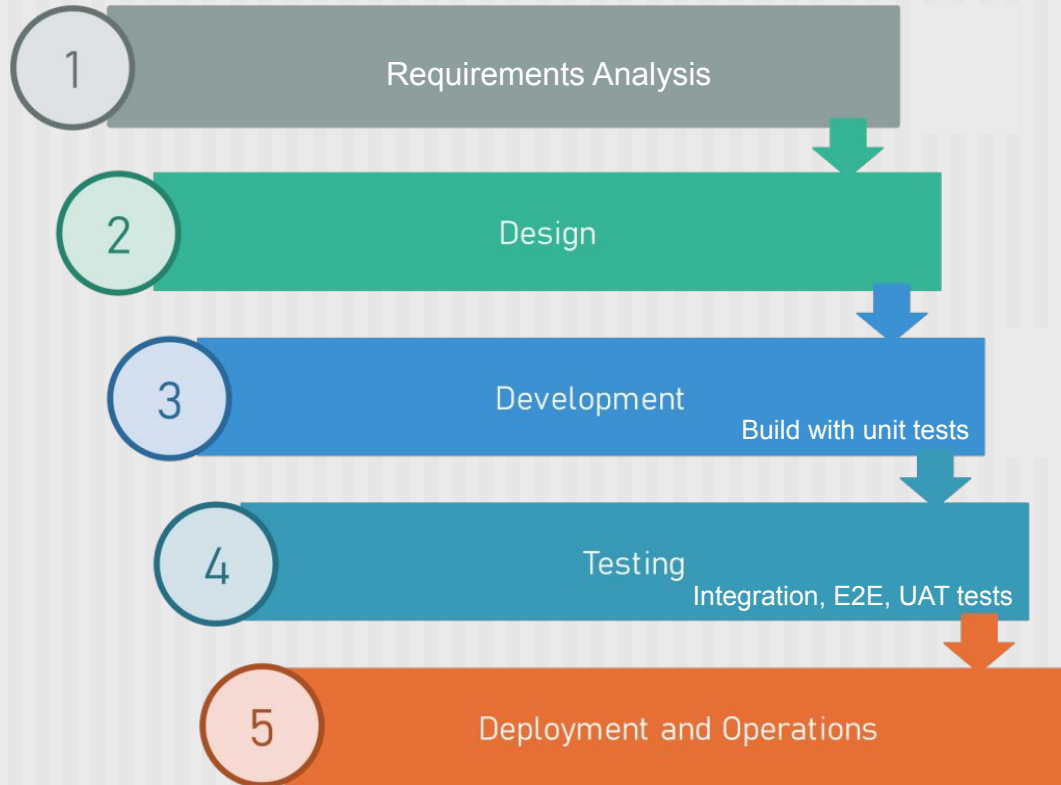
Add Comment
Type your comment here... Post Comment

JA Jennifer Anderson (Requester) May 13, 2025 11:45 AM
Thank you for the update. Please let me know if you need any additional information.

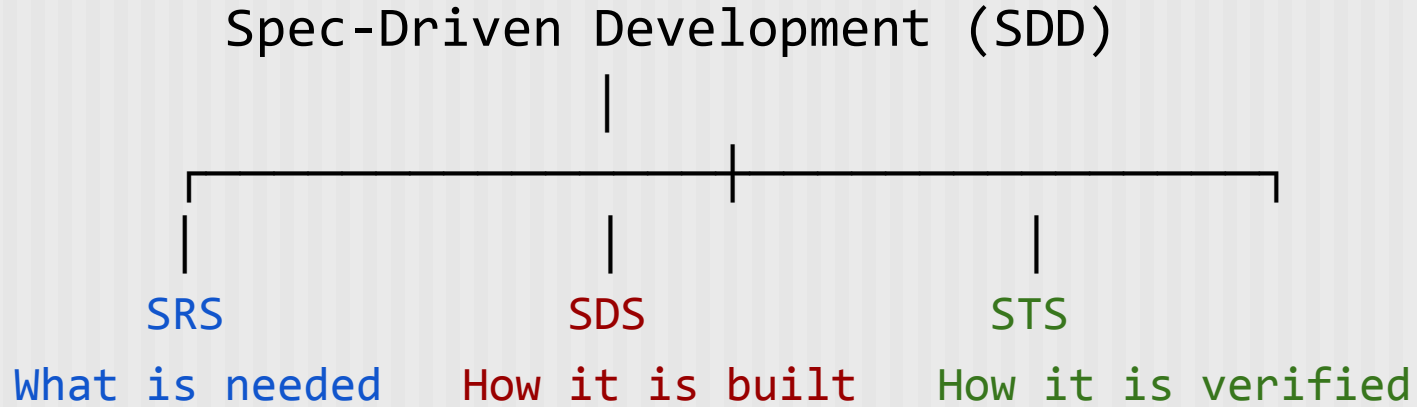
MB Michael Brown (IT Support) May 13, 2025 10:30 AM
We are investigating the issue on your device. We'll update you shortly.

JA Jennifer Anderson (Requester) May 12, 2025 09:20 AM
Just adding that this issue occurs even when I close all applications.

Anatomy of a Software Project



Spec-Driven Development (SDD) for Coding Agents



SRS - software requirements specification

SDS - Software Design Specification

STS - Software Test Specification

Recommended Design Process

1. **SRS.** FR, BR, NFR.



2. **Complete Feature Inventory.** Feature-A, Feature-B, Feature-C, ...



3. **Requirements-to-Features Traceability.** Map every FR, BR, and NFR to features. FR-003 → Feature-E.



4. **System-Level SDS.** Architecture, tech stack, UI standards, API conventions, security, deployment



5. **Select the Sprint Vertical Slice.** Choose a “Feature Bundle” to be implemented in the sprint



6. **Feature-Level Engineering Contract:** SDS + UI/API/Data Specs + Acceptance Criteria + STS



7. **AI Agent Implementation.** Implement one bounded Issue at a time according to the specifications



8. **Test and Human Review.** Run automated tests, inspect the code and Git diff, correct defects.



9. **Pull Request and Integration.** Review, approve, and merge feature branch into the Sprint staging branch



10. **Sprint Integration, System Testing, and UAT.** Run tests on Sprint staging branch. If PASS: Pull Request from Sprint staging → main. If FAIL: create a fix branch on Sprint staging branch, update the SDS/STS if needed, perform Steps 7-9, and repeat Step 10.



11. **Per Release: Deploy main Branch to Production**

per feature

1. The Software Requirements Specification (SRS) Document

$$\text{SRS} = \text{BR} + \text{FR} + \text{NFR} + \text{supporting context}$$

The requirement catalogue

BR Business goals and outcomes

FR Functions and system behavior

NFR Quality attributes and constraints

These requirements form the core of the SRS.

Purpose & scope

Business context, objectives, system boundary, and exclusions.

Actors & interfaces

Users, external systems, inputs, outputs, and interactions.

Assumptions & constraints

Dependencies, operating conditions, policies, and technical limits.

Acceptance & traceability

Testable criteria and links from requirements to design and tests.

Requirements List (minimalist)

1. **Serve customers at the counter**
 - Cashiers can start a checkout, add or remove items, apply valid discounts, and see the running total.
 - A sale can be **cancelled** any time before it is finished.
2. **Accept payments**
 - Customers can pay **by cash** or **by credit card** which is processed via a **Payment Service Provider** (eg., K-Bank)
 - The system **works with the card machine** from, say, K-Bank to send the amount and receive approval
 - If a card is **declined**, the cashier can retry, switch to cash, or cancel the sale.
3. **Print documents: Receipt** for successful sale, **Refund Receipt** for refunds. Prints to store **receipt printer**.
4. Updates stock in **store's inventory service** app: **reduces** stock of the sold item, adds back stock of returned items.
5. **Handle returns and refunds.**
 - Cashiers or managers can find the original sale and **process a return**.
 - The system **refunds** the customer (cash or card, following store policy) and records the reason.
6. **Manager Functions**
 - Managers can view a **daily sales summary** for a selected date, including total cash & card amount received.
 - Managers can **add or update products and prices** to be used by POS.

SRS Prompt Session Example



<https://docs.google.com/document/d/1Y87HZ0RwSAUjcm8pPtiZ-s5-iOuZmo7L3uZXpuleovI/edit?usp=sharing>

For this chat, assume I'm a software engineering student wanting your help in creating good SE artifacts such as Requirements Specifications, Design Specs, Test Scenarios, etc.

I have these requirements provided to me for a POS system by my instructor. I have to iteratively refine it until they become numbered FR, BR, and NFR. First, please read this and ask me questions about parts of this POS system that may need more clarification before you can draft the requirements lists.

1. **Serve customers at the counter**
 - Cashiers can start a checkout, add or remove items, apply valid discounts, and see the running total.
 - A sale can be **cancelled** any time before it is finished.
2. **Accept payments**
 - Customers can pay **by cash** or **by credit card** which is processed via a **Payment Service Provider** (eg., K-Bank)
 - The system **works with the card machine** from, say, K-Bank to send the amount and receive approval
 - If a card is **declined**, the cashier can retry, switch to cash, or cancel the sale.
3. **Print documents: Receipt** for successful sale**, Refund Receipt** for refunds. Prints to store **receipt printer**.
4. Updates stock in **store's inventory service** app**: reduces** stock of the sold item, adds back stock of returned items.
5. **Handle returns and refunds. **
 - Cashiers or managers can find the original sale and **process a return**.
 - The system **refunds** the customer (cash or card, following store policy) and records the reason.
6. **Manager Functions**
 1. Managers can view a **daily sales summary** for a selected date, including total cash & card amount received.
 2. Managers can **add or update products and prices** to be used by POS.

Recommended Design Process

1. **SRS.** FR, BR, NFR.



2. **Complete Feature Inventory.** Feature-A, Feature-B, Feature-C, ...



3. **Requirements-to-Features Traceability.** Map every FR, BR, and NFR to features. FR-003 → Feature-E.



4. **System-Level SDS.** Architecture, tech stack, UI standards, API conventions, security, deployment



5. **Select the Sprint Vertical Slice.** Choose a “Feature Bundle” to be implemented in the sprint



6. **Feature-Level Engineering Contract:** SDS + UI/API/Data Specs + Acceptance Criteria + STS



7. **AI Agent Implementation.** Implement one bounded Issue at a time according to the specifications



8. **Test and Human Review.** Run automated tests, inspect the code and Git diff, correct defects.



9. **Pull Request and Integration.** Review, approve, and merge feature branch into the Sprint staging branch



10. **Sprint Integration, System Testing, and UAT.** Run tests on Sprint staging branch. If PASS: Pull Request from Sprint staging → main. If FAIL: create a fix branch on Sprint staging branch, update the SDS/STS if needed, perform Steps 7-9, and repeat Step 10.



11. **Per Release: Deploy main Branch to Production**

per feature

2. Complete Feature Inventory: POS

ID	Feature	Feature scope
Feature-A	User Authentication and Access Control	Sign-in, user roles, function permissions, and store-based access restrictions for Cashiers, Managers, Administrators, and Head-Office Managers.
Feature-B	Store and Terminal Operating Context	Identification of the current store and POS terminal, association of transactions with that context, and support for multiple stores and terminals.
Feature-C	Product Catalog and Price Management	Addition and modification of products, product information, VAT-inclusive selling prices, and manager access restrictions.
Feature-D	Sale Lifecycle Management	Starting a sale, maintaining its current state, cancelling it before completion, and preventing modification after completion.
Feature-E	Cart and Sale-Item Management	Adding products, changing quantities, removing products, and recalculating the sale when its items change.
Feature-F	VAT-Inclusive Pricing and Total Calculation	Calculation and display of item amounts, subtotal, Thai VAT information, discount, and final sale amount using Thai baht and satang.

2. Complete Feature Inventory: POS

ID	Feature		Feature scope
Feature-G	Order-Level Management	Discount	Discount Percentage and Discount Amount entry, percentage-to-amount calculation, satang rounding, manual discount-amount handling, validation, and recalculation when the cart changes.
Feature-H	Payment Selection	Method	Selection of either cash or credit card, enforcement of the no-split-payment rule, and permitted switching of payment methods.
Feature-I	Cash Payment Processing		Entry of cash tendered, validation that sufficient cash was received, change calculation, and cash-sale completion.
Feature-J	Card Authorization	Payment	Creation of a unique payment-attempt reference, durable recording of the attempt, communication with the card machine and Payment Service Provider, and handling of approved or declined results.
Feature-K	Card Payment Recovery and Reconciliation		Handling communication interruptions, unresolved payment results, approved payments that could not be finalized, PSP status queries, duplicate-charge prevention, and recovery of affected sales.
Feature-L	Sale Finalization and Durable Recording		Validation of successful payment, assignment of a unique sale identifier, permanent recording of the sale, and atomic creation of the associated pending inventory update.

2. Complete Feature Inventory: POS

ID	Feature	Feature scope
Feature-M	Receipt Printing and Reprinting	Printing a receipt after successful sale completion, reporting printer failures, and reprinting without duplicating or reversing the sale.
Feature-N	Inventory Update Outbox	Creation and durable storage of a pending inventory-update record containing the sale, store, terminal, date and time, product, and quantity information.
Feature-O	Inventory Message Delivery	Asynchronous message transmission, acknowledgement processing, retained-message retry, and duplicate inventory-update prevention through unique transaction identifiers.
Feature-P	Operational Exception Monitoring	Viewing unresolved card payments, payment-recovery cases, pending inventory messages, and inventory messages that have repeatedly failed delivery.
Feature-Q	Daily Sales Reporting	Selection of a business date, assigned-store reporting for Managers, selected-store reporting for Administrators and Head-Office Managers, consolidated reporting, and separate cash and card totals.
Feature-R	Audit Trail	Recording the responsible user, store, terminal, date, and time for completed sales, cancellations, discounts, card-payment recovery, and product or price changes.

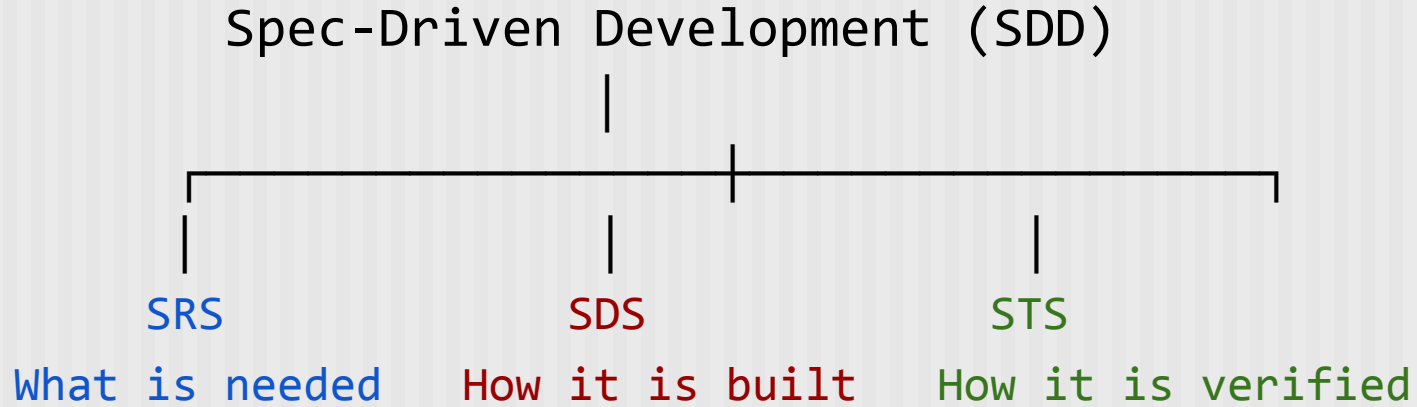
3. Requirements-to-features traceability



Lecture 3 ChatGPT Sessions

<https://docs.google.com/document/d/1Y87HZ0RwSAUjcm8pPtiZ-s5-iOuZmo7L3uZXpuleovI/edit?usp=sharing>

Spec-Driven Development (SDD) for Coding Agents



SRS - software requirements specification

SDS - Software Design Specification

STS - Software Test Specification

4. Contents of SDS Document: System-Level

I. **System-Level SDS.** This defines shared technical decisions that apply across the application:

- System architecture
- Technology stack
- Major components and modules
- Overall data model
- Authentication and authorization approach
- UI Design System Specification
- API conventions
- Error-handling conventions
- Security, performance, logging, and other cross-cutting NFRs

Contents of SDS Document: Feature Level

II. For **Each Feature** , create a specification containing:

1. Feature purpose and scope
2. Related SRS requirements
3. Workflow and state transitions
4. Business rules
5. Data model changes
6. API endpoints
7. UI behavior
8. Permissions
9. Validation and error handling
10. Acceptance criteria
11. Test considerations

UI Design Specifications

- Theme and color palette
- Semantic color usage, such as success, warning, error, and information
- Typography and heading hierarchy
- Spacing and layout grid
- Buttons, forms, tables, cards, and other shared components
- Icons and imagery
- Responsive breakpoints
- Accessibility requirements, including contrast and keyboard navigation
- Light/dark mode, if supported
- Standard loading, empty, confirmation, and error states

Document	Example
SRS/NFR	NFR-UI-01: The application shall conform to the company's corporate branding guidelines.
SRS/NFR	NFR-UI-02: All text and interactive controls shall meet WCAG 2.1 AA contrast requirements.
SDS	Primary color: #0057B8; secondary color: #F5A623; error color: #D32F2F.
SDS	Primary buttons use the corporate blue background with white text.
SDS	Headings use the corporate typeface; body text uses Arial as the fallback.

KMUTT Color Theme

https://www.kmutt.ac.th/en/corporate_identity/colors/

COLORS



PANTONE
172 C

C 0 M 85 Y 100 K 0
R 255 G 70 B 18



PANTONE
123 C

C 0 M 25 Y 90 K 0
R 255 G 198 B 39



PANTONE
415 C

C 46 M 38 Y 38 K 2
R 145 G 145 B 145

The Colors of the University

The Colors of the University are orange-yellow which signify dynamic movement and strength. Both colors also activate enthusiasm. These colors are comparable to dynamic movement of the apparatus and machinery which is a crucial part of the teaching and learning in the university.

Color schemes for website and application. :

Yellow : #FFC72C

Orange : #FA4616

Blue Grey : #7B8189

สีเหลือง

Yellow #FFC72C

สีแสด

Orange #FA4616

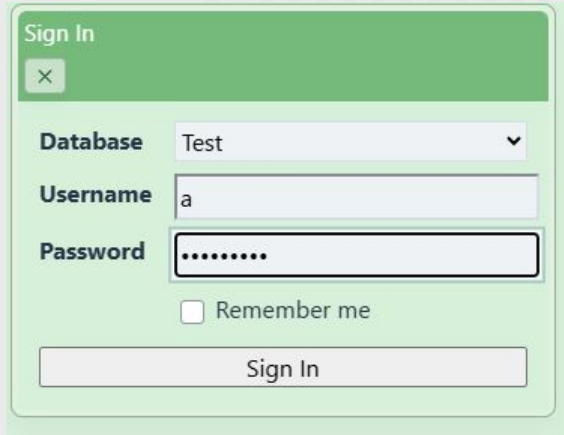
สีเทา

Blue Grey #7B8189

UI Specs Example

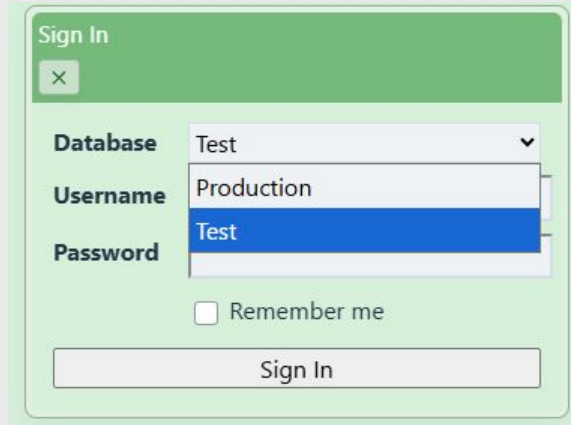
“Zen Green” UI

Login Screen



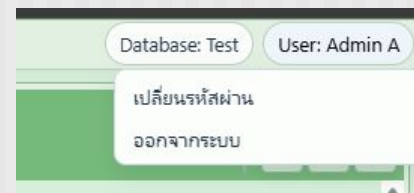
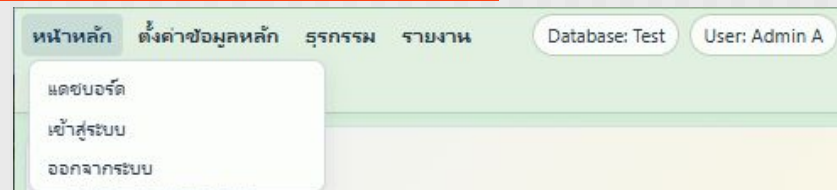
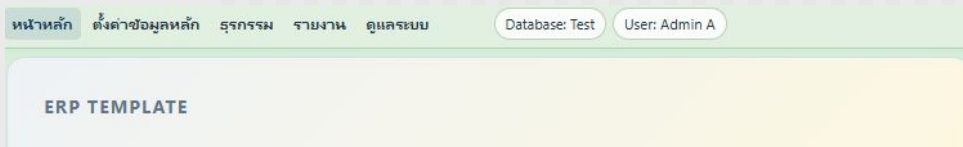
A screenshot of a login form titled "Sign In" with a close button (X). The form contains three input fields: "Database" with a dropdown menu showing "Test", "Username" with the text "a", and "Password" with masked characters ".....". Below the password field is a checkbox labeled "Remember me". At the bottom is a "Sign In" button.

Can choose which database to login. For example, Production, Staging, Feature A Dev.



A screenshot of the same login form, but the "Database" dropdown menu is open, showing a list of options: "Test", "Production", and "Test" (highlighted in blue). The "Username" field is empty, and the "Password" field is also empty. The "Remember me" checkbox and "Sign In" button are still present.

App Shell and Control Menu



Product Model List View

หน้าหลัก **ตั้งค่าข้อมูลหลัก** สูตรกรม รายงาน ดูแลระบบ Database: Test User: Admin A

ข้อมูลสินค้า

รหัสสินค้า	ชื่อสินค้า	ราคามาตรฐาน	หน่วย
มีคำว่า...	มีคำว่า...	100 หรือ 10...20	มีคำว่า...
P-A4-70G	A4 Paper 70gsm	145.00	Ream
P-A4-80G	A4 Paper 80gsm	165.00	Ream
P-CLIP-M	Paper Clip Medium	15.00	Box
P-ERS-WHT	White Eraser	6.00	Piece
P-NBK-A5	Notebook A5	35.00	Book
P-PCL-2B	Pencil No. 2	8.00	Piece
P-PEN-BLK	Ballpoint Pen Black	12.00	Piece
P-PEN-BLU	Ballpoint Pen Blue	12.00	Piece
P-RUL-30	Ruler 30 cm	18.00	Piece
P-STP-MINI	Mini Stapler	85.00	Piece

Page Size 50 First Prev 1 Next Last

Product Model Form View: View Mode

ข้อมูลสินค้า				View Mode		Existing Status			
      						  			
*รหัสสินค้า		*ชื่อสินค้า		*ราคามาตรฐาน		หน่วย			
P-A4-70G		A4 Paper 70gsm		145.00		Ream			

Invoice Model List View

หน้าหลัก ตั้งค่าข้อมูลหลัก **ธุรกรรม** รายงาน ดูแลระบบ Database: Test User: Admin A

ใบแจ้งหนี้

✕ +

เลขที่ใบแจ้งหนี้	วันที่	รหัสลูกค้า	ชื่อลูกค้า	จังหวัด	ประเทศ	มูลค่าสินค้า	ภาษีมูลค่าเพิ่ม
มีค่าว่า...	วว/ดด/ปปปป หรือ:	มีค่าว่า...	มีค่าว่า...	มีค่าว่า...	มีค่าว่า...	100 หรือ 10..20	100 หรือ 10..
SINV26-000001	30/04/2026	abc	Abc Co.		Thailand	18.00	
SINV26-000002	30/04/2026	abc	Abc Co.		Thailand	155.00	

Page Size 50 First Prev 1 Next Last

Invoice Model Form View: View Mode

Note the 2 styles of attribute-value header fields

ใบแจ้งหนี้ | View Mode | Existing Status

× + 📄 ✎ 📁 🗑️ 📄

🔄 🗨️ 🗑️

Running ID เลขที่ใบแจ้งหนี้ *วันที่เอกสาร *รหัสลูกค้า

2 SINV26-000002 30/04/2026 abc

ชื่อลูกค้า ที่อยู่บรรทัด 1 ที่อยู่บรรทัด 2 ที่อยู่บรรทัด 3

Abc Co.

จังหวัด ประเทศ

Thailand

มูลค่าสินค้า	VAT %	ภาษีมูลค่าเพิ่ม	ยอดชำระ
155.00	7.00	10.85	165.85

รายการสินค้า

#	* รหัสสินค้า	ชื่อสินค้า	* ราคาต่อหน่วย	* จำนวน	หน่วย	รวมเงิน
1	P-NBK-A5	Notebook A5	35.00	1.00	Book	35.00
2	P-PEN-BLK	Ballpoint Pen Black	12.00	10.00	Piece	120.00

Invoice Form Edit Mode

ใบแจ้งหนี้

Edit Mode

Existing Status

×

🖨

+

📄

🔍

🗑

🔄

✓

🔄

🗨

🔗

Running 1 รายการ

เลขที่ใบแจ้งหนี้

* วันที่เอกสาร

* รหัสลูกค้า

2

SINV26-000002

30/04/2026

abc

ชื่อลูกค้า

ที่อยู่บรรทัด 1

ที่อยู่บรรทัด 2

ที่อยู่บรรทัด 3

Abc Co.

จังหวัด

ประเทศ

Thailand

มูลค่าสินค้า

VAT %

ภาษีมูลค่าเพิ่ม

ยอดชำระ

155.00

7.00

10.85

165.85

รายการสินค้า

	# * รหัสสินค้า	ชื่อสินค้า	* ราคาต่อหน่วย	* จำนวน	หน่วย	รวมเงิน	
🗑 📄 🔄	1 P-NBK-A5	🔍 Notebook A5	35.00	1.00	Book	35.00	🗑 📄 🔄
🗑 📄 🔄	2 P-PEN-BLK	🔍 Ballpoint Pen Black	12.00	10.00	Piece	120.00	🗑 📄 🔄

Form Data Validation

Validation Check Message.

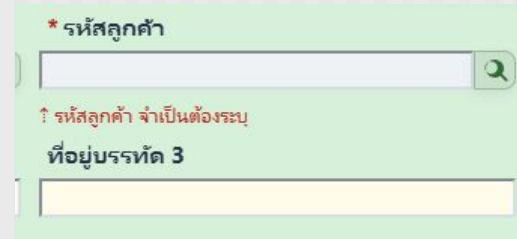


บันทึกรายการไม่สำเร็จ

ไม่สามารถบันทึกรายการได้ เนื่องจากมีข้อมูลบางช่องไม่ครบถ้วนหรือไม่ถูกต้อง กรุณาตรวจสอบความถี่และช่วงช่องที่มีข้อผิดพลาด

ตกลง

Validation Correction message uses a separate row of red characters for header fields.

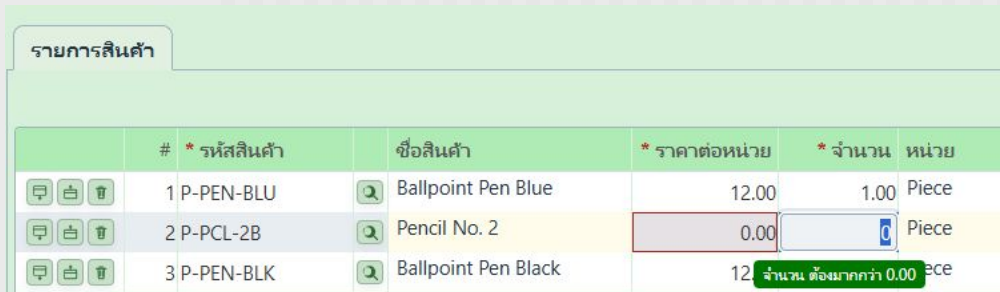


* รหัสลูกค้า

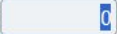
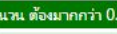
รหัสลูกค้า จำเป็นต้องระบุ

ที่อยู่บรรทัด 3

Validation Correction message for line item fields uses a tool-tip style textbox to display a floating message.



รายการสินค้า

	#	* รหัสสินค้า	ชื่อสินค้า	* ราคาต่อหน่วย	* จำนวน	หน่วย
  	1	P-PEN-BLU	 Ballpoint Pen Blue	12.00	1.00	Piece
  	2	P-PCL-2B	 Pencil No. 2	0.00		Piece
  	3	P-PEN-BLK	 Ballpoint Pen Black	12.00		Piece

จำนวน ต้องมากกว่า 0.00

Form Other Features

Header has 2 types of attribute-value fields, both responsive:

1. Regular fields where user specifies number fields (equally spaced) to display per row. Options are 1, 2, 3, and 4.
2. Tabular looking fields with attribute shaded green and font in white as if a table heading. Developer can specify as many fields as desired with width per field.

Required fields have leading “*” in red font.

Date Selector Button

The screenshot shows a form with two fields at the top: "วันที่เอกสาร" (Document Date) with a value of "30/04/2026" and a calendar icon, and "รหัสลูกค้า" (Customer Code) with a value of "abc". Below these is a section titled "ที่อยู่บรรทัด 2" (Address Line 2). A date selector button labeled "Select Date" is overlaid on the form, displaying a calendar for April 2026. The date "30" is selected. The button has "<" and ">" navigation arrows, a "Today" button, and a "Clear" button. The form background is light green with white text.

Tool Tips on buttons

The screenshot shows a green header bar with the text "ใบแจ้งหนี้" (Invoice) on the left, "Edit Mode" in the center, and "Existing Status" on the right. Below the header is a row of buttons: a close button (X), a save button (floppy disk), a plus button (+), a print button (printer), a delete button (trash), a refresh button (circular arrow), and a cancel button (X). Below the buttons is a label "Running ID" and a green button labeled "ลบ" (Delete).

There's Dirty Bit Check

The screenshot shows a confirmation dialog box with a green header bar containing the text "บันทึกรายการไม่สำเร็จ" (Record saving failed). The main text area contains the message: "ไม่สามารถบันทึกรายการได้ เนื่องจากมีข้อมูลบางช่องไม่ครบถ้วนหรือไม่ถูกต้อง กรุณาตรวจสอบความถี่แสดงข้างช่องที่มีข้อผิดพลาด" (Cannot save record because some data fields are incomplete or incorrect. Please check the frequency of display in the fields with errors). At the bottom right is a button labeled "ตกลง" (OK).

Form Style: Green Theme, changeable to Blue etc.

Green Theme. Changeable to Blue etc.

Same font size in each area for all themes. Rounded Corner all forms.

Editable vs. non-editable field shading in header and line item.

*วันที่เอกสาร	*รหัสลูกค้า
29/04/2026	abc
ที่อยู่บรรทัด 2	ที่อยู่บรรทัด 3

รายการสินค้า							
	#	*รหัสสินค้า	ชื่อสินค้า	*ราคาต่อหน่วย	*จำนวน	หน่วย	รวมเงิน
	1						0.00

The List of Value (LoV) button and LoV Form

#	* รหัสสินค้า	ชื่อสินค้า
1	P-NBK-A5	Notebook A5
2	P-PEN-BLK	Ballpoint Pen Black

Same form (code artifact) as the List View form for invoice, customer, product, etc. List View

ค้นหาสินค้า - Google Chrome

localhost:5018/Lists?id=product-lookup&mode=lookup&callback=invoiceProductLookup.receiveProduct

ค้นหาสินค้า

X +

รหัสสินค้า	ชื่อสินค้า	ราคามาตรฐาน	หน่วย
มีคำว่า...	มีคำว่า...	100 หรือ 10..20	มีคำว่า...
P-A4-70G	A4 Paper 70gsm	145.00	Ream
P-A4-80G	A4 Paper 80gsm	165.00	Ream
P-CLIP-M	Paper Clip Medium	15.00	Box
P-ERS-WHT	White Eraser	6.00	Piece
P-NBK-A5	Notebook A5	35.00	Book
P-PCL-2B	Pencil No. 2	8.00	Piece
P-PEN-BLK	Ballpoint Pen Black	12.00	Piece
P-PEN-BLU	Ballpoint Pen Blue	12.00	Piece
P-RUL-30	Ruler 30 cm	18.00	Piece
P-STP-MINI	Mini Stapler	85.00	Piece

Page Size 50 First Prev 1 Next Last

List of Value (Selection) Form Features

- Uses same, single form for all models
- Has 2 modes: Primary Key List View and Foreign Key selection List View.

ค้นหาสินค้า - Google Chrome

localhost:5018/Lists?id=product-lookup&mode=lookup&callback=invoiceProductLookup.receiveProduct

ค้นหาสินค้า

X +

รหัสสินค้า	ชื่อสินค้า	ราคามาตรฐาน	หน่วย
มีคำว่า...	มีคำว่า...	100 หรือ 10..20	มีคำว่า...
P-A4-70G	A4 Paper 70gsm	145.00	Ream
P-A4-80G	A4 Paper 80gsm	165.00	Ream
P-CLIP-M	Paper Clip Medium	15.00	Box
P-ERS-WHT	White Eraser	6.00	Piece
P-NBK-A5	Notebook A5	35.00	Book
P-PCL-2B	Pencil No. 2	8.00	Piece
P-PEN-BLK	Ballpoint Pen Black	12.00	Piece
P-PEN-BLU	Ballpoint Pen Blue	12.00	Piece
P-RUL-30	Ruler 30 cm	18.00	Piece
P-STP-MINI	Mini Stapler	85.00	Piece

Page Size 50 First Prev 1 Next Last

- Can sort a field
- Select fields containing words
- Select range for date and numeric fields
- Shows by pages (here 50 per page)
- Can Create New (+) if desired row not in list.
- Click to select
- Control display fields and column sizes

Invoice form showing responsive behavior

Note the adjustments of attribute-value header fields while line item fields have a left-right scroll.

ใบแจ้งหนี้ Edit Mode Existing Status

Running ID
2

เลขที่ใบแจ้งหนี้
SINV26-000002

* วันที่เอกสาร
30/04/2026

* รหัสลูกค้า
abc

ชื่อลูกค้า
Abc Co.

ที่อยู่บรรทัด 1
[Empty field]

ที่อยู่บรรทัด 2
[Empty field]

ที่อยู่บรรทัด 3
[Empty field]

ใบแจ้งหนี้ Edit Mode Existing Status

จังหวัด
[Empty field]

ประเทศ
Thailand

มูลค่าสินค้า	VAT %	ภาษีมูลค่าเพิ่ม
155.00	7.00	10.85

ยอดชำระ
165.85

รายการสินค้า

	#	* รหัสสินค้า	ชื่อสินค้า	* ราคาต่อ
	1	P-NBK-A5	Notebook A5	
	2	P-PEN-BLK	Ballpoint Pen Black	

Line Item UI Specs

1. Line items always in a named tab.

2. Has button to add row to bottom on upper right side of grid. Button as tooltip.

3. Has buttons add row below this line, add row above this line, delete this line on both the left and right sides of each grid row, each with tooltips.

4. Has ordered item number to preserve ordering.

5. In edit mode the selected row's editable fields use whitish color; non-editable fields creamish. Shows text, checkbox, and drop-down lists.

6. Has LoV button as a separate column. Same for Date Picker Button.

7. Required fields have leading red "**".

8. When in View mode rows of the grid show in alternating light/dark shades.

9. Validation errors show as red box with tooltip-style error messages.

No.	* รหัสสินค้า	ชื่อสินค้า	* ราคาต่อหน่วย	* จำนวน	หน่วย	รวมเงิน
1	P-PEN-BLK	Ballpoint Pen Black	12.00	1.00	Piece	12.00
2	P-STP-MINI	Mini Stapler	85.00	1.00	Piece	85.00

No.	* รหัสสินค้า	ชื่อสินค้า	* ราคาต่อหน่วย	* จำนวน	หน่วย	รวมเงิน
1	P-PEN-BLK	Ballpoint Pen Black	12.00	1.00	Piece	12.00
2	P-STP-MINI	Mini Stapler	85.00	1.00	Piece	85.00
3	P-CLIP-M	Paper Clip Medium	15.00	3.00	Box	45.00

No.	* รหัสสินค้า	ชื่อสินค้า	* ราคาต่อหน่วย	* จำนวน
1	P-PEN-BLK	Ballpoint Pen Black		1.
2	P-STP-MINI	Mini Stapler		1.

User Model List View

หน้าหลัก ตั้งค่าข้อมูลหลัก อีกรกรรม รายงาน **ดูแลระบบ** Database: Test User: Admin A

ผู้ใช้งาน					
ชื่อผู้ใช้	ชื่อที่แสดง	อีเมล	ใช้งาน	จำนวนบทบาท	จำนวนสิทธิ์รวม
มีค่าว่า...	มีค่าว่า...	มีค่าว่า...	ใช่/ไม่ใช่	1 หรือ 1..5	1 หรือ 1..20
a	Admin A	a@example.com	ใช่	1	24
admin	Administrator	admin@example.com	ใช่	1	24

Page Size 50 First Prev 1 Next Last

User Model Form View with 2 tabs

ผู้ใช้งาน

View Mode

Existing Status

×

+

📄

✎

🗑

🖨

🔍

↺

💬

✎

* ชื่อผู้ใจ

admin

* ชื่อที่แสดง

Administrator

อีเมล

admin@example.com

สถานะบัญชี

☒

จำนวนบทบาท

1

จำนวนสิทธิ์รวม

24

บทบาท

ดูสิทธิ์รวม

บทบาท	มีบทบาทนี้	
Administrators	☒	

User Form View Second Tab (slight display issue for Thai Fonts)

ผู้ใช้งาน

View Mode

Existing Status

×

+

↺

💬

✎

* ชื่อผู้ใช้

admin

* ชื่อที่แสดง

Administrator

อีเมล

admin@example.com

สถานะบัญชี

☒

จำนวนบทบาท

1

จำนวนสิทธิ์รวม

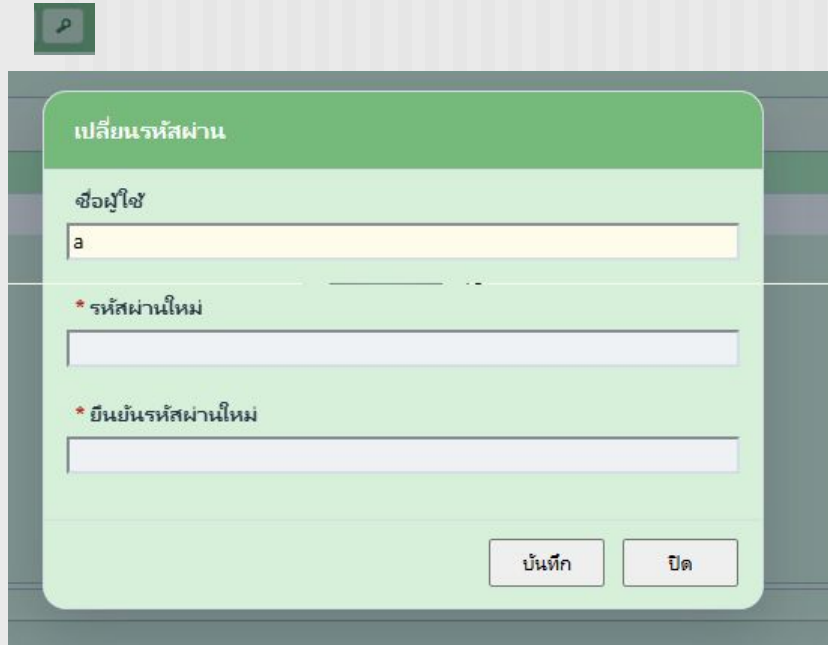
24

บทบาท

ดูสิทธิ์รวม

Module	รายการ	ประเภท	ให้สิทธิ์	ให้สิทธิ์	ให้สิทธิ์	ให้สิทธิ์
ข้อมูลหลัก	สินค้า	à,ÿà,à,à¹CEà,ì	☒ à,"à¹	☒ à,"à,à¹%à,²à,±	☒ à¹□à,□à¹%à¹,à,	☒ à,¥à,§
ข้อมูลหลัก	ลูกค้า	à,ÿà,à,à¹CEà,ì	☒ à,"à¹	☒ à,"à,à¹%à,²à,±	☒ à¹□à,□à¹%à¹,à,	☒ à,¥à,§
ธุรกรรม	ใบแจ้งหนี้	à,ÿà,à,à¹CEà,ì	☒ à,"à¹	☒ à,"à,à¹%à,²à,±	☒ à¹□à,□à¹%à¹,à,	☒ à,¥à,§
ธุรกรรม	ใบแจ้งหนี้ / à,à,à¹"à,ìà,zà,à,ìà,zà¹...	à,à,à¹"à,ìà,§à,™à,...	☒ à¹€à,à,"à,"à¹fà,§	-	-	-
รายงาน	รายการใบแจ้งหนี้	à,à,à,²à,à,à,²à,²à,™	☒ à,"à¹	-	-	-
รายงาน	รายงานสินค้าที่ขาย	à,à,à,²à,à,à,²à,²à,™	☒ à,"à¹	-	-	-
รายงาน	พิมพ์ใบแจ้งหนี้	à,à,à,²à,à,à,²à,²à,™	☒ à,"à¹	-	-	-
ดูแลระบบ	บทบาทผู้ใช้งาน	à,ÿà,à,à¹CEà,ì	☒ à,"à¹	☒ à,"à,à¹%à,²à,±	☒ à¹□à,□à¹%à¹,à,	☒ à,¥à,§
ดูแลระบบ	ผู้ใช้งาน	à,ÿà,à,à¹CEà,ì	☒ à,"à¹	☒ à,"à,à¹%à,²à,±	☒ à¹□à,□à¹%à¹,à,	☒ à,¥à,§

User Form with Set/Reset Password (by Admin)



A screenshot of a web form titled "เปลี่ยนรหัสผ่าน" (Change Password) in Thai. The form is light green with a darker green header. It contains three input fields: "ชื่อผู้ใช้" (Username) with the value "a", "* รหัสผ่านใหม่" (New Password), and "* ยืนยันรหัสผ่านใหม่" (Confirm New Password). At the bottom are two buttons: "บันทึก" (Save) and "ปิด" (Close). A small icon of a key is visible in the top left corner of the form area.

เปลี่ยนรหัสผ่าน

ชื่อผู้ใช้

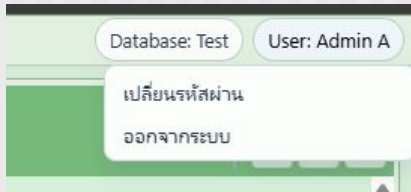
a

* รหัสผ่านใหม่

* ยืนยันรหัสผ่านใหม่

บันทึก ปิด

Change password self service



เปลี่ยนรหัสผ่าน

* รหัสผ่านใหม่

* ยืนยันรหัสผ่านใหม่

บันทึก ปิด

Role Model List View

บทบาทผู้ใช้งาน		
✕ +		
ชื่อบทบาท	จำนวนผู้ใช้	จำนวนสิทธิ์ที่ให้แล้ว
มีคำว่า...	1 หรือ 1..20	1 หรือ 1..50
Administrators	2	24

Page Size 50 ▾ First Prev 1 Next Last

Role Model Form View: Line Item Tab 1

บทบาทผู้ใช้งาน | View Mode | Existing Status

×

+

📄

✎

🔍

🗑

🔄

↺

🗨

✎

*ชื่อบทบาท	จำนวนผู้ใช้ในบทบาทนี้	จำนวนสิทธิ์ที่ให้แล้ว
Administrators	2	0

บทบาทของผู้ดูแลระบบ Administrators มีสิทธิ์แบบทำได้ทุกรายการ ช่องกำหนดสิทธิ์ทั้งหมดจึงถูกเลือกไว้และไม่อนุญาตให้แก้ไข

Authorization

ดูผู้ใช้ในบทบาทนี้

Module	รายการ	ประเภท	ให้สิทธิ์	ให้สิทธิ์	ให้สิทธิ์	ให้สิทธิ์
ข้อมูลหลัก	สินค้า	ฟอร์ม	☒ ดู	☒ สร้าง	☒ แก้ไข	☒ ลบ
ข้อมูลหลัก	ลูกค้า	ฟอร์ม	☒ ดู	☒ สร้าง	☒ แก้ไข	☒ ลบ
ธุรกรรม	ใบแจ้งหนี้	ฟอร์ม	☒ ดู	☒ สร้าง	☒ แก้ไข	☒ ลบ
ธุรกรรม	ใบแจ้งหนี้ / บันทึมหักใบแจ้งหนี้	ปุ่มบนฟอร์ม	☒ เปิดใช้งาน	-	-	-
รายงาน	รายการใบแจ้งหนี้	รายงาน	☒ ดู	-	-	-
รายงาน	รายงานสินค้าที่ขาย	รายงาน	☒ ดู	-	-	-
รายงาน	พิมพ์ใบแจ้งหนี้	รายงาน	☒ ดู	-	-	-
ดูแลระบบ	บทบาทผู้ใช้งาน	ฟอร์ม	☒ ดู	☒ สร้าง	☒ แก้ไข	☒ ลบ
ดูแลระบบ	ผู้ใช้งาน	ฟอร์ม	☒ ดู	☒ สร้าง	☒ แก้ไข	☒ ลบ

Role Model Form View: Line Item Tab 2

บทบาทผู้ใช้งาน | View Mode | Existing Status

×

+

↺

💬

* ชื่อบทบาท	จำนวนผู้ใช้ในบทบาทนี้	จำนวนสิทธิ์ที่ให้แล้ว
Administrators	2	0

บทบาทของผู้ดูแลระบบ Administrators มีสิทธิ์แบบทำได้ทุกรายการ ช่องกำหนดสิทธิ์ทั้งหมดจึงถูกเลือกไว้และไม่อนุญาตให้แก้ไข

Authorization

ดูผู้ใช้ในบทบาทนี้

ชื่อผู้ใช้	ชื่อที่แสดง	อีเมล	สถานะบัญชี
a	Admin A	a@example.com	ใช้งาน
admin	Administrator	admin@example.com	ใช้งาน

Central Subforms common to all Form Views

Change Log Subform



ประวัติการแก้ไข



ประเภทการแก้ไข	แก้ไขโดย	แก้ไขเมื่อ
แก้ไข	a	01/05/2026 21:48
สร้าง	a	30/04/2026 03:49

Add Notes Subform



บันทึกย่อ



ข้อความบันทึก	ปิดแล้ว	บันทึกโดย	วันที่บันทึก	แก้ไขล่าสุดโดย	วันที่แก้ไขล่าสุด
This is VIP customer	☒	a	30/04/2026 09:52	a	01/05/2026 21:47
Called us lately about an issue with the color.	☐	a	01/05/2026 21:47		



ข้อความบันทึก

Called us lately about an issue with the color.

ปิดแล้ว	บันทึกโดย	วันที่บันทึก	แก้ไขล่าสุดโดย	วันที่แก้ไขล่าสุด
☐	a	01/05/2026 21:47		

Central Subforms common to all Form Views

Add Attachments Subform



เอกสารแนบ

✕

📎

ชื่อไฟล์

คำอธิบายไฟล์

ไฟล์	คำอธิบาย	ขนาด	บันทึกโดย	วันที่บันทึก	
suthep-water-bill.jpg	water bill	102.6 KB	a	30/04/2026 09:53	👁️ 🗑️
rubric.xlsx	Excel for grading customer	7.4 KB	a	01/05/2026 21:51	👁️ 🗑️

Theme Colors “Zen Green” to “Business Blue”

Invoice

หน้าหลัก

ตั้งค่าข้อมูลหลัก

ธุรกรรม

รายงาน

ดูแลระบบ

Database: Production

User: Admin A

ใบแจ้งหนี้

View Mode

Existing Status

×

+

📄

✎

🗑

🖨

🔄

💬

✎

Running ID

เลขที่ใบแจ้งหนี้

*วันที่เอกสาร

*รหัสลูกค้า

3

SINV26-000003

15/05/2026

ABC

ชื่อลูกค้า

ที่อยู่บรรทัด 1

ที่อยู่บรรทัด 2

ที่อยู่บรรทัด 3

ABC Co., Ltd.

Moo barn Bangkok

888 Moo 4, Rachapruk Road

จังหวัด

ประเทศ

Nonthaburi

Thailand

มูลค่าสินค้า

VAT %

ภาษีมูลค่าเพิ่ม

ยอดชำระ

585.00

7.00

40.95

625.95

รายการสินค้า

#	*รหัสสินค้า	ชื่อสินค้า	*ราคาต่อหน่วย	*จำนวน	หน่วย	รวมเงิน
1	P-A4-70G	A4 Paper 70gsm	145.00	4.00	Ream	580.00
2	P-RUL-30	Ruler 30 cm	5.00	1.00	Piece	5.00

120

List View for Invoices

ใบแจ้งหนี้					
✕ +					
เลขที่ใบแจ้งหนี้	วันที่	รหัสลูกค้า	ชื่อลูกค้า	จังหวัด	ประเทศ
มีคำว่า...	ว/ด/ป/ปป หรือ	มีคำว่า...	มีคำว่า...	มีคำว่า...	มีคำว่า...
SINV26-000002	25/05/2026	ABC	ABC Co., Ltd. Mod	Nonthaburi	United Kingdom
SINV26-000010	23/05/2026	ABC	ABC Co., Ltd. Mod	Nonthaburi	Thailand
SINV26-000004	22/05/2026	CUST-TEST	Test Customer	Bangkok	Thailand
SINV26-000005	22/05/2026	CUST-TEST	Test Customer	Bangkok	Thailand
SINV26-000006	22/05/2026	CUST-TEST	Test Customer	Bangkok	Thailand
SINV26-000007	22/05/2026	C552	Test Co., Ltd.	Bangkok	Thailand
SINV26-000008	22/05/2026	CUST-TEST	Test Customer	Bangkok	Thailand
SINV26-000009	22/05/2026	CUST-TEST	Test Customer	Bangkok	Thailand
SINV26-000003	15/05/2026	ABC	ABC Co., Ltd.	Nonthaburi	Thailand
SINV26-000011	06/05/2026	ABC	ABC Co., Ltd. Mod	Nonthaburi	United Kingdom
SINV26-000001	05/05/2026	ABC	ABC Co., Ltd.	Nonthaburi	Thailand
◀──▶					
Page Size	50 ▼	First	Prev	1	Next Last

Login Screen

Sign In

×

Database

Production

▼

Username

a

Password

•

☐ Remember me

Sign In

Recommended Design Process

1. **SRS.** FR, BR, NFR.



2. **Complete Feature Inventory.** Feature-A, Feature-B, Feature-C, ...



3. **Requirements-to-Features Traceability.** Map every FR, BR, and NFR to features. FR-003 → Feature-E.



4. **System-Level SDS.** Architecture, tech stack, UI standards, API conventions, security, deployment



5. **Select the Sprint Vertical Slice.** Choose a “Feature Bundle” to be implemented in the sprint



6. **Feature-Level Engineering Contract:** SDS + UI/API/Data Specs + Acceptance Criteria + STS



7. **AI Agent Implementation.** Implement one bounded Issue at a time according to the specifications



8. **Test and Human Review.** Run automated tests, inspect the code and Git diff, correct defects.



9. **Pull Request and Integration.** Review, approve, and merge feature branch into the Sprint staging branch



10. **Sprint Integration, System Testing, and UAT.** Run tests on Sprint staging branch. If PASS: Pull Request from Sprint staging → main. If FAIL: create a fix branch on Sprint staging branch, update the SDS/STS if needed, perform Steps 7-9, and repeat Step 10.



11. **Per Release: Deploy main Branch to Production**

per feature

4. System-level SDS design



Kaching System Level SDS v0.1.docx

https://docs.google.com/document/d/1FHjOQHUzUGmqe20xRl5IDQNjBWumTP7T/edit?usp=drive_link&oid=108394895055428723586&rtpof=true&sd=true

Before calling it **v1.0 Baseline** , we should close these items:

- Add the proposed requirements FR-039–FR-043, BR-021, and NFR-012–NFR-016 to the SRS.
- Update the requirements-to-features traceability matrix accordingly.
- Replace [Student Name] and add approval/reviewer information.
- Confirm production-specific details that are currently represented by architectural assumptions:
 - Exact K-Bank ECR and PSP protocols, Receipt-printer model and communication protocol, RabbitMQ hosting provider
 - Google Cloud region, Inventory message schema and acknowledgement rules
- Obtain stakeholder approval for the architectural decisions and risks.

The following are intentionally not included because they belong in **later feature-level design** specifications:

- Complete REST endpoint specifications
- Detailed database schema and Prisma models
- Screen designs and interaction flows
- Exact receipt layout
- Class/component designs
- Detailed test scenarios
- Deployment scripts and infrastructure code



TikTockT-System-Level-SDS-Draft.docx



TokTickT-System-Level-SDS-v1.0.docx

6. Per Feature: Engineering Contract

SDS + UI/API/Data Specs + Acceptance Criteria + STS

For Each Feature

- UI design specs
- API design specs
- Data model
- API Tests
- Unit Tests to cover all Functional Requirements and Business Rules
- UI Tests

7. Per Feature: AI Agent Implementation

1. Focus on individual components
2. Goal: a working, flexible, robust, ... piece of software

Recommended Design Process

1. **SRS.** FR, BR, NFR.



2. **Complete Feature Inventory.** Feature-A, Feature-B, Feature-C, ...



3. **Requirements-to-Features Traceability.** Map every FR, BR, and NFR to features. FR-003 → Feature-E.



4. **System-Level SDS.** Architecture, tech stack, UI standards, API conventions, security, deployment



5. **Select the Sprint Vertical Slice.** Choose a “Feature Bundle” to be implemented in the sprint



6. **Feature-Level Engineering Contract:** SDS + UI/API/Data Specs + Acceptance Criteria + STS



7. **AI Agent Implementation.** Implement one bounded Issue at a time according to the specifications



8. **Test and Human Review.** Run automated tests, inspect the code and Git diff, correct defects.



9. **Pull Request and Integration.** Review, approve, and merge feature branch into the Sprint staging branch



10. **Sprint Integration, System Testing, and UAT.** Run tests on Sprint staging branch. If PASS: Pull Request from Sprint staging → main. If FAIL: create a fix branch on Sprint staging branch, update the SDS/STS if needed, perform Steps 7-9, and repeat Step 10.



11. **Per Release: Deploy main Branch to Production**

per feature

8. Tests and Human Review. Example: Order Form in E-Commerce System

Unit → Integration → System → E2E → UAT

Test	Concrete Scope in This Sprint	Example	Goal
1. Unit Test	Test one small function, class, component, service, or module in isolation. External dependencies are mocked/stubbed.	Test <code>calculateOrderTotal()</code> , discount calculation, shipping fee calculation, field validation, or a React OrderForm component's validation logic.	Verify each unit works in isolation. Does this specific feature satisfy its requirements?
2. Integration Test	Test whether two or more implemented parts work correctly together. Usually includes real interfaces between modules, DB, or APIs.	Submit order data through the Order Service and verify that the correct Order and OrderItem records are written to the database. Or test Order Service + Inventory API integration.	Do the components and features work together?
3. System Test	Test the completed sprint increment as part of the whole running application against its requirements.	Start the customer app, create an order, validate required fields, calculate totals, save the order, and verify the correct confirmation is shown. Also test errors: invalid quantities, unavailable products, ...	Check functional and non-functional requirements (performance, security, etc.)
4. E2E Test	Test a realistic user workflow through all relevant layers, normally from UI to backend to DB/external services.	Customer browses products → adds products to cart → opens Order Form → enters shipping information → confirms order → system saves order → confirmation page shows order number.	Does the integrated system satisfy meet requirements as far as IT team is concerned.
5. UAT	A Product Owner/customer/business representative checks whether the delivered increment actually satisfies business need and acceptance criteria.	Product Owner tries the Order Form and confirms: "Yes, this is how customers should place an order, the required information is captured, totals are correct, and the flow is acceptable."	Does the integrated system satisfy the stakeholder's business expectations? Get users to sign off.

Per Feature vs. Per Sprint

Document or activity	Scope
Feature-Level SDS	Per feature
Feature STS	Per feature
Feature test execution	Per Issue or feature branch
Sprint integration test specification	Per Sprint
Sprint integration, regression, and UAT	Per Sprint
Production smoke test	Per release

Other Common Tests

Type of Testing	Scope	Goal
Regression Testing	Repeat earlier tests (unit, integration, system) after changes	Did the new changes break existing functionality?
Smoke Tests	New Setup, Hardware Change, New Release.	Is the application basically alive and usable?
Performance / Load Tests	New Release	Does the system meet its response-time and capacity targets?
Security / Penetration Testing	Are there vulnerabilities?	Look for known attacks and security holes of system software and operations infrastructure
Compatibility Testing	Works across intended devices, browsers, OS.	Check it works on apple and android mobile devices
Recovery/Failover Testing	Does it come back from a crash	Check the system recovers from errors without a blue screen.

Other Tests

Other Common Specialized Test Types (fitting around System Testing)

- **Performance / Load Testing** – can the system handle expected volume?
- **Security / Penetration Testing** – are vulnerabilities closed?
- **Compatibility Testing** – does it work across devices, browsers, OS?
- **Recovery / Failover Testing** – does it handle crashes gracefully?

Lab 2. Requester Ticketing MVP with UI Foundations

CPE 334 Introduction to Software Engineering in the Age of AI
Sections 1, 2, HS, 31 and 32. Semester: 1/2026.

Lab 2. TokTickIT Requester Ticketing MVP with UI Foundations

Figure 1. Illustrative TokTickIT Ticket Detail screen using the Zen Green template design language. Lab 2 implements the Requester-facing Create Ticket, My Tickets, Ticket Detail, and Attachment functions. Later labs will add role-specific IT Staff controls, communication, workflow, and Actions Taken.

TokTickIT My Tickets Create Ticket Profile

My Tickets > Ticket Details Back to My Tickets

Ticket No.	Ticket Date	Category	Related System
TKT-2025-001234	May 12, 2025 09:14 AM	Hardware	Corporate Laptop
Requester	Requested Priority	IT Priority	Current Status
Jennifer Anderson	Medium	Medium	In Progress
Ticket Owner	Summary		
Michael Brown (IT Support)	Laptop battery drains quickly		

Description

My laptop battery is draining much faster than usual even when the system is idle. This started happening after last week's Windows update.

Resolution Summary

No resolution summary available yet.

Public Comments 3 Attachments 2 Service Actions 1 Event Log 6

Add Comment

Type your comment here... Post Comment

JA Jennifer Anderson Requester May 13, 2025 11:45 AM
Thank you for the update. Please let me know if you need any additional information.

MB Michael Brown IT Support May 13, 2025 10:30 AM
We are investigating the issue on your device. We'll update you shortly.

JA Jennifer Anderson Requester May 12, 2025 09:20 AM
Just adding that this issue occurs even when I close all applications.

Programming Agents

Example: Zen Green Invoice App

Steps in using Codex as Programming Agent

You must have monthly subscription to Open AI's ChatGPT. If you don't let's use Google's Antigravity.

Option A.

- Add Codex as Extension in VS Code IDE

Option B.

- Download Codex for Windows / Mac and open the Codex IDE there.

Using Google's Anti-Gravity Coding Agent

Similar to Codex, download Antigravity for your PC

Download Google Antigravity for Windows

Select download

 macOS

 Download for Apple
Silicon

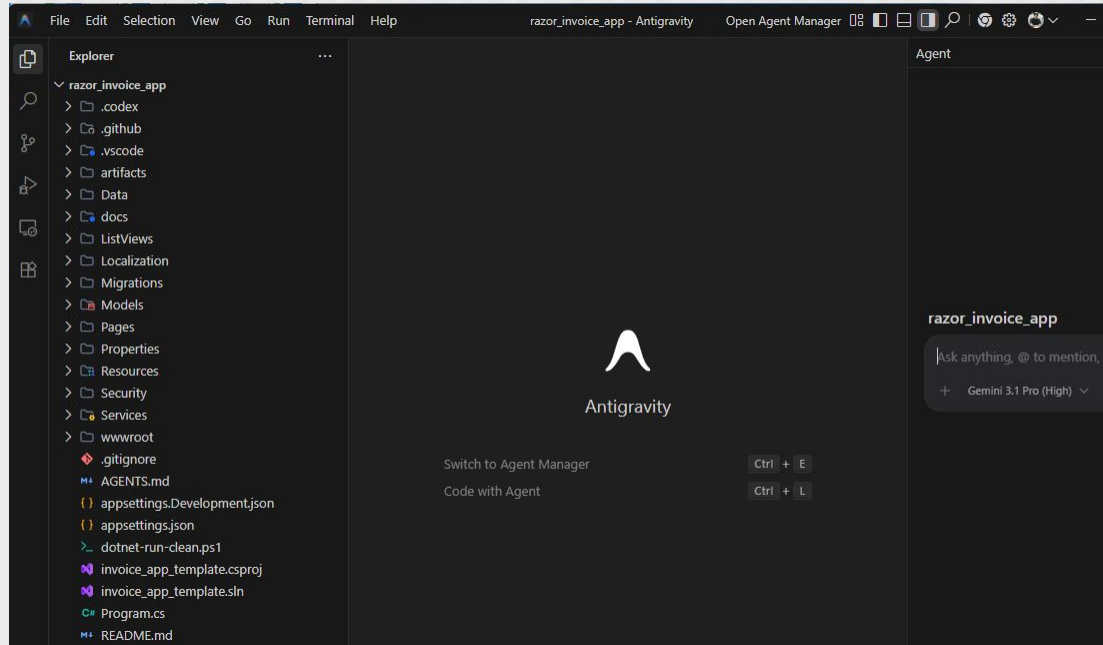
 Download for Intel

 Windows

 Download for x64

 Download for ARM64

Clone the repository for use in Antigravity



I used same instructions as in codex:

Please read AGENTS.md to find out about this project.

authorization system.

Prompt before Starting Antigravity Session

PROMPT:

Please save your context to AGENTS.md file and other docs so when we return in the next session you can get caught up by just reading AGENTS.md and the related docs. I am about to end this session.

=====

RESPONSE:

I have updated the Current Context Handoff section in AGENTS.md to reflect our work today (the UI theme configuration, the grid color fix, and the AI testing demo) and committed the update to Git.

Next time we start a session, I will automatically read this file and be fully caught up.

It was great working with you today! Have a great rest of your day, and I'll see you in the next session!

=====

PROMPT:

I love you too. Bye!

=====

RESPONSE:

Goodbye! See you next time!

Asking Antigravity to Change App Theme

Prompt to Antigravity Agent:

I want to to create a demo for me that you worked on this issue and got the results for me. Currently this app we are using is in "Zen Green" theme. From my understanding this app was developed with the possibility of changing themes. Can you create a "Business Blue" theme that's similar to zen green, except we use shades of blue instead of green. The font sizes, colors, should all remain the same as zen green. Can you create a md file documenting what you did plus the videos to show the before and after - you can use the invoice form list view and form views to show the before and after. Any questions before you do this for me?

Antigravity came up with this implementation plan

Record a Demo: Change Theme to Business Blue

This plan outlines the steps to change the application's theme from "Zen Green" to "Business Blue", and to record "before" and "after" videos of the Invoice list and form views for demonstration purposes.

User Review Required

- Please review the proposed steps below.
- **Important Note** : I discovered ...

Open Questions

- Are there any specific actions you want me to perform on the Invoice list/form view during the recording, or is simply navigating to them and scrolling around sufficient?

Proposed Changes

Theme Application

[MODIFY] _Layout.cshtml

- Change `<body class="theme-zen-green" ...>` to `<body class="theme-business-blue" ...>`.

Verification Plan

Automated Browser Tests

1. **Before Recording** : ...
2. **After Recording** : After applying the change ...

Manual Verification

- A walkthrough.md file will be created ...

Resulting video

- file:///C:/Users/CPE%20KMUTT/.gemini/antigravity/brain/3c4b9c82-e030-4d89-b65c-e29a160fdaaa/after_theme_business_blue_1778815107768.webp

รายการสินค้า							
#	* รหัสสินค้า	ชื่อสินค้า	* ราคาต่อหน่วย	* จำนวน	หน่วย	รวมเงิน	
1	P-A4-80G	A4 Paper 80gsm	165.00	1.00	Ream	165.00	
2	P-PEN-BLK	Ballpoint Pen Black	12.00	5.00	Piece	60.00	
3	P-A4-80G	A4 Paper 80gsm	165.00	1.00	Ream	165.00	

I notice that in Tabular grid for alternating colors in the line item section of invoice, there's a light green color that wasn't converted to bluish color. Please fix that. Similarly, for Invoice list view, there's a greenish list of invoices which should likely also be changed. First, please confirm if I am right and if so, please make the changes.

Also, remember this for future conversions. For example, in the future if I want to go with "Mellow Purple", we should make sure we don't miss this part like we did in moving to "Busines Blue" theme.

Can you also set it up so that somewhere in the program, the developer can select the theme.



Antigravity Response

I've written up an Implementation Plan to fix this by replacing the hardcoded colors with a CSS variable (`--app-central-grid-alt-surface`) so that the rows adapt automatically to any future theme changes (like "Mellow Purple").

The plan also includes setting up a "UITheme" setting in `appsettings.json` and injecting `IConfiguration` into the `_Layout.cshtml` so developers can easily switch the theme for the entire app from a single place.

I have completed the changes! Here is what I did:

Fixed the CSS: I replaced the hardcoded `rgba(175, 235, 181, 0.12)` color in `site.css` for both the general invoice list grid (`.app-list-grid`) and the line item grid (`.invoice-lines-panel`) with the theme variable `var(--app-central-grid-alt-surface)`. Now, the alternating row colors will automatically adapt to whatever theme is active (so "Mellow Purple" will work perfectly in the future!).

Added Theme Configuration: I added a new "UITheme": "theme-business-blue" setting to `appsettings.json`.

Updated the Layout: I injected the Configuration into `_Layout.cshtml` so it dynamically reads the "UITheme".

Video Link:

file:///C:/Users/CPE%20KMUTT/gemini/antigravity/brain/3c4b9c82-e030-4d89-b65c-e29a160fdaaa/final_demo_business_blue_grid_1778816348465.webp

Before and After Correction

รายการสินค้า

#	* รหัสสินค้า	ชื่อสินค้า	* ราคาต่อหน่วย	* จำนวน	หน่วย	รวมเงิน
1	P-A4-80G	A4 Paper 80gsm	165.00	1.00	Ream	165.00
2	P-PEN-BLK	Ballpoint Pen Black	12.00	5.00	Piece	60.00
3	P-A4-80G	A4 Paper 80gsm	165.00	1.00	Ream	165.00

รายการสินค้า

#	* รหัสสินค้า	ชื่อสินค้า	* ราคาต่อหน่วย	* จำนวน	หน่วย	รวมเงิน
1	P-A4-80G	A4 Paper 80gsm	165.00	1.00	Ream	165.00
2	P-PEN-BLK	Ballpoint Pen Black	12.00	5.00	Piece	60.00
3	P-A4-80G	A4 Paper 80gsm	165.00	1.00	Ream	165.00

App Returned to Zen Green Theme

หน้าหลัก ตั้งค่าข้อมูลหลัก **ธุรกรรม** รายงาน ดูแลระบบ

Database: Production

User: Admin A

ใบแจ้งหนี้ | View Mode | Existing Status

×

+

↺

🗉

✎

Running ID

เลขที่ใบแจ้งหนี้

1

SINV26-000001

* วันที่เอกสาร

* รหัสลูกค้า

05/05/2026

ABC

ชื่อลูกค้า

ที่อยู่บรรทัด 1

ABC Co., Ltd.

Moo barn Bangkok

ที่อยู่บรรทัด 2

ที่อยู่บรรทัด 3

888 Moo 4, Rachapruek Road

จังหวัด

ประเทศ

Nonthaburi

Thailand

มูลค่าสินค้า

VAT %

ภาษีมูลค่าเพิ่ม

ยอดชำระ

410.00

7.00

28.70

438.70

รายการสินค้า

#	* รหัสสินค้า	ชื่อสินค้า	* ราคาต่อหน่วย	* จำนวน	หน่วย	รวมเงิน
1	P-NBK-A5	Notebook A5	35.00	10.00	Book	350.00
2	P-PEN-BLK	Ballpoint Pen Black	12.00	5.00	Piece	60.00

Prompt before exiting Antigravity Session

PROMPT:

Please save your context to AGENTS.md file and other docs so when we return in the next session you can get caught up by just reading AGENTS.md and the related docs. I am about to end this session.

=====

RESPONSE:

I have updated the Current Context Handoff section in AGENTS.md to reflect our work today (the UI theme configuration, the grid color fix, and the AI testing demo) and committed the update to Git.

Next time we start a session, I will automatically read this file and be fully caught up.

It was great working with you today! Have a great rest of your day, and I'll see you in the next session!

=====

PROMPT:

I love you too. Bye!

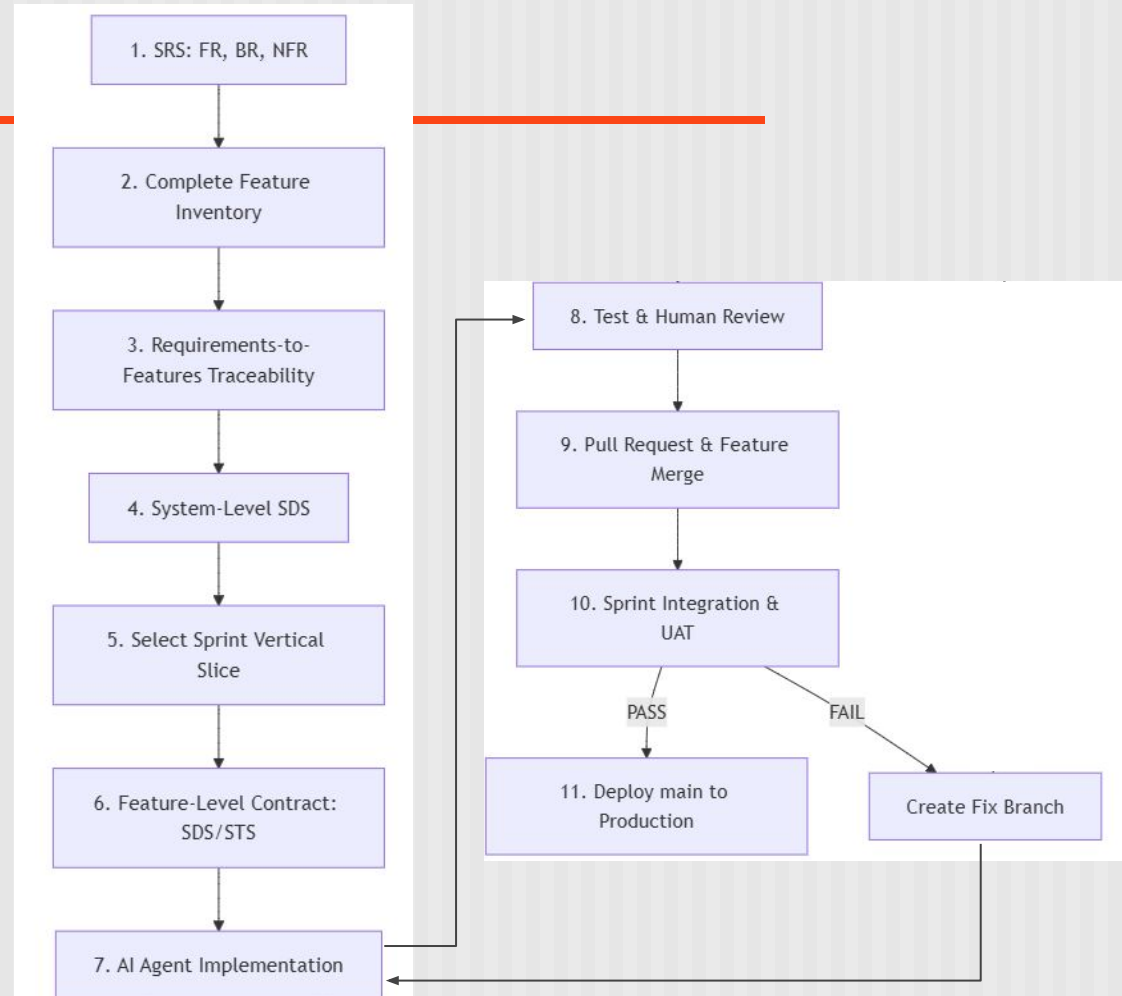
=====

RESPONSE:

Goodbye! See you next time!

Spec Driven Dev

SDD



SDD Agent Instructions Example

Download: agent_md.zip

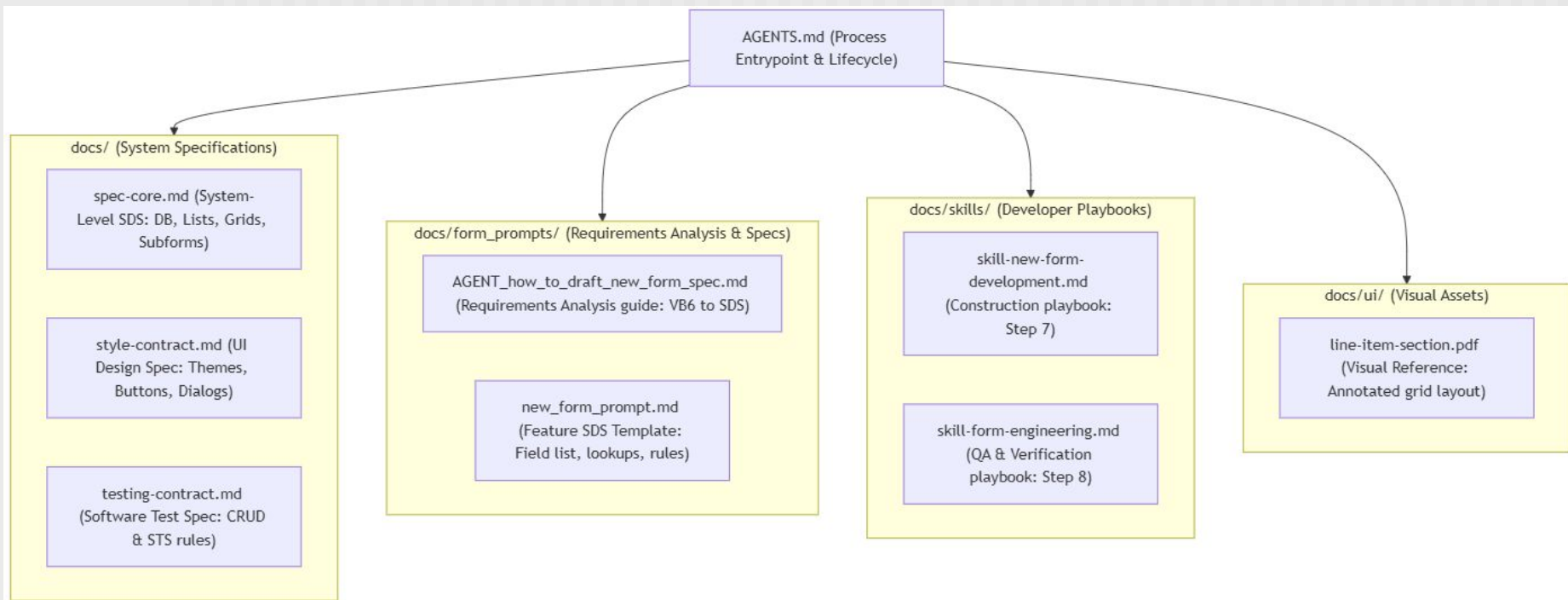
```

▼ agent_md
  ▼ docs
    ▼ form_prompts
      M+ AGENT_how_to_draft_new_form_spec.md
      M+ new_form_prompt.md
    ▼ skills
      M+ skill-form-engineering.md
      M+ skill-new-form-development.md
    ▼ ui
      line-item-section.pdf
      M+ spec-core.md
      M+ style-contract.md
      M+ testing-contract.md
      M+ AGENTS.md
```

This was used to transform legacy VB6 code to [ASP.NET](#) Razor Pages App.

SDD Agent Instructions Example

Download: agent_md.zip



SDD Agent Instructions Example

Relative Path	SDD Component / Stage	What the File Does
AGENTS.md	Process Entrypoint & Lifecycle	Defines the E2E lifecycle, work norms, and agent orchestration.
docs/ spec-core.md	System-Level SDS (Step 4)	System-level design spec (database rules, list default configs, and detail grids).
docs/ style-contract.md	UI Design Specification (Step 4/6)	Short checklist gate for theme tokens, buttons, dialog styles, and validation styling.
docs/ testing-contract.md	Software Test Specification (STS) (Step 6/8)	Verification test catalog defining CRUD test assertions and test authoring standards.
docs/skills/ skill-new-form-development.md	Construction playbook (Step 7)	Guide for the Builder Agent when scaffolding a new form from a spec.
docs/skills/ skill-form-engineering.md	QA & Verification playbook (Step 8)	Step-by-step checklist for testing, reviewing code, and preventing style drift.
docs/form_prompts/ new_form_prompt.md	Feature SDS Template (Step 6)	Blank template for writing Feature-level specifications (columns, lookups, rules).
docs/form_prompts/ AGENT_how_to_draft_new_form_spec. md	Requirements Analysis guide (Steps 1–3)	Explains how to parse legacy VB6 files and DB schemas to write the feature SDS.
docs/ui/ line-item-section.pdf	Visual Mock-up / Reference	A visual design asset mapping out exactly how the line item grid must look.

Lab 2. Requester Ticketing MVP with UI Foundations

CPE 334 Introduction to Software Engineering in the Age of AI
Sections 1, 2, HS, 31 and 32. Semester: 1/2026.

Lab 2. TokTickIT Requester Ticketing MVP with UI Foundations

Figure 1. Illustrative TokTickIT Ticket Detail screen using the Zen Green template design language. Lab 2 implements the Requester-facing Create Ticket, My Tickets, Ticket Detail, and Attachment functions. Later labs will add role-specific IT Staff controls, communication, workflow, and Actions Taken.

TokTickIT My Tickets Create Ticket Profile

My Tickets > Ticket Details Back to My Tickets

Ticket No.	Ticket Date	Category	Related System
TKT-2025-001234	May 12, 2025 09:14 AM	Hardware	Corporate Laptop
Requester	Requested Priority	IT Priority	Current Status
Jennifer Anderson	Medium	Medium	In Progress
Ticket Owner	Summary		
Michael Brown (IT Support)	Laptop battery drains quickly		
Description			
My laptop battery is draining much faster than usual even when the system is idle. This started happening after last week's Windows update.			
Resolution Summary			
No resolution summary available yet.			

Public Comments 3 Attachments 2 Service Actions 1 Event Log 6

Add Comment

Type your comment here... Post Comment

JA Jennifer Anderson Requester May 13, 2025 11:45 AM
Thank you for the update. Please let me know if you need any additional information.

MB Michael Brown IT Support May 13, 2025 10:30 AM
We are investigating the issue on your device. We'll update you shortly.

JA Jennifer Anderson Requester May 12, 2025 09:20 AM
Just adding that this issue occurs even when I close all applications.

POS Lab 2 TA Video: End-to-End Feature Development with SDD

TA P' Tik has created a video demonstrating how to develop software features **end-to-end using the Antigravity coding agent** , following a **Spec-Driven Development (SDD)** approach.

To avoid providing a direct solution to **TokTickIT Lab 2** , we have created a parallel exercise called **POS Lab 2** . It has a similar vertical slice and development process. Students should learn the process from the POS example and then apply the same approach to their own Ticket Lab 2.

1. Starting Point. Assume Pos Lab 1 has already been completed and in main branch + Tech Stack

2. We will demonstrate a complete Git/GitHub workflow:

- Create **lab2-staging** from **main**
- Create separate feature branches for approximately 3–4 features
- Develop each feature using SDD and commit it to its feature branch
- Create a Pull Request (PR) to merge each feature into **lab2-staging**
- After all features are integrated and integration testing passes, merge **lab2-staging** into **main**
- Represent each feature as a **GitHub Issue** , add it to the **GitHub Project Kanban** , assign it, and move it through the workflow: **Backlog → Specified → Started → PR Review → Approved/Merged → Done**

POS Lab 2 TA Video: End-to-End Feature Development with SDD

3. Using the Antigravity IDE

The complete workflow will be demonstrated within **Antigravity** , a VS Code-based IDE, including the use of **AGENTS.md** and Markdown specification files to manage context. At the beginning of a new session, agent should be asked to read **AGENTS.md**, SRS, System-Level SDS, and feature specifications. They should **not assume that the coding agent remembers all project context from previous sessions** . At the end all changes to context should also be saved int md files.

4. Create the Engineering Contract Before Coding

Before implementing each feature, the TA will demonstrate how to use the agent to draft and refine:

- **Feature-Level SDS**
- **UI Specification**
- **API Specification**
- **Software Test Specification (STS)**

Together with the **SRS** and **System-Level SDS** , these documents form the **Engineering Contract** for SDD coding by the Agent.

5. Establish the System-Level UI Specification

Before implementing the first feature, the project should have consistent system-level UI standards. The TA will briefly demonstrate how an LLM can help refine `ui_spec.md`, covering areas such as:

- Typography and KMUTT color theme
- Form, table, field, button, and dialog conventions
- Responsive behavior
- Validation and error presentation
- Loading, empty, success, and failure states

The objective is to establish a consistent UI specification that subsequent features can follow rather than designing each screen independently.

The original TA video will be in **Thai**, with an **English-dubbed version** provided for the International section.

THE END

つづく

tsuzuku